



ΑΡΙΣΤΟΤΕΛΕΙΟ ΠΑΝΕΠΙΣΤΗΜΙΟ ΘΕΣΣΑΛΟΝΙΚΗΣ

ΣΧΟΛΗ ΘΕΤΙΚΩΝ ΕΠΙΣΤΗΜΩΝ

ΤΜΗΜΑ ΠΛΗΡΟΦΟΡΙΚΗΣ

ΠΤΥΧΙΑΚΗ ΕΡΓΑΣΙΑ

ΑΛΓΟΡΙΘΜΟΙ ΔΙΚΑΙΗΣ ΚΑΤΑΝΟΜΗΣ ΣΕ ΓΡΑΦΟΥΣ

Algorithms for fair allocation in graphs

Σπουδαστής

Κατσάρας Ευάγγελος

ΑΕΜ: 3634

Τμήμα Πληροφορικής ΑΠΘ

Επιβλέπων Καθηγητής

Χριστοδούλου Γεώργιος

Αναπληρωτής Καθηγητής

Τμήμα Πληροφορικής ΑΠΘ

ΘΕΣΣΑΛΟΝΙΚΗ 2025

ΔΗΛΩΣΗ ΑΚΑΔΗΜΑΪΚΗΣ ΑΚΕΡΑΙΟΤΗΤΑΣ

Με πλήρη επίγνωση των συνεπειών του νόμου περί πνευματικών δικαιωμάτων, δηλώνω ρητά ότι η παρούσα πτυχιακή εργασία, καθώς και τα ηλεκτρονικά αρχεία και πηγαίοι κώδικες που αναπτύχθηκαν ή τροποποιήθηκαν στο πλαίσιο αυτής της εργασίας, αποτελεί αποκλειστικά προϊόν προσωπικής μου εργασίας, δεν προσβάλλει κάθε μορφής δικαιώματα διανοητικής ιδιοκτησίας, προσωπικότητας και προσωπικών δεδομένων τρίτων, δεν περιέχει έργα/εισφορές τρίτων για τα οποία απαιτείται άδεια των δημιουργών/δικαιούχων και δεν είναι προϊόν μερικής ή ολικής αντιγραφής, οι πηγές δε που χρησιμοποιήθηκαν περιορίζονται στις βιβλιογραφικές αναφορές και μόνον και πληρούν τους κανόνες της επιστημονικής παράθεσης. Τα σημεία όπου έχω χρησιμοποιήσει ιδέες, κείμενο, αρχεία ή/και πηγές άλλων συγγραφέων, αναφέρονται ευδιάκριτα στο κείμενο με την κατάλληλη παραπομπή και η σχετική αναφορά περιλαμβάνεται στο τμήμα των βιβλιογραφικών αναφορών με πλήρη περιγραφή. Αναλαμβάνω πλήρως, ατομικά και προσωπικά, όλες τις νομικές και διοικητικές συνέπειες που δύναται να προκύψουν στην περίπτωση κατά την οποία αποδειχθεί, διαχρονικά, ότι η εργασία αυτή ή τμήμα της δεν μου ανήκει διότι είναι προϊόν λογοκλοπής.

ΕΥΧΑΡΙΣΤΙΕΣ

Με την παρούσα πτυχιακή εργασία ολοκληρώνεται ο κύκλος σπουδών μου στο Τμήμα Πληροφορικής ΑΠΘ και θα ήθελα να ευχαριστήσω όλους όσους με βοήθησαν και συνέβαλαν στην προσπάθεια μου αυτή.

Πρώτα από όλα, θα ήθελα να ευχαριστήσω τον επιβλέποντα καθηγητή μου, τον Καθηγητή Γ. Χριστοδούλου, για τη στήριξη και τη βοήθεια του. Επιπλέον, η βοήθεια του Συμεών ήταν πολύτιμη, τον οποίο επίσης ευχαριστώ.

Τέλος, θα ήθελα να ευχαριστήσω την οικογένεια μου και τους φίλους μου που με στήριζαν όλα αυτά τα χρόνια.

Με εκτίμηση,
Κατσάρας Ευάγγελος

ΠΕΡΙΕΧΟΜΕΝΑ

ΠΕΡΙΕΧΟΜΕΝΑ.....	4
ΚΑΤΑΛΟΓΟΣ ΣΧΗΜΑΤΩΝ.....	6
ΚΑΤΑΛΟΓΟΣ ΠΙΝΑΚΩΝ.....	7
ΚΑΤΑΛΟΓΟΣ ΕΙΚΟΝΩΝ.....	8
ΠΕΡΙΛΗΨΗ.....	9
ABSTRACT.....	11
ΚΕΦΑΛΑΙΟ 1: Εισαγωγή.....	12
1.1 Προσωπική Συνεισφορά.....	13
1.2 Σχετική Έρευνα.....	13
ΚΕΦΑΛΑΙΟ 2: Μαθηματικό Μοντέλο.....	15
2.1 Βασικοί Ορισμοί.....	15
2.2 Κριτήρια Δικαιοσύνης.....	18
2.3 Προσανατολισμός (Orientation).....	20
ΚΕΦΑΛΑΙΟ 3: Αλγόριθμος και Υλοποίηση.....	22
3.1 Περιγραφή Γενικής Ιδέας.....	22
3.2 Συμβάσεις απεικόνισης γραφήματος.....	24
3.3 Φάση 1: Άπληστη Κατανομή (Greedy Allocation).....	25
3.4 Φάση 2: Μείωση Ζήλειας (Reduce Envy).....	27
3.5 Φάση 3: Ανάθεση μη Κατανεμημένων Αγαθών.....	30
3.6 Τελικός Αλγόριθμος.....	33
ΚΕΦΑΛΑΙΟ 4: Γραφικό Περιβάλλον (GUI) & Υλοποίηση.....	34
4.1 Επισκόπηση Menu Εισόδου.....	34
4.2 Είσοδος δεδομένων.....	35
4.2.1 Manual Text.....	35
4.2.2 Load from File.....	36
4.2.3 Wizard (βήμα-βήμα).....	37
4.3 Παράθυρα Αποτελεσμάτων - Tabs.....	38
4.3.1 Allocation Tables (3 tabs).....	38
4.3.2 Details.....	38
4.3.3 Valuations Tab.....	40
4.3.4 Graph Views (4 tabs).....	41
ΚΕΦΑΛΑΙΟ 5: Πειραματική Ανάλυση.....	44
5.1 Στόχος και ερευνητικά ερωτήματα.....	44
5.2 Πειραματική Διάταξη.....	44
5.2.1 Τοπολογίες και παράμετροι.....	44
5.2.2 Εύρος αποτιμήσεων.....	45

5.2.3 Μετρικές.....	45
5.3 Αποτελέσματα ανά τοπολογία με εύρος αποτίμησης 1–10.....	46
5.3.1 Random Graphs.....	46
5.3.2 Cycles.....	54
5.3.3 Random Trees.....	55
5.4 Συγκριτική ανάλυση: εύρος αποτίμησης 1–10 έναντι 1–100.....	57
5.4.1 Random Graphs (1-10 vs 1-100).....	57
5.4.2 Cycles (1-10 vs 1-100).....	62
5.4.3 Random Trees (1-10 vs 1-100).....	64
5.5 Παρατήρηση για την απουσία unoriented ακμών.....	65
5.6 Συμπεράσματα Πειραματικής Ανάλυσης.....	66
ΚΕΦΑΛΑΙΟ 6: Συμπεράσματα και Μελλοντικές Κατευθύνσεις.....	67
Μελλοντικές κατευθύνσεις και προτάσεις.....	68
Τελική αποτίμηση.....	68
ΒΙΒΛΙΟΓΡΑΦΙΑ.....	69
ΠΑΡΑΡΤΗΜΑ.....	71

ΚΑΤΑΛΟΓΟΣ ΣΧΗΜΑΤΩΝ

Σχήμα 2.1: Γράφος G με $|V| = 4$ και $|E| = 5$.

Σχήμα 2.2: Παράδειγμα κατανομής αγαθών του γράφου G (Σχήμα 2.1).

Σχήμα 2.3: Παράδειγμα EF, EFX και EF1 κατανομής του Σχήμα 2.1.

Σχήμα 2.4: Γράφος G με $|V| = 4$ και $|E| = 6$.

Σχήμα 2.5: EFX κατανομή του Σχήμα 2.4. Δίπλα στους κόμβους αναγράφονται οι ακμές που έχουν κατανεμηθεί συγκεκριμένα σε αυτούς και η αποτίμηση των κόμβων προς τις ακμές.

Σχήμα 3.1: Γράφος G με $|V| = 5$ και $|E| = 10$.

Σχήμα 3.2: Greedy allocation του Σχήμα 3.1.

Σχήμα 3.3: Αποτέλεσμα μετά την εφαρμογή Reduce Envy στο Σχήμα 3.2.

Σχήμα 3.4: Αποτέλεσμα μετά την εφαρμογή Allocate Remaining Goods στο Σχήμα 3.3 και τελικός γράφος EFX κατανομής του Σχήμα 3.1.

Σχήμα 3.5: Αποτέλεσμα της εφαρμογής των αλγορίθμων Greedy Allocation και Reduce Envy στο Σχήμα 2.4

ΚΑΤΑΛΟΓΟΣ ΠΙΝΑΚΩΝ

[Πίνακας 2.1:](#) Τιμές συναρτήσεων αποτίμησης των agents για κάθε αγαθό (Σχήμα 2.1) .

[Πίνακας 2.2:](#) Πίνακας κατανομής και αποτιμήσεις κάθε agent i του Σχήμα 2.2.

[Πίνακας 2.3:](#) Πίνακας κατανομής EF, EFX και EF1 του Σχήμα 2.3.

[Πίνακας 2.4:](#) Πίνακας αποτιμήσεων του Σχήμα 2.4 .

[Πίνακας 2.5:](#) Πίνακας κατανομής του Σχήμα 2.5.

[Πίνακας 3.1:](#) Πίνακας αποτίμησης του Σχήμα 3.1 .

[Πίνακας 3.2:](#) Πίνακας κατανομής του Σχήμα 3.2.

[Πίνακας 3.3:](#) Πίνακας κατανομής του Σχήμα 3.3.

[Πίνακας 3.4:](#) Πίνακας κατανομής του Σχήμα 3.4.

[Πίνακας 3.5:](#) Πίνακας κατανομής του Σχήμα 3.5.

[Πίνακας 5.4.1:](#) Μέσοι χρόνοι εκτέλεσης ανά κατανομή (uniform, center, extremes) και μεταβολές Δ , $\Delta\%$ από [1–10] σε [1–100], από όλα τα n και όλες τις πυκνότητες δ .

[Πίνακας 5.4.2:](#) Μέσος λόγος (σε %) του Our/Optimal ανά κατανομή, από [1–10] σε [1–100], από όλα τα n και όλες τις δ .

[Πίνακας 5.4.3:](#) Τελικό ποσοστό ζήλιας ανά κατανομή, με Δ , $\Delta\%$ από [1–10] σε [1–100], μεσοποιημένα σε όλα τα n και όλες τις δ .

[Πίνακας 5.4.4-5:](#) Χρόνος ανά $\delta \in \{0.2, 0.4, 0.6, 0.8, 1.0\}$ για κάθε κατανομή, με [1–10], [1–100], Δ , $\Delta\%$ και τελική ζήλια ανά δ , με [1–10], [1–100], Δ , $\Delta\%$.

[Πίνακας 5.4.6:](#) Ποιότητα λύσης (OurOverOptimal_%) ανά δ για [1–10] και [1–100], με Δ , $\Delta\%$.

[Πίνακας 5.4.7-8:](#) Χρόνος ανά $n \in \{10, 30, 60, 80, 100\}$ για κάθε κατανομή, με [1–10], [1–100], Δ , $\Delta\%$ και τελική ζήλια ανά n , με [1–10], [1–100], Δ , $\Delta\%$.

[Πίνακας 5.4.9:](#) Ποιότητα λύσης (OurOverOptimal_%) ανά n για [1–10] και [1–100], με Δ , $\Delta\%$.

[Πίνακας 5.4.10:](#) Μέσοι χρόνοι εκτέλεσης ανά κατανομή (uniform, center, extremes) και μεταβολές Δ , $\Delta\%$ από [1–10] σε [1–100].

[Πίνακας 5.4.11:](#) Μέσος λόγος (σε %) του Our/Optimal ανά κατανομή (uniform, center, extremes) και μεταβολές Δ , $\Delta\%$ από [1–10] σε [1–100].

[Πίνακας 5.4.12:](#) Τελικό ποσοστό ζήλιας ανά κατανομή, με Δ , $\Delta\%$ από [1–10] σε [1–100].

[Πίνακας 5.4.13:](#) Μέσοι χρόνοι εκτέλεσης ανά κατανομή (uniform, center, extremes) και μεταβολές Δ , $\Delta\%$ από [1–10] σε [1–100], από όλα τα n και όλες τις πυκνότητες δ .

[Πίνακας 5.4.14:](#) Μέσος λόγος (σε %) του Our/Optimal ανά κατανομή (uniform, center, extremes) και μεταβολές Δ , $\Delta\%$ από [1–10] σε [1–100].

[Πίνακας 5.4.15:](#) Τελικό ποσοστό ζήλιας ανά κατανομή, με Δ , $\Delta\%$ από [1–10] σε [1–100].

ΚΑΤΑΛΟΓΟΣ ΕΙΚΟΝΩΝ

[Εικόνα 4.1](#): Κύρια οθόνη εισόδου. Επιλογή μεθόδου (Text/File/Wizard), επιλογές εμφάνισης λεπτομερειών και γραφήματος, κουμπιά καθαρισμού/εκτέλεσης.

[Εικόνα 4.2.1](#): Εισαγωγή ακμών με manual text: μορφή “A,B:valA,valB”.

[Εικόνα 4.2.2.1-2](#): Είσοδος δεδομένων με τη μορφή αρχείου και μετάβαση στον φάκελο αρχείων για επιλογή του .txt με τα δεδομένα..

[Εικόνα 4.2.2.3-4](#): Το περιεχόμενο του αρχείου .txt που επιλέξαμε και εμφάνιση μηνύματος κατάστασης με το πλήθος agents/ακμών που αναγνωρίστηκαν.

[Εικόνα 4.2.3.1-2](#): Είσοδος δεδομένων βήμα-βήμα.

Εισαγωγή πλήθους των agents και στη συνέχεια ονομασία του καθένα.

[Εικόνα 4.2.3.3-5](#): Εισαγωγή πλήθους αγαθών-ακμών, επισήμανση-εισαγωγή ακμών και εισαγωγή αποτίμησης των προσκείμενων agents για την ακμή.

[Εικόνα 4.3.1.1-3](#): Πίνακες κατανομών ανά στάδιο.

Για κάθε agent: δέσμη ακμών, συνολική αξία και ένδειξη EFX-safe (πράσινο/κόκκινο).

[Εικόνα 4.3.2](#): βήμα-βήμα του αλγορίθμου, έλεγχος EFX

και μετρικές εκτέλεσης (χρόνος, % χωρίς ακμή, % envied), με δυνατότητα αντιγραφής.

[Εικόνα 4.3.3](#): Πίνακας με τις αποτιμήσεις του κάθε άκρου ανά ακμή, με επιλογή εξαγωγής σε CSV.

[Εικόνα 4.3.4.1](#): Initial Graph: κυκλική διάταξη, ουδέτερο χρώμα κόμβων, ακμές χωρίς βέλη (μόνο για τοπολογία).

[Εικόνα 4.3.4.2-4](#): Greedy Allocation / After Reduce Envy / Final Allocation:

κατευθυνόμενες ακμές όπου το βέλος δείχνει σε ποιον agent έχει κατανεμηθεί το αγαθό.

[Εικόνα 5.3.1-3](#): Γραφήματα χρόνου σε ms ως προς την πυκνότητα δ των γράφων. Οι συγκεκριμένοι γράφοι έχουν πλήθος κόμβων 10, 60 και 100 αντίστοιχα.

[Εικόνα 5.3.4-6](#): Γραφήματα χρόνου σε ms ως προς το πλήθος n των γράφων. Οι συγκεκριμένοι γράφοι έχουν πυκνότητα 0.2, 0.6 και 1 αντίστοιχα.

[Εικόνα 5.3.7-9](#): Γραφήματα ποσοστού ποιότητας λύσης vs πυκνότητα δ των γράφων. Οι συγκεκριμένοι γράφοι έχουν πλήθος κόμβων 10, 60 και 100 αντίστοιχα.

[Εικόνα 5.3.10-12](#): Γραφήματα ποσοστού ποιότητας λύσης vs πλήθος n των γράφων. Οι συγκεκριμένοι γράφοι έχουν πυκνότητα 0.2, 0.6 και 1 αντίστοιχα.

[Εικόνα 5.3.13-15](#): Γραφήματα ποσοστού envied πρακτόρων ως προς την πυκνότητα δ των γράφων. Οι συγκεκριμένοι γράφοι έχουν πλήθος κόμβων 10, 30 και 60 αντίστοιχα.

[Εικόνα 5.3.16-19](#): Γραφήματα ποσοστού envied πρακτόρων ως προς το πλήθος κόμβων n του γράφου. Οι συγκεκριμένοι γράφοι έχουν πυκνότητα 0.2, 0.4, 0.6 και 0.8 αντίστοιχα.

[Εικόνα 5.3.20-22](#): Μέσος όρος χρόνου σε ms, ποσοστού ποιοτικής λύσης και ποσοστού envied πρακτόρων vs πυκνότητα.

[Εικόνα 5.3.23-25](#): Μέσος όρος χρόνου σε ms, ποσοστού ποιοτικής λύσης και ποσοστού envied vs πλήθος πρακτόρων.

[Εικόνα 5.3.26-28](#): Χρόνος σε ms, ποσοστό ποιοτικής λύσης και Ποσοστό envied πρακτόρων vs πλήθος πρακτόρων.

[Εικόνα 5.3.29-31](#): Χρόνος σε ms, ποσοστό ποιοτικής λύσης και Ποσοστό envied πρακτόρων vs πλήθος πρακτόρων.

ΠΕΡΙΛΗΨΗ

Η παρούσα πτυχιακή εργασία ασχολείται με τη μελέτη και πειραματική αξιολόγηση ενός αλγορίθμου που αφορά τη δίκαιη κατανομή αδιαίρετων αγαθών σε γράφους, με βάση την αρχή της Envy-Freeness up to any good (EFX). Στόχος της εργασίας είναι να διερευνηθεί πώς μπορούν να μοιραστούν δίκαια αγαθά μεταξύ πρακτόρων που συνδέονται μέσω σχέσεων γειτνίασης, εξασφαλίζοντας ότι κανένας πράκτορας δεν ζηλεύει τη θέση κάποιου άλλου. Η μελέτη βασίστηκε στο θεωρητικό πλαίσιο του paper *EFX Allocations on Graphs* (Christodoulou et al., 2023), ενώ ο αλγόριθμος αναπτύχθηκε και υλοποιήθηκε πλήρως σε γλώσσα προγραμματισμού Python.

Για την αξιολόγηση πραγματοποιήθηκαν εκτενή πειράματα σε διαφορετικές δομές γραφημάτων — γενικούς γράφους, κύκλους και δέντρα — με ποικίλα μεγέθη, πυκνότητες και αποτιμήσεις τιμών. Εξετάστηκαν διαφορετικοί τρόποι κατανομής τιμών και συγκρίθηκαν τα αποτελέσματα για εύρη αποτιμήσεων 1–10 και 1–100. Τα πειράματα έδειξαν ότι ο αλγόριθμος παρήγαγε πάντοτε EFX κατανομές, χωρίς να προκύψουν unoriented ακμές, επιβεβαιώνοντας τη θεωρητική του ορθότητα. Η εργασία καταλήγει ότι η προτεινόμενη προσέγγιση είναι αποδοτική, σταθερή και μπορεί να εφαρμοστεί σε προβλήματα κατανομής πόρων και συνεργατικών συστημάτων, όπου η δικαιοσύνη αποτελεί βασικό ζητούμενο.

ABSTRACT

This undergraduate thesis investigates and evaluates an algorithm for fair allocation of indivisible goods on graphs, based on the principle of Envy-Freeness up to any good (EFX). The goal is to explore how goods can be distributed fairly among agents connected through neighborhood relationships, ensuring that no agent envies another’s allocation. The study builds on the theoretical framework of the paper *EFX Allocations on Graphs* (Christodoulou et al., 2023), while the algorithm was fully developed and implemented in the Python programming language.

To assess its performance, extensive experiments were conducted on different graph structures—general graphs, cycles, and trees—with varying sizes, edge densities, and valuation ranges. Several value distributions were examined, and results were compared between valuation ranges of 1–10 and 1–100. The experiments showed that the algorithm consistently produced EFX allocations with no unoriented edges, confirming its theoretical validity. The study concludes that the proposed approach is efficient, robust, and applicable to problems of resource distribution and cooperative systems where fairness is a key objective.

ΚΕΦΑΛΑΙΟ 1: Εισαγωγή

Η δίκαιη κατανομή πόρων αποτελεί ένα από τα θεμελιώδη προβλήματα της θεωρίας παιγνίων και της αλγοριθμικής θεωρίας της κατανομής. Το πρόβλημα αυτό αφορά την ανάθεση ενός συνόλου αγαθών (goods) σε ένα σύνολο πρακτόρων (agents), με τρόπο που να ικανοποιεί κάποιες επιθυμητές ιδιότητες δικαιοσύνης και έχει απασχολήσει ερευνητές για πολλές δεκαετίες, με εφαρμογές στη διανομή αγαθών σε αγορές και στην διαχείριση πόρων του δημοσίου, δικτύων και κοινωνικών συστημάτων. Η έννοια της **κατανομής χωρίς φθόνο (envy-freeness)** είναι ένα κριτήριο δικαιοσύνης με το οποίο κανένας πράκτορας δεν προτιμά το μερίδιο κάποιου άλλου αντί του δικού του.

Σε περιβάλλοντα με διαιρετά αγαθά, όπου οι πράκτορες μπορούν να λάβουν τμήματα ενός αγαθού, η επίτευξη envy-freeness είναι δυνατή σε ένα μεγάλο φάσμα περιπτώσεων. Αντιθέτως, στην περίπτωση αδιαίρετων αγαθών, η ύπαρξη κατανομής χωρίς φθόνο δεν είναι εγγυημένη, γεγονός που καθιστά επιτακτική την αναζήτηση εναλλακτικών, ασθενέστερων μορφών δικαιοσύνης. Μία τέτοια μορφή είναι η **κατανομή χωρίς φθόνο μέχρι ένα αγαθό (EF1 – envy-freeness up to one good)**, όπου η ζήλια ενός πράκτορα προς έναν άλλο μπορεί να εξαιρεθεί με την αφαίρεση ενός μοναδικού αγαθού από τον παραλήπτη και γνωρίζουμε ότι τέτοιου είδους κατανομή υπάρχει πάντα. Η ισχυρότερη έννοια, η **κατανομή χωρίς φθόνο με οποιοδήποτε αγαθό (EFX – envy-freeness up to any good)**, προϋποθέτει η ζήλια να εξαλείφεται με όποιο αγαθό κι αν αφαιρεθεί από τον παραλήπτη. Η ύπαρξη EFX σε γενικά σύνολα πρακτόρων και αγαθών παραμένει αντικείμενο ανοιχτής έρευνας.

Πρόσφατα, προτάθηκε ένα νέο μοντέλο που εισάγει γραφική δομή στο πρόβλημα της δίκαιης κατανομής. Στο μοντέλο αυτό, οι πράκτορες αναπαρίστανται ως κόμβοι ενός γράφου και τα αγαθά ως ακμές που συνδέουν ζεύγη πρακτόρων. Κάθε πράκτορας αποτιμά μόνο τα αγαθά (ακμές) που τον αφορούν άμεσα, δηλαδή εκείνα στα οποία συμμετέχει ως άκρο. Το μοντέλο αυτό είναι ιδιαίτερα ρεαλιστικό για εφαρμογές όπως η κατανομή πόρων σε δίκτυα συνεργασίας, η ανάλυση peer effects σε κοινωνικά δίκτυα, και η διαχείριση κοινών έργων μεταξύ φορέων. Στο πλαίσιο αυτό αποδείχθηκε ότι είναι δυνατή η εύρεση EFX κατανομής σε πολυωνυμικό χρόνο για το συγκεκριμένο μοντέλο, μέσω ενός συνδυασμού greedy κατανομής, αλγορίθμων μείωσης ζήτησης και στρατηγικών για την ανάθεση των υπολειπόμενων αγαθών.

Ωστόσο, εξίσου σημαντικές είναι οι υπολογιστικές και πρακτικές πτυχές: πώς υλοποιείται βήμα-βήμα ένας τέτοιος αλγόριθμος; πώς οπτικοποιείται η εξέλιξη των φάσεων; ποια διαγνωστικά μάς επιτρέπουν να ελέγξουμε/εξηγήσουμε την απουσία ζήτησης; Στο πλαίσιο αυτό, η παρούσα εργασία δίνει έμφαση (α) στην πιστή υλοποίηση του μοντέλου και του αλγορίθμου EFX σε Python, (β) στην ανάπτυξη γραφικού περιβάλλοντος (GUI) που επιτρέπει τη διαδραστική διερεύνηση παραδειγμάτων και τη γραφική απεικόνιση των κατανομών ανά φάση, και (γ) σε μια πειραματική διερεύνηση με στοχαστικά παραγόμενες τοπολογίες/αποτιμήσεις ώστε να αναδειχθεί η σταθερότητα του σχήματος και η ερμηνευσιμότητα των αποτελεσμάτων.

1.1 Προσωπική Συνεισφορά

Η παρούσα πτυχιακή εργασία επικεντρώνεται στη μελέτη του μοντέλου και του αλγορίθμου εύρεσης EFX κατανομών (Christodoulou et al., 2023). Συγκεκριμένα:

- Υλοποιείται το πλήρες μοντέλο του paper, βασισμένο στις φάσεις **Greedy Allocation**, **Reduce Envy** και **Allocate Remaining Goods** σε Python.
- Αναπτύσσεται γραφικό περιβάλλον (GUI) σε Python για εύκολη αλληλεπίδραση με τον αλγόριθμο, καθώς και για την απεικόνιση του γράφου κατανομής.
- Πραγματοποιείται πειραματική μελέτη με αυθαίρετες τοπολογίες και αποτιμήσεις, ώστε να διαπιστωθεί η σταθερότητα και η εγκυρότητα της EFX κατανομής

Το σύστημα επιτρέπει τόσο την χειροκίνητη εισαγωγή δεδομένων (agents, goods, valuations), όσο και την αυτόματη δημιουργία τους. Ιδιαίτερη έμφαση δίνεται στην οπτική κατανόηση του τελικού allocation και στη δυνατότητα παρακολούθησης της αναλυτικής διαδικασίας εύρεσης της λύσης.

1.2 Σχετική Έρευνα

Η έννοια της δίκαιης κατανομής μελετάται ήδη από τον Steinhaus (Steinhaus, 1949), ενώ οι Foley (Foley, 1966) και Varian (Varian, 1973) διατύπωσαν επίσημα την έννοια του envy-freeness (EF) και τη σχέση του με την αποδοτικότητα και την ισότητα. Για τα διαιρετά αγαθά, το EF μπορεί να επιτευχθεί μέσω μηχανισμών όπως το Competitive Equilibrium from Equal Incomes (CEEI).

Για αδιαίρετα αγαθά, η ακριβής έννοια EF αποδεικνύεται συνήθως ανέφικτη. Οι Lipton, Markakis, Mossel & Saberi (Lipton et al., 2004) απέδειξαν ότι είναι δυνατή η εύρεση envy-free up to one good (EF1) κατανομών σε πολυωνυμικό χρόνο, ενώ ο Budish (Budish, 2011) ανέδειξε τη χρησιμότητά τους σε εφαρμογές κατανομής πόρων.

Το **EFX (envy-freeness up to any positively valued good)** εισήχθη από τους Caragiannis et al. (Caragiannis et al., 2019a), οι οποίοι παρουσίασαν τη στενή σχέση του με το μέγιστο Nash welfare. Ωστόσο, η ύπαρξη EFX κατανομών σε γενικά περιβάλλοντα παραμένει ανοιχτό πρόβλημα. Για το λόγο αυτό αναπτύχθηκαν προσεγγιστικές λύσεις: οι Plaut & Roughgarden (Plaut & Roughgarden, 2018) απέδειξαν την ύπαρξη 1/2-EFX κατανομών, ενώ οι Amanatidis et al. (Amanatidis et al., 2020) βελτίωσαν το όριο αυτό σε περίπου 0.618-EFX.

Παράλληλα, προτάθηκαν παραλλαγές του κριτηρίου: οι Akrami et al. (Akrami et al., 2022) πρότειναν το EF k X (envy-freeness up to k goods), ενώ οι Caragiannis et al. (Caragiannis et al., 2025) το epistemic EFX, όπου η ζήλια αίρεται εφόσον υπάρχει ανακατανομή των bundles των άλλων που καθιστά τον πράκτορα ικανοποιημένο. Μια άλλη

σημαντική γραμμή είναι το **EFX with charity**, όπου κάποια αγαθά μπορεί να παραμένουν αδιάθετα· σε αυτό το πλαίσιο, οι Caragiannis et al.(Caragiannis et al., 2019b), οι Chaudhury et al.(Chaudhury et al., 2020) και οι Berendsohn et al.(Berendsohn et al., 2022) ανέδειξαν εγγυήσεις για το πόσο μικρή είναι η ποσότητα αγαθών που χρειάζεται να δωριστούν για να υπάρξει EFX κατανομή.

Μια πρόσφατη εξέλιξη είναι το **γραφικό μοντέλο κατανομής** των Christodoulou, Fiat, Koutsoupias & Sgouritsa (Christodoulou et al., 2023). Εδώ οι πράκτορες αναπαριστώνται ως κόμβοι και τα αγαθά ως ακμές, με κάθε πράκτορα να αποτιμά μόνο τις ακμές που τον περιέχουν. Στο συγκεκριμένο μοντέλο παρουσιάστηκε πολυωνυμικός αλγόριθμος τριών φάσεων (Greedy, Reduce Envy, Allocate Remaining Goods) που εγγυάται EFX κατανομή για κάθε γράφο. Αυτό το αποτέλεσμα αποτελεί τη βάση πάνω στην οποία στηρίζεται η παρούσα εργασία.

Πρόσφατα αποτελέσματα στο γραφικό μοντέλο ενισχύουν το τοπίο γύρω από EFX και MMS. Οι Sgouritsa & Sotiriou (Sgouritsa & Sotiriou, 2025) μελετούν EFX σε πολύ-γράφους (multigraphs) με γενικές μονοτονικές αποτιμήσεις και δείχνουν ύπαρξη EFX κατανομών υπό τρεις δομικές προϋποθέσεις: (α) όταν ο πολύ-γράφος είναι διμερής (bipartite), (β) όταν κάθε πράκτορας έχει το πολύ $\lfloor n/4 \rfloor - 1$ γείτονες, και (γ) όταν ο συντομότερος κύκλος χωρίς παράλληλες ακμές έχει μήκος τουλάχιστον 6. Τα αποτελέσματα αυτά επεκτείνουν ουσιαστικά τα προηγούμενα για απλούς γράφους και προσφέρουν νέα κριτήρια δομικής επάρκειας για EFX.

Παράλληλα, οι Christodoulou & Mastrakoulis (Christodoulou & Mastrakoulis, 2025) εξετάζουν MMS/PMMS στο ίδιο γραφικό μοντέλο. Για αθροιστικές αποτιμήσεις αποδεικνύουν ύπαρξη ακριβούς MMS (και επιπλέον PMMS) μέσω προσανατολισμών ακμών· για XOS αποτιμήσεις δίνουν εγγύηση $2/3$ -MMS για $n \geq 3$ και καθορίζουν οξέα άνω φράγματα που εξαρτώνται από το n · για υποπροσθετικές αποτιμήσεις αποδεικνύουν ακριβές όριο $1/2$ -MMS. Συνολικά, καταγράφεται καθαρός διαχωρισμός ανάμεσα σε κλάσεις αποτιμήσεων εντός του γραφικού μοντέλου και ένας «χάρτης» θετικών/αρνητικών αποτελεσμάτων για MMS/PMMS.

Συμπερασματικά, παρόλο που το ζήτημα της ύπαρξης EFX κατανομών παραμένει ανοιχτό, προκύπτει από την εν λόγω βιβλιογραφία πως είναι επιτεύξιμες κατανομές που προσεγγίζουν ή εξασφαλίζουν ισχυρή δικαιοσύνη σε ειδικές δομές ή με παραλλαγές.

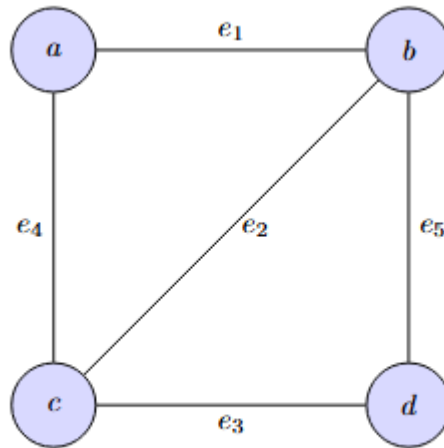
ΚΕΦΑΛΑΙΟ 2: Μαθηματικό Μοντέλο

Το μοντέλο που εξετάζεται αποτελεί μια εξειδικευμένη περίπτωση του γενικότερου προβλήματος της δίκαιης κατανομής αδιαίρετων αγαθών, προσαρμοσμένο σε δομημένα περιβάλλοντα που αναπαρίστανται με τη μορφή γραφημάτων.

2.1 Βασικοί Ορισμοί

Έστω ένα απλό, μη κατευθυνόμενο γράφημα $G = (V, E)$, όπου:

- Το σύνολο των κόμβων $V = \{v_1, v_2, \dots, v_n\}$ αναπαριστά τους n **πράκτορες** (agents).
- Το σύνολο των ακμών $E = \{e_1, e_2, \dots, e_m\}$ αναπαριστά αδιαίρετα m **αγαθά** (indivisible goods) που πρόκειται να κατανεμηθούν. Κάθε ακμή $e \in E$ συνδέει δύο κόμβους $u, w \in V$, τους οποίους ονομάζουμε προσκείμενους κόμβους (incident vertices) στην ακμή e . Το σύνολο των προσκείμενων κόμβων σε μια ακμή e συμβολίζεται ως $inc(e)$.



Σχήμα 2.1: Γράφος G με $|V| = 4$ και $|E| = 5$.

Κάθε πράκτορας $v_i \in V$ διαθέτει μία μονοτονική συνάρτηση αποτίμησης $v_i: 2^E \rightarrow \mathbb{R}_{\geq 0}$, η οποία ικανοποιεί την ιδιότητα ότι για κάθε υποσύνολο $S \subseteq E$:

$$v_i(S) = \sum_{e \in S} v_i(\{e\}),$$

κάθε πράκτορας αποτιμά μόνο τα αγαθά (ακμές) που τον αφορούν άμεσα, δηλαδή τις ακμές στις οποίες συμμετέχει ως άκρο. Οι συναρτήσεις αποτίμησης έχουν τις παρακάτω ιδιότητες:

- **Μονοτονικότητα (Monotonicity):** $v_i(S) \leq v_i(T)$, $S \subseteq T \subseteq E$.
- **Μη αρνητικότητα (Non-negativity):** $v_i(S) \geq 0$, $S \subseteq E$.
- **Γραφική εξάρτηση (Graph dependency):**
 - $v_i(\{e\}) = 0$, $v_i \notin inc(e)$.
 - $v_i(S) = v_i(S \cap E_i)$, $E_i = \{e \in E: v_i \in inc(e)\}$.
- $v_i(\emptyset) = 0$.

Agent i	$v_i(e_1)$	$v_i(e_2)$	$v_i(e_3)$	$v_i(e_4)$	$v_i(e_5)$
a	3	0	0	4	0
b	5	4	0	0	2
c	0	4	5	2	0
d	0	0	1	0	3

Πίνακας 2.1: Τιμές συναρτήσεων αποτίμησης των agents για κάθε αγαθό (Σχήμα 2.1).

Παράδειγμα 2.1:

Με βάση τον Πίνακα 2.1 και το Σχήμα 2.1, η τιμή της συνάρτησης αποτίμησης για $S = \{e_2, e_3\}$ είναι:

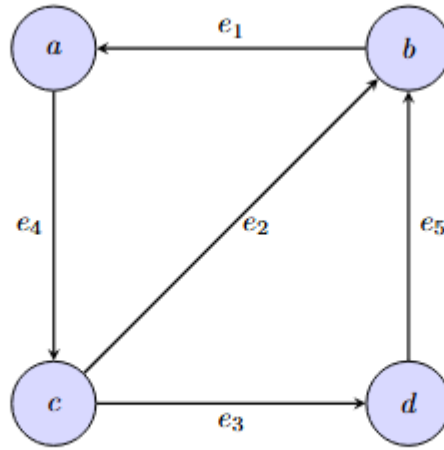
$$v_c(S) = v_c(\{e_2\}) + v_c(\{e_3\}) = 4 + 5 = 9,$$

$$v_a(S) = v_a(\{e_2\}) + v_a(\{e_3\}) = 0 + 0 = 0.$$

Δηλαδή ο c αποτιμά το S επειδή είναι άκρο και στις δύο ακμές e_2, e_3 , ενώ ο a όχι (γραφική εξάρτηση). Η **μονοτονικότητα** υπάρχει διότι, $v_c(T) \geq v_c(S) = 9$, για $T \supseteq S$.

Μια κατανομή $A = (A(1), A(2), \dots, A(n))$ αποτελείται από υποσύνολα, όπου $A(k)$ είναι η μερίδα των αγαθών (bundle) που έχουν έχουν ανατεθεί στον πράκτορα v_k . Οι κατανομές έχουν τις παρακάτω ιδιότητες:

- $A(k) \cap A(l) = \emptyset$, $\forall k \neq l$.
- $\bigcup_{k=1} A(k) = E$.



Σχήμα 2.2: Παράδειγμα κατανομής αγαθών του γράφου G (Σχήμα 2.1).

Η αποτίμηση ενός πράκτορα i για μια κατανομή A δίνεται από:

$$v_i(A(i)) = \sum_{e \in A(i)} v_i(e).$$

Παράδειγμα 2.2:

Με βάση την κατανομή του Σχήμα 2.2:

$$A(a) = \{e_1\}, \quad A(b) = \{e_2, e_5\}, \quad A(c) = \{e_4\} \quad \text{και} \quad A(d) = \{e_3\}.$$

Άρα η αποτίμηση κάθε πράκτορα για την κατανομή A είναι:

$$\begin{aligned} v_a(A(a)) &= A(\{e_1\}) = 3, & v_b(A(b)) &= A(\{e_2\}) + A(\{e_5\}) = 4 + 2 = 6, \\ v_c(A(c)) &= A(\{e_4\}) = 2 & \text{και} & & v_d(A(d)) &= A(\{e_3\}) = 1 \end{aligned}$$

Agent i	$A(i)$	$v_i(A(i))$
a	$\{e_1\}$	3
b	$\{e_2, e_5\}$	6
c	$\{e_4\}$	2
d	$\{e_3\}$	1

Πίνακας 2.2: Πίνακας κατανομής και αποτιμήσεις κάθε agent i του Σχήμα 2.2.

2.2 Κριτήρια Δικαιοσύνης

- **Envy-freeness (EF)**

Μια κατανομή είναι EF (χωρίς φθόνο), όταν για κάθε ζεύγος πρακτόρων i, j ο πράκτορας i δεν αξιολογεί το μερίδιο του j υψηλότερα από το δικό του. Δηλαδή, κανένας πράκτορας δεν έχει κίνητρο να ανταλλάξει το μερίδιό του με κάποιου άλλου και η κατανομή θεωρείται πλήρως δίκαιη από όλες τις οπτικές.

$$v_i(A(i)) \geq v_i(A(j)) \quad , \forall i, j \in N .$$

- **Envy-freeness up to one good (EF1)**

Μια κατανομή είναι EF1 (χωρίς φθόνο έως ένα αγαθό), όταν για κάθε ζεύγος πρακτόρων i, j , υπάρχει τουλάχιστον ένα αγαθό στο μερίδιο του j που, αν αφαιρεθεί, ο i παύει να αξιολογεί το μερίδιο του j υψηλότερα από το δικό του. Δηλαδή, η ζήλια ενός πράκτορα μπορεί να αποδοθεί σε ένα και μόνο «προβληματικό» αντικείμενο στο μερίδιο του άλλου. Αφαιρώντας αυτό το αντικείμενο, η ζήλια εξαλείφεται.

$$\exists e \in A(j) : v_i(A(i)) \geq v_i(A(j) \setminus \{e\}) \quad , \forall i, j \in N , A(j) \neq \emptyset .$$

- **Envy-freeness up to any good (EFX)**

Μια κατανομή είναι EFX (χωρίς φθόνο για οποιοδήποτε αγαθό), όταν για κάθε ζεύγος πρακτόρων i, j και για κάθε αγαθό e στο μερίδιο του j , ο i δεν αξιολογεί το μερίδιο του j υψηλότερα από το δικό του μετά την αφαίρεση του e . Δηλαδή, η ζήλια του i εξαλείφεται με την αφαίρεση οποιουδήποτε αγαθού από το μερίδιο του j . Πρόκειται για αυστηρότερο κριτήριο από το EF1, καθώς δεν βασίζεται στην επιλογή ενός «κατάλληλου» αγαθού αλλά απαιτεί ανθεκτικότητα για κάθε αφαίρεση.

$$\forall e \in A(j) : v_i(A(i)) \geq v_i(A(j) \setminus \{e\}) \quad \forall i, j \in N , A(i) \neq \emptyset .$$

Ο συμβολισμός $\mathbf{EF} \Rightarrow \mathbf{EFX} \Rightarrow \mathbf{EF1}$ εκφράζει μια **ιεραρχία αυστηρότητας** των κριτηρίων δικαιοσύνης:

- Αν μια κατανομή είναι **EF**, τότε είναι αυτόματα και **EFX** και—κατ' επέκταση—**EF1**.
- Αν μια κατανομή είναι **EFX**, τότε είναι κατ' ανάγκη **EF1**.
- Τα αντίστροφα δεν ισχύουν γενικά: υπάρχουν κατανομές που είναι **EFX** αλλά όχι **EF**, καθώς και κατανομές που είναι **EF1** αλλά όχι **EFX**.

Επομένως, το EF είναι ισχυρότερο κριτήριο από το EFX, και το EFX ισχυρότερο από το EF1.

Παράδειγμα 2.3:

Με βάση την κατανομή του Πίνακα 2.2, ισχύει ότι:

- Η κατανομή δεν είναι EF, διότι τρεις πράκτορες προτιμούν δέσμες άλλων πρακτόρων από την δική τους.
Π.χ. $v_c(A(b)) = 4 > 2 = v_c(A(c))$,

$$v_a(A(c)) = 3 > 4 = v_a(A(a)),$$

$$v_d(A(b)) = 4 > 1 = v_d(A(d)).$$

- Η κατανομή είναι EF1, εφόσον υπάρχει αγαθό που αν αφαιρεθεί από την ζηλευτή δέσμη, ο πράκτορας παύει να ζηλεύει.

$$\text{Π.χ. } v_c(A(b) \setminus e_2) = 0 < 2 = v_c(A(c)),$$

$$v_a(A(c) \setminus e_4) = 0 < 4 = v_a(A(a)),$$

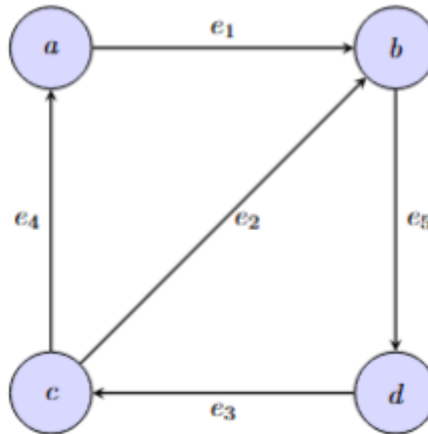
$$v_d(A(b) \setminus e_5) = 0 < 1 = v_d(A(d)).$$

- Η κατανομή δεν είναι EFX, εφόσον οι πράκτορας συνεχίζει να ζηλεύει, ακόμα κι αν αφαιρεθεί κάποιο αγαθό από τη ζηλευτή δέσμη.

$$\text{Π.χ. } v_c(A(b) \setminus e_5) = 4 > 2 = v_c(A(c)),$$

$$v_d(A(b) \setminus e_2) = 4 > 1 = v_d(A(d)).$$

Η συγκεκριμένη κατανομή δεν είναι EF και EFX, αλλά είναι EF1. Αυτό δε σημαίνει απαραίτητα ότι δεν μπορεί να υπάρξει EF και EFX κατανομή σε αυτόν το γράφο.



Σχήμα 2.3: Παράδειγμα EF, EFX και EF1 κατανομής του Σχήμα 2.1

Agent i	$A(i)$	$v_i(A(i))$
a	$\{e_4\}$	4
b	$\{e_1, e_2\}$	9
c	$\{e_3\}$	5
d	$\{e_5\}$	3

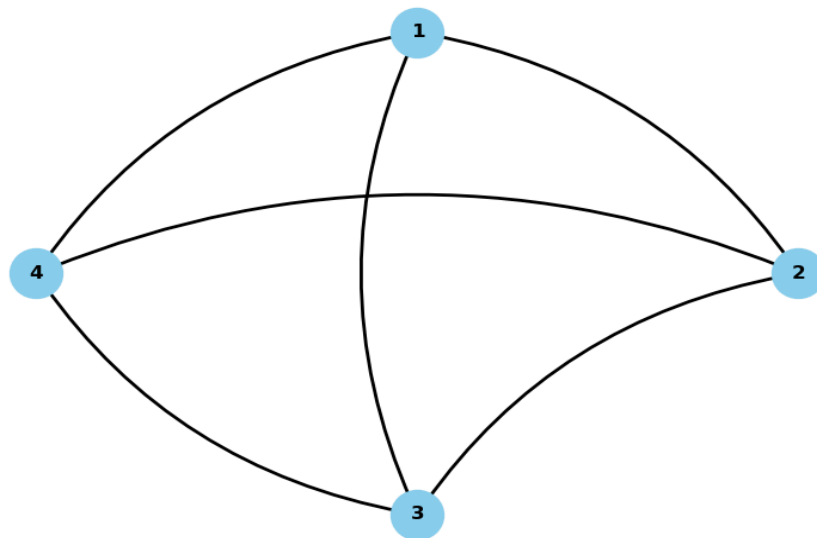
Πίνακας 2.3: Πίνακας κατανομής EF, EFX και EF1 του Σχήμα 2.3.

Παραπάνω (Σχήμα 2.3 και Πίνακας 3) βλέπουμε ένα παράδειγμα κατανομής του γράφου, η οποία ικανοποιεί τα κριτήρια EF1, EFX και EF. Με τον αλγόριθμο που θα αναλυθεί στη συνέχεια, δημιουργείται κατανομή που ικανοποιεί πάντα το κριτήριο EFX, συνεπώς και το EF1, αλλά όχι το EF.

2.3 Προσανατολισμός (Orientation)

Ένας προσανατολισμός είναι μια κατανομή $A = (A(1), A(2), \dots, A(n))$ όπου, κάθε ακμή $e \in E$ κατανέμεται **αποκλειστικά** σε έναν από τους δύο προσκείμενους κόμβους της. Δηλαδή, έστω μια $e = (u, w)$, τότε $e \in A(u)$ ή $e \in A(w)$, αλλά όχι και στις δύο.

Ένα **EFX orientation** είναι ένας προσανατολισμός, όπου το κριτήριο EFX ικανοποιείται για όλους τους πράκτορες. Η ύπαρξη EFX orientation δεν είναι πάντα δεδομένη και ανήκει στα NP-complete προβλήματα.



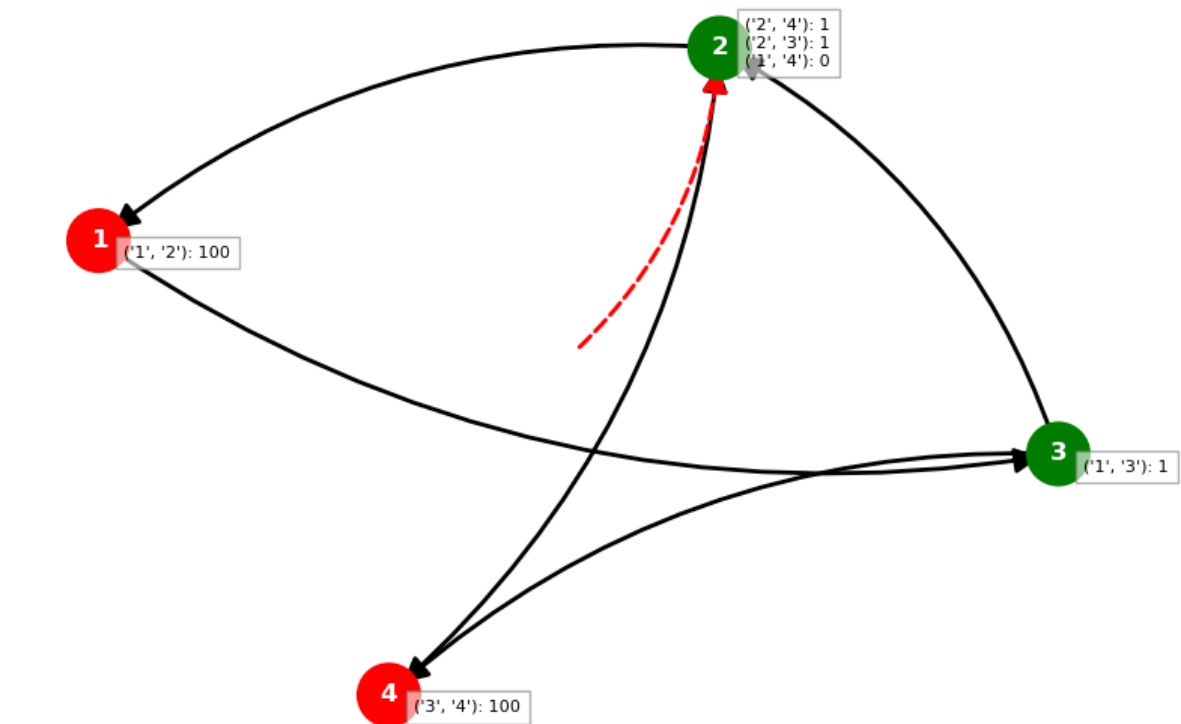
Σχήμα 2.4: Γράφος G με $|V| = 4$ και $|E| = 6$.

Edge (u, v)	u 's valuation	v 's valuation
(1, 2)	100	100
(1, 3)	1	1
(1, 4)	1	1
(2, 3)	1	1
(2, 4)	1	1
(3, 4)	100	100

Πίνακας 2.4: Πίνακας αποτιμήσεων του Σχήμα 2.4.

Παράδειγμα 2.4:

Στο Σχήμα 2.4, με τις αποτιμήσεις του Πίνακα 2.4 δε μπορεί να υπάρξει EFX orientation, με όποιον τρόπο και αν διαμοιράσουμε τα αγαθά-ακμές στους πράκτορες-κόμβους.



Σχήμα 2.5: EFX κατανομή του Σχήμα 2.4 . Λίπλα στους κόμβους αναγράφονται οι ακμές που έχουν κατανεμηθεί συγκεκριμένα σε αυτούς και η αποτίμηση των κόμβων προς τις ακμές.

Agent	Allocated Goods	Total Value
1	('1', '2')	100
2	('2', '4'), ('2', '3'), ('1', '4')	2
4	('3', '4')	100
3	('1', '3')	1

Πίνακας 2.5: Πίνακας κατανομής του Σχήμα 2.5.

Παρόλο που δεν υπάρχει EFX orientation (προσανατολισμός), υπάρχει EFX allocation (κατανομή) (Σχήμα 2.5, Πίνακας 2.5). Αντίθετα με την κατανομή, στον προσανατολισμό δεν υπάρχει η δυνατότητα να δοθεί σε κάποιον πράκτορα μια ακμή, που δεν είναι προσκείμενη του (στο Σχήμα 2.5 η ακμή (2,3) έχει δοθεί στον κόμβο 2).

ΚΕΦΑΛΑΙΟ 3: Αλγόριθμος και Υλοποίηση

Ο σκοπός του παρόντος κεφαλαίου είναι να παρουσιάσει συγκεντρωμένα τον αλγόριθμο που υλοποιήθηκε για την εύρεση EFX κατανομής σε γράφους και τον τρόπο με τον οποίο ενσωματώθηκε σε πλήρες λογισμικό με γραφικό περιβάλλον. Η μεθοδολογία ακολουθεί τον τριφασικό σκελετό του Christodoulou, Fiat, Koutsoupias & Sgouritsa (Christodoulou et al., 2023) — **Greedy Allocation, Reduce Envy, Allocate Remaining Goods** — ο οποίος αποδεικνύει την ύπαρξη και υπολογισμό EFX κατανομής στο γραφικό μοντέλο και αποτελεί τη θεωρητική βάση της υλοποίησης.

Όλες οι εικόνες του κεφαλαίου (γράφοι, πίνακες bundles, ενδιάμεσα στάδια) είναι απευθείας στιγμιότυπα (screenshots) από τον δικό μου κώδικα/GUI, ώστε τα αποτελέσματα να είναι αναπαραγώγιμα και να αντικατοπτρίζουν ακριβώς την εκτέλεση των αλγορίθμων στα δεδομένα που περιγράφονται στο *Κεφάλαιο 2*. Η δομή που ακολουθεί είναι: συνοπτική περιγραφή της ιδέας (3.1), επεξήγηση της απεικόνισης των γραφημάτων (3.2), αναλυτική παρουσίαση κάθε φάσης με παραδείγματα (3.3–3.5) και σύνοψη/ψευδοκώδικας του τελικού αλγορίθμου με έλεγχο ορθότητας (3.6).

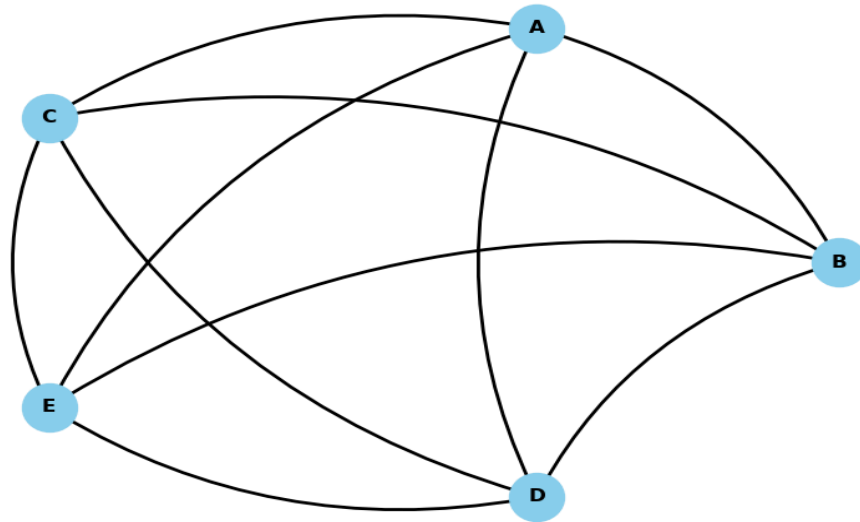
3.1 Περιγραφή Γενικής Ιδέας

Ο αλγόριθμος που χρησιμοποιείται για για την εύρεση EFX κατανομής στον γράφο βασίζεται σε τρεις διακριτές φάσεις:

1. **Greedy Κατανομή** - Απόδοση αγαθών με βάση την μέγιστη αποτίμηση σε πράκτορες χωρίς μερίδιο.
2. **Μείωση Ζήλιας (Envy Reduction)** - Ανακατανομή αγαθών με στόχο την άρση φθόνου
3. **Κατανομή Μη-κατανεμημένων Αγαθών** - Απόδοση αγαθών που δεν έχουν κατανεμηθεί στις προηγούμενες φάσεις αποφεύγοντας φθόνο στον γράφο

Η διαδικασία εξασφαλίζει EFX κατανομή στην γραφική δομή.

Initial Graph (Before Allocation)



Σχήμα 3.1: Γράφος G με $|V| = 5$ και $|E| = 10$.

Edge (u, v)	u's valuation	v's valuation
(A, B)	7	6
(A, C)	10	4
(A, D)	6	2
(A, E)	4	3
(B, C)	7	4
(B, D)	4	5
(B, E)	7	8
(C, D)	5	4
(C, E)	4	7
(D, E)	2	5

Πίνακας 3.1: Πίνακας αποτίμησης του Σχήμα 3.1.

3.2 Συμβάσεις απεικόνισης γραφήματος

- **Κόμβοι (agents):** κάθε κόμβος παριστά έναν agent.
- **Ακμές (edges):** κάθε ακμή $e = (u, v)$ παριστά ένα αγαθό.
- **Βέλος πάνω στην ακμή:** η κατεύθυνση του βέλους δείχνει τον **κόμβο-agent** που κατέχει το αγαθό (παραλήπτης της κατανομής).
- **Πλαίσια δίπλα στους κόμβους:** το πλαίσιο κάθε κόμβου περιέχει τις ακμές που έχουν κατανεμηθεί στον συγκεκριμένο κόμβο **μαζί με την αποτίμησή του** για καθεμία. π.χ. $\{(a, b): 5.0, (b, c): 4.0\}$ σημαίνει ότι ο agent κρατά τις ακμές (a, b) και (b, c) και τις αποτιμά σε 5.0 και 4.0 αντίστοιχα.
- **Συνεχής μαύρη ακμή:** με συνεχή γραμμή σχεδιάζονται οι ακμές που είναι προσκείμενες στους κόμβους-άκρα τους. Μπορεί να είναι κατανεμημένες ή όχι. Αν είναι, το βέλος δείχνει τον παραλήπτη.
- **Διακεκομμένη κόκκινη ακμή:** όταν μια ακμή έχει κατανεμηθεί σε κόμβο που **δεν είναι προσκείμενος** σε αυτήν, η ακμή σχεδιάζεται **διακεκομμένη και κόκκινη**. Και εδώ το βέλος δείχνει τον **agent-παραλήπτη** της ακμής, υπογραμμίζοντας ότι η κατανομή είναι εκτός γειτνίασης.
- **Χρωματισμός κόμβων (όπου χρησιμοποιείται):**
 - Πράσινο = ο agent **δεν είναι envied** στην τρέχον φάση.
 - Κόκκινο = ο agent **είναι envied** στην τρέχον φάση.

3.3 Φάση 1: Άπληστη Κατανομή (Greedy Allocation)

Σε αυτή τη φάση, ο αλγόριθμος κατανέμει αγαθά σε πράκτορες που δεν έχουν ακόμα λάβει μερίδιο. Για κάθε τέτοιο πράκτορα i , επιλέγεται το προσκείμενο αγαθό $e \in E$ για το οποίο η αποτίμηση $v_i(e)$ είναι μέγιστη. Αν το άλλο άκρο (agent) της ακμής e δεν έχει πάρει μερίδιο, τοποθετείται πρώτο στην ουρά προσπέλασης των πρακτόρων (agents).

Algorithm 1: Greedy Allocation

Input: A graph $G = (V, E)$ with $|V| \geq 3$ and valuation functions v_i for each agent $i \in V$

Output: A partial allocation A

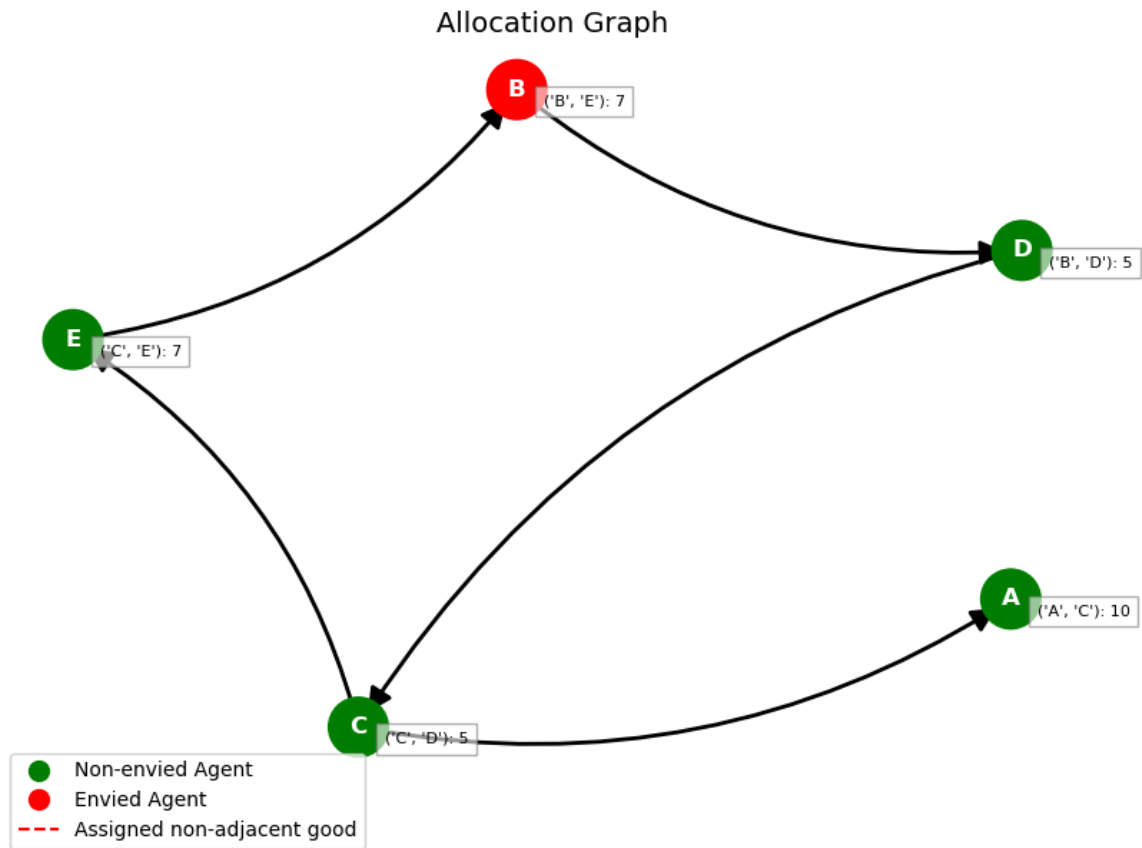
```
1  $Q \leftarrow$  queue with all agents in  $V$ ;  
2  $A(i) \leftarrow \emptyset$  for all  $i \in V$ ;  
3 while  $Q$  is not empty do  
4    $i \leftarrow Q.dequeue()$ ;  
5   if  $A(i) = \emptyset$  then  
6     choose  $e \in E$  with  $i \in e$  maximizing  $v_i(e)$ ;  
7     if  $e$  is not allocated then  
8        $A(i) \leftarrow A(i) \cup \{e\}$ ;  
9        $E \leftarrow E \setminus \{e\}$ ;  
10       $j \leftarrow$  the other agent in  $e$ ;  
11      if  $A(j) = \emptyset$  then  
12         $Q.enqueue(j)$ ;  
13 return  $A$ ;
```

Παράδειγμα 3.1:

Εφαρμόζοντας τον αλγόριθμο της Greedy κατανομής στον αρχικό γράφο, ο αλγόριθμος αποδίδει σε κάθε πράκτορα το πιο πολύτιμο διαθέσιμο αγαθό από τις γειτονικές του ακμές. Το αποτέλεσμα φαίνεται παρακάτω.

Greedy Allocation

```
Agent: D  
-> Gets ('B', 'D') (value 5)  
Agent: B  
-> Gets ('B', 'E') (value 7)  
Agent: E  
-> Gets ('C', 'E') (value 7)  
Agent: C  
-> Gets ('C', 'D') (value 5)  
Agent: A  
-> Gets ('A', 'C') (value 10)
```



Σχήμα 3.2: Greedy allocation του Σχήμα 3.1.

Agent	Allocated Goods	Total Value	EFX-safe
D	('B', 'D')	5	Yes
B	('B', 'E')	7	No
E	('C', 'E')	7	Yes
C	('C', 'D')	5	Yes
A	('A', 'C')	10	Yes

Πίνακας 3.2: Πίνακας κατανομής του Σχήμα 3.2.

Στο τέλος της Greedy Allocation κάθε πράκτορας έχει τουλάχιστον ένα αγαθό, αλλά η κατανομή δεν είναι πάντα απαλλαγμένη από ζήλια.

3.4 Φάση 2: Μείωση Ζήλειας (Reduce Envy)

Μετά την αρχική greedy κατανομή, είναι πιθανό κάποιοι πράκτορες να **φθονούν** άλλους, δηλαδή να προτιμούν το μερίδιο ενός άλλου πράκτορα έναντι του δικού τους. Η φάση αυτή στοχεύει στη **μείωση της ζήλειας** μέσω στρατηγικής αφαίρεσης και επαναδιάθεσης αγαθών, με σκοπό τη σταδιακή αποκατάσταση της δικαιοσύνης σύμφωνα με το κριτήριο **EFX**.

Ορισμός ζήλειας

Ένας πράκτορας i **φθονεί - ζηλεύει έντονα** κάποιον άλλο j , αν:

$$\exists g \in A(j) \text{ με } v_i(g) > 0 \text{ και } v_i(A(i)) < v_i(A(j) \setminus \{g\}).$$

Δηλαδή, ακόμα και αν αφαιρέσουμε οποιοδήποτε αγαθό που εκτιμά ο i από τον j , ο i συνεχίζει να προτιμά το μερίδιο του j .

Για να αντιμετωπίσουμε τη ζήλια, χρησιμοποιούμε το σύνολο U_i , το οποίο ορίζεται ως:

$$U_i = \left\{ e \in E \setminus \bigcup_{j \in V} A(j) \mid i \in e \right\}$$

Δηλαδή, περιλαμβάνει **όλα τα αγαθά (ακμές) που δεν έχουν κατανεμηθεί και που είναι προσκείμενα στον πράκτορα i** . Το σύνολο αυτό εκφράζει το **εν δυνάμει όφελος** που μπορεί να αποκτήσει ο i εάν του αποδοθούν τα αγαθά που συνδέονται με αυτόν και δεν έχουν κατανεμηθεί.

Algorithm 2 Reducing Envy Algorithm

Input: A graph $G = (V, E)$, valuation functions v_i for each $i \in V$, and a partial allocation A

Output: A partial allocation A satisfying envy reduction conditions

- 1: **while** there exists an envied agent $i \in V$ such that $v_i(U_i(A)) > v_i(A(i))$ **do**
 - 2: $A(i) \leftarrow U_i(A)$ ▷ Edges in $A(i)$ become unallocated
 - 3: **while** there exists $j \in V$ and $e \in U_j(A)$ such that $v_j(e) > v_j(A(j))$ **do**
 - 4: $A(j) \leftarrow \{e\}$ ▷ Assign edge e to agent j
 - 5: **end while**
 - 6: **end while**
 - 7: **return** A
-

Παράδειγμα 3.2:

Αφού ολοκληρωθεί η φάση Greedy, εξετάζουμε εάν υπάρχουν envied πράκτορες μεταξύ των πρακτόρων και αν ο envied πράκτορας i έχει $v_i(U_i) > v_i(A_i)$. Στην παρούσα κατανομή ο πράκτορας E ζηλεύει τον πράκτορα B , καθώς η αποτίμησή του για το δικό του bundle είναι μικρότερη από την αποτίμησή του για το bundle του B . Επίσης $v_B(U_B) > v_B(A_B)$, άρα δίνουμε το U_B στον πράκτορα B και απελευθερώνουμε την (B, E) . Εφόσον ο E προτιμάει την (B, E) από το δικό του bundle, το απελευθερώνει και παίρνει την ακμή (B, E) . Δεν υπάρχει κάποιος που προτιμάει το προηγούμενο bundle του E από το δικό του, συνεπώς η διαδικασία σταματάει. Τέλος δεν υπάρχει άλλος envied πράκτορας, οπότε ο Reduce Envy τερματίζει και έχει σαν αποτέλεσμα την παρακάτω κατανομή.

Reduce Envy

Agent B is envied by E (E prefers 8 > 7)

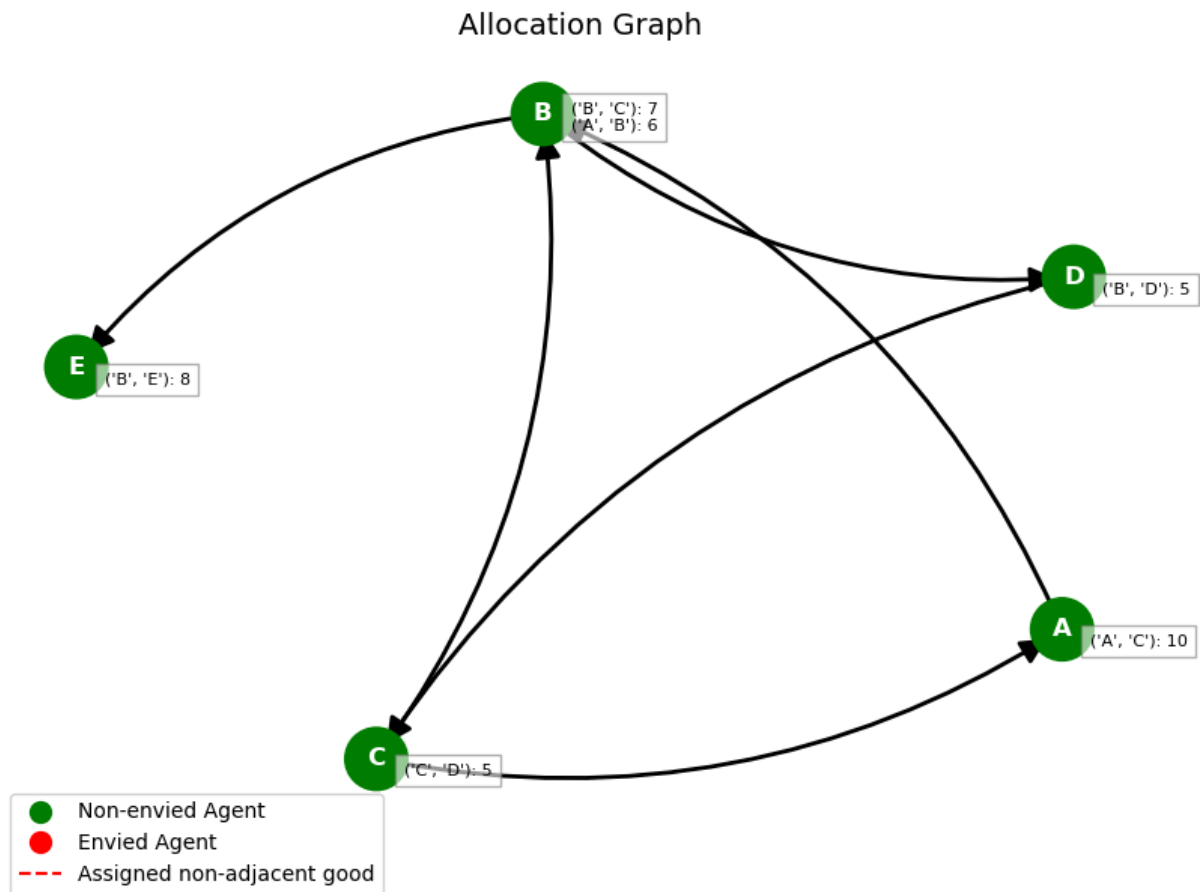
Agent B is envied and prefers unallocated goods. Replacing X_i (value 7) with $U_i(X)$ (value 13)

Old allocation: {('B', 'E')}

New allocation: {('A', 'B'), ('B', 'C')}

-> Agent E reallocated to ('B', 'E') (value 8)

No envy found with $v_i U_i > v_i X_i$. Terminating.



Σχήμα 3.3: Αποτέλεσμα μετά την εφαρμογή *Reduce Envy* στο Σχήμα 3.2.

Agent	Allocated Goods	Total Value	EFX-safe
D	('B', 'D')	5	Yes
B	('B', 'C'), ('A', 'B')	13	Yes
E	('B', 'E')	8	Yes
C	('C', 'D')	5	Yes
A	('A', 'C')	10	Yes

Πίνακας 3.3: Πίνακας κατανομής του Σχήμα 3.3.

Μετά τη μετακίνηση, ο πράκτορας E δεν ζηλεύει πλέον τον B. Στο σχήμα απεικονίζεται η νέα κατάσταση του γράφου μετά την εφαρμογή της φάσης *Reduce Envy*.

3.5 Φάση 3: Ανάθεση μη Κατανεμημένων Αγαθών

Στην τελική φάση του αλγορίθμου επιχειρείται η κατανομή των αγαθών που δεν έχουν ακόμη διαμοιραστεί. Για κάθε τέτοιο αγαθό $e = \{i, j\}$, εξετάζεται κατά πόσο μπορεί να αποδοθεί σε έναν από τους δύο πράκτορες χωρίς να δημιουργηθεί ζήλια.

Αν τουλάχιστον ένας από τους δύο πράκτορες δεν είναι είναι *envied* (κανένας πράκτορας δεν τον ζηλεύει), το αγαθό μπορεί να δοθεί σε αυτόν με ασφάλεια. Όταν όμως **και οι δύο πράκτορες είναι *envied***, τότε χρησιμοποιούνται τα **σύνολα ασφαλείας** $S_i(A)$ και $S_j(A)$.

Το **σύνολο ασφαλείας** $S_i(A)$ ενός πράκτορα i περιλαμβάνει όλους τους πράκτορες $j \in V$ που **δεν είναι *envied***, και για τους οποίους ισχύει:

$$v_i(A(i)) \geq v_i(A(j) \cup U_i(A))$$

Δηλαδή, ο i δεν φθονεί τον j ακόμη και αν ο j αποκτήσει όλα τα αγαθά που γειτονεύουν με αυτόν και παραμένουν μη-κατανεμημένα.

Συνεπώς, το αγαθό e μπορεί τότε να αποδοθεί σε κάποιον πράκτορα που ανήκει στην τομή $S_i(A) \cap S_j(A)$, καθώς μια τέτοια ανάθεση δεν προκαλεί ζήλια ούτε στον i ούτε στον j .

Αυτός ο μηχανισμός επιτρέπει την ολοκλήρωση της κατανομής χωρίς να παραβιαστεί η ισχύς της EFX.

Algorithm 3 Final Allocation of Remaining Goods

Input: A graph $G = (V, E)$, valuation functions v_i for each $i \in V$, and partial allocation A

Output: A complete allocation A with all goods assigned, preserving EFX

```

1:  $R \leftarrow E \setminus \bigcup_{i \in V} A(i)$  ▷ Unallocated goods
2: for each  $e = \{i, j\} \in R$  do
3:   if  $i$  is not envied then
4:      $A(i) \leftarrow A(i) \cup \{e\}$ 
5:   else if  $j$  is not envied then
6:      $A(j) \leftarrow A(j) \cup \{e\}$ 
7:   else
8:     Compute  $S_i(A)$  and  $S_j(A)$  ▷ Safe sets
9:     if  $S_i(A) \cap S_j(A) \neq \emptyset$  then
10:      Choose  $k \in S_i(A) \cap S_j(A)$  arbitrarily
11:       $A(k) \leftarrow A(k) \cup \{e\}$ 
12:     end if
13:   end if
14: end for
15: return  $A$ 

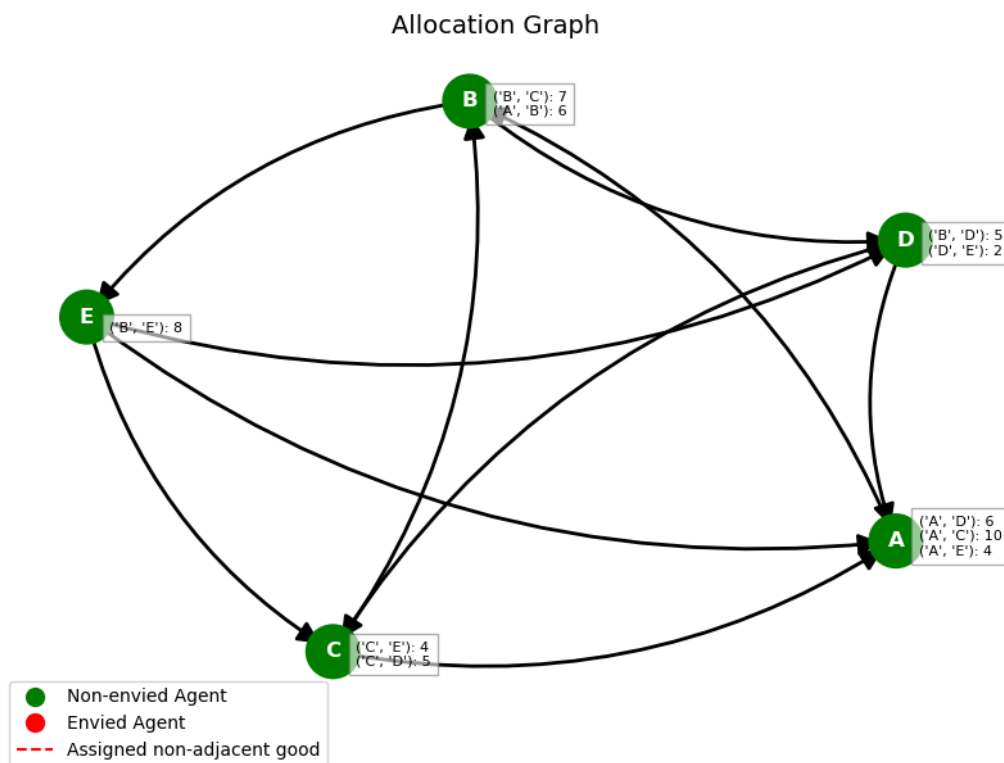
```

Παράδειγμα 3.3:

Αφού ολοκληρωθεί η φάση *Reduce Envy*, μπορεί να έχουν απομείνει αγαθά (ακμές) που δεν έχουν ακόμη κατανεμηθεί. Στο βλέπουμε πώς θα κατανεμηθούν τα αγαθά ένα - ένα.

Allocate Remaining Goods

```
('C', 'E'):  
  Agent C envied: False  
  Agent E envied: False  
Unallocated good ('C', 'E') given to non-envied agent C  
  
('A', 'D'):  
  Agent A envied: False  
  Agent D envied: False  
Unallocated good ('A', 'D') given to non-envied agent A  
  
('D', 'E'):  
  Agent D envied: False  
  Agent E envied: False  
Unallocated good ('D', 'E') given to non-envied agent D  
  
('A', 'E'):  
  Agent A envied: False  
  Agent E envied: False  
Unallocated good ('A', 'E') given to non-envied agent A
```

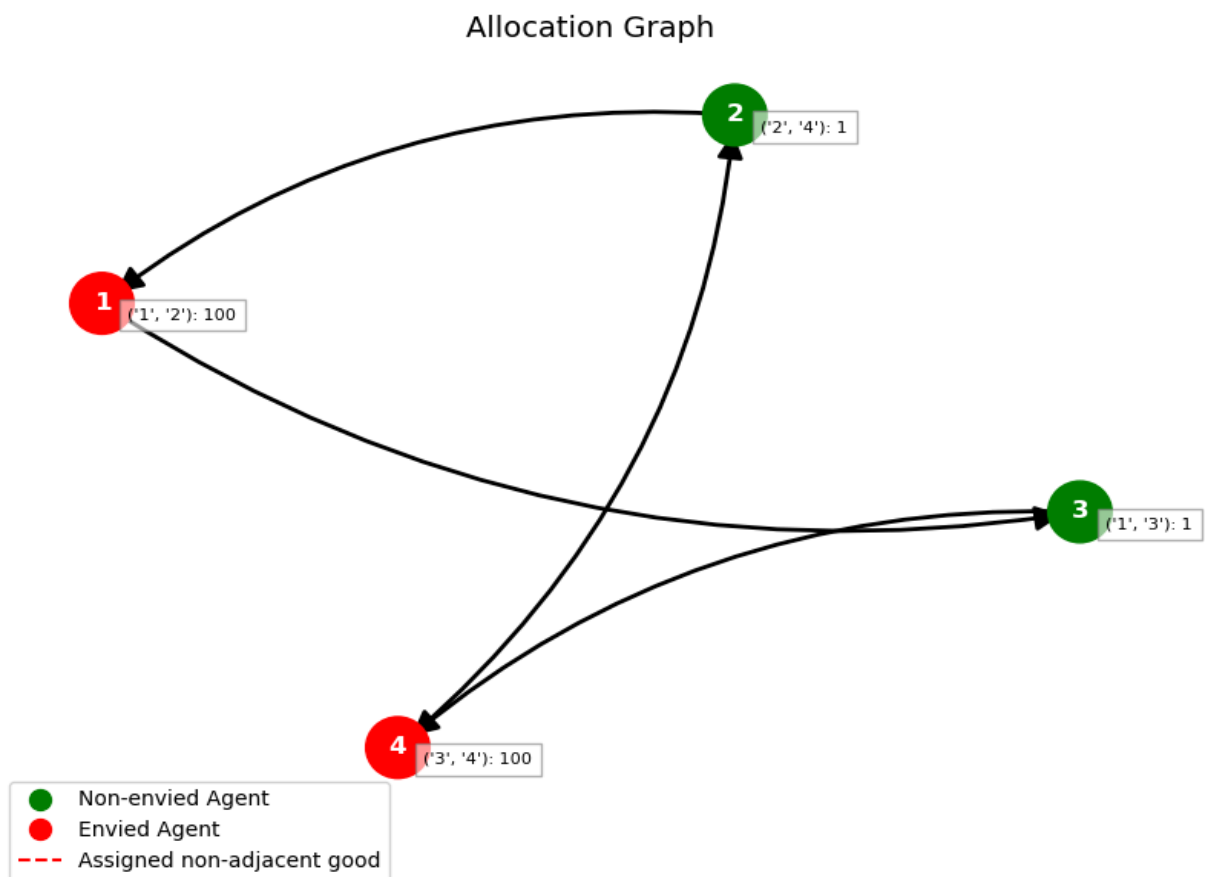


Σχήμα 3.4: Αποτέλεσμα μετά την εφαρμογή *Allocate Remaining Goods* στο Σχήμα 3.3 και τελικός γράφος EFX κατανομής του Σχήμα 3.1.

Agent	Allocated Goods	Total Value	EFX-safe
D	('B', 'D'), ('D', 'E')	7	Yes
B	('B', 'C'), ('A', 'B')	13	Yes
E	('B', 'E')	8	Yes
C	('C', 'E'), ('C', 'D')	9	Yes
A	('A', 'D'), ('A', 'C'), ('A', 'E')	20	Yes

Πίνακας 3.4: Πίνακας κατανομής του Σχήμα 3.4.

Σε αυτήν την κατανομή του γράφου, συναντάται μόνο η περίπτωση που τουλάχιστον ένας από τους δύο προσκείμενους πράκτορες στα μη κατανεμημένα αγαθά δεν είναι envid. Ένα παράδειγμα που μπορεί να δοθεί στην περίπτωση που και οι δύο προσκείμενοι πράκτορες στα μη κατανεμημένα αγαθά είναι envid, είναι στο γράφο που είδαμε προηγουμένως (Σχήμα 3.4) και η τελική κατανομή του είναι στο Σχήμα 2.5 και στον Πίνακα 5. Ο γράφος Σχήμα 2.5 είναι το αποτέλεσμα της εφαρμογής του αλγορίθμου Final Allocation of Remaining Goods στον παρακάτω γράφο Σχήμα 3.5. Οι προσκείμενοι πράκτορες της ακμής (1,4) είναι και οι δύο envid, συνεπώς η ακμή θα δοθεί στον safe agent 2.



Σχήμα 3.5: Αποτέλεσμα της εφαρμογής των αλγορίθμων Greedy Allocation και Reduce Envy στο Σχήμα 2.4

Agent	Allocated Goods	Total Value	EFX-safe
1	('1', '2')	100	No
2	('2', '4')	1	Yes
4	('3', '4')	100	No
3	('1', '3')	1	Yes

Πίνακας 3.5: Πίνακας κατανομής του Σχήμα 3.5.

Με την ολοκλήρωση της φάσης, όλα τα αγαθά έχουν πλέον κατανεμηθεί και η τελική κατανομή ικανοποιεί το κριτήριο EFX. Στο Σχήμα 3.4 παρουσιάζεται η τελική μορφή του γράφου, καθώς και ο πίνακας των μεριδίων με τις συνολικές αποτιμήσεις κάθε πράκτορα.

3.6 Τελικός Αλγόριθμος

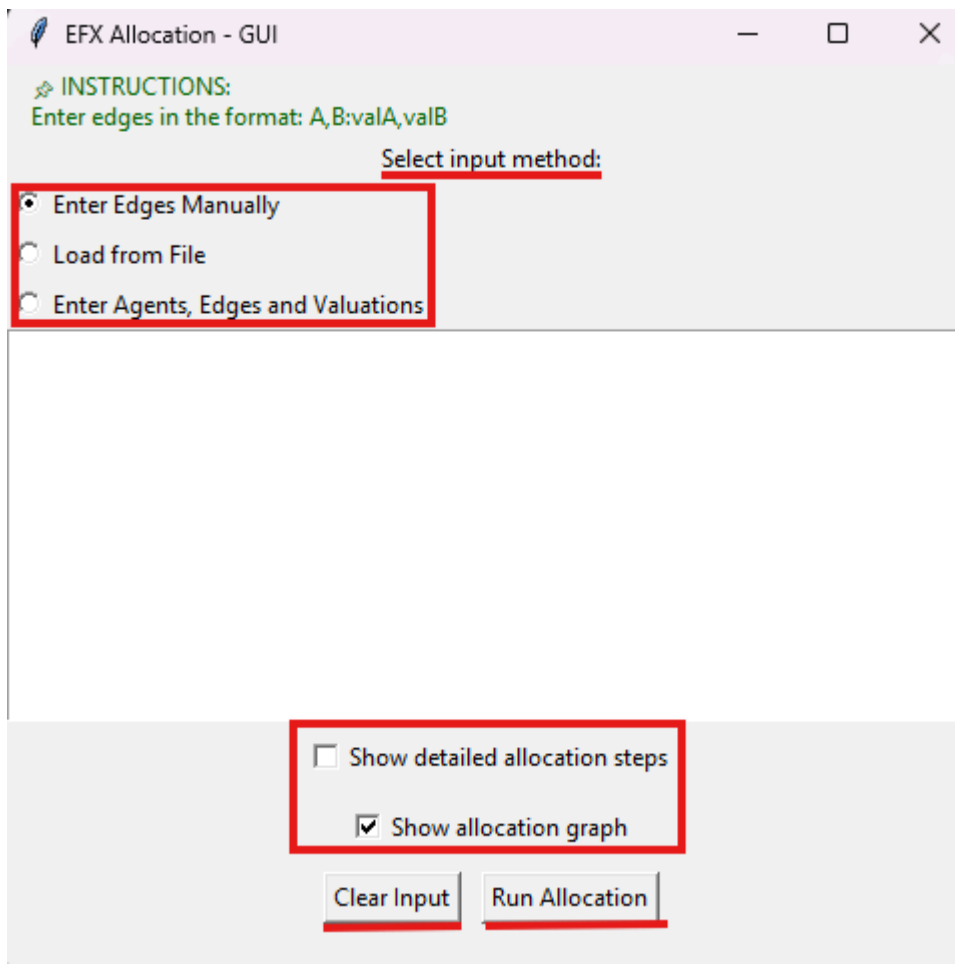
Ο συνδυασμός των φάσεων συνθέτει έναν αποτελεσματικό αλγόριθμο που εγγυάται την ύπαρξη και τον υπολογισμό EFX κατανομής σε πολυωνυμικό χρόνο, σύμφωνα με το paper που βασίζεται η παρούσα εργασία (Christodoulou et al., 2023). Στον παρακάτω ψευδοκώδικα παρουσιάζεται η συνολική διαδικασία.

Algorithm	RunEFXAllocation
1:	Input: A graph $G = (V, E)$ and valuation functions $v_i : E \rightarrow \mathbb{R}_{\geq 0}$ for each $i \in V$
2:	Output: An EFX allocation A
3:	$A_1 \leftarrow \text{GREEDYALLOCATION}(G, v)$
4:	$A_2 \leftarrow \text{REDUCEENVY}(A_1, G, v)$
5:	$A_3 \leftarrow \text{ALLOCATEREMAININGGOODS}(A_2, G, v)$
6:	return A_3

ΚΕΦΑΛΑΙΟ 4: Γραφικό Περιβάλλον (GUI) & Υλοποίηση

Στο κεφάλαιο αυτό παρουσιάζεται το γραφικό περιβάλλον της εφαρμογής και ο τρόπος με τον οποίο ο χρήστης αλληλεπιδρά με τον αλγόριθμο που περιγράφηκε στο ΚΕΦΑΛΑΙΟ 3. Όλες οι εικόνες που ακολουθούν είναι **στιγμιότυπα από τον κώδικα/GUI** που υλοποιήθηκε σε Python, ώστε τα αποτελέσματα να είναι αναπαραγώγιμα. Στόχος είναι να καταγραφεί με σαφήνεια πώς εισάγονται τα δεδομένα (γράφος και αποτιμήσεις), πώς εκτελείται ο αλγόριθμος και πώς ερμηνεύονται οι απεικονίσεις και οι πίνακες αποτελεσμάτων.

4.1 Επισκόπηση Menu Εισόδου

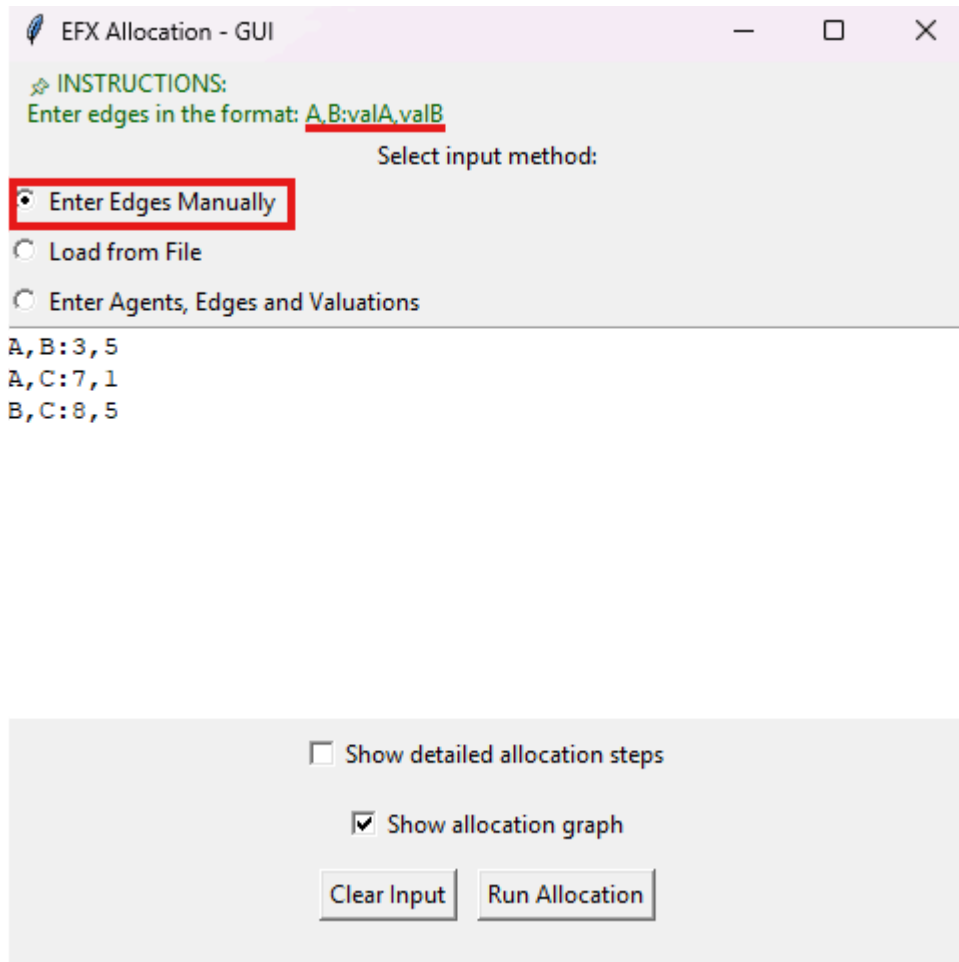


Εικόνα 4.1: Κύρια οθόνη εισόδου. Επιλογή μεθόδου (Text/File/Wizard), επιλογές εμφάνισης λεπτομερειών και γραφήματος, κουμπιά καθαρισμού/εκτέλεσης.

Η αρχική οθόνη του GUI περιέχει τις τρεις μεθόδους εισαγωγής δεδομένων (Text / File / Wizard). Τα κουμπιά *Clear Input* και *Run Allocation* εκτελούν καθαρισμό και τρέξιμο του αλγορίθμου. Τα checkboxes ενεργοποιούν την εμφάνιση αναλυτικών βημάτων του αλγορίθμου και γραφημάτων.

4.2 Είσοδος δεδομένων

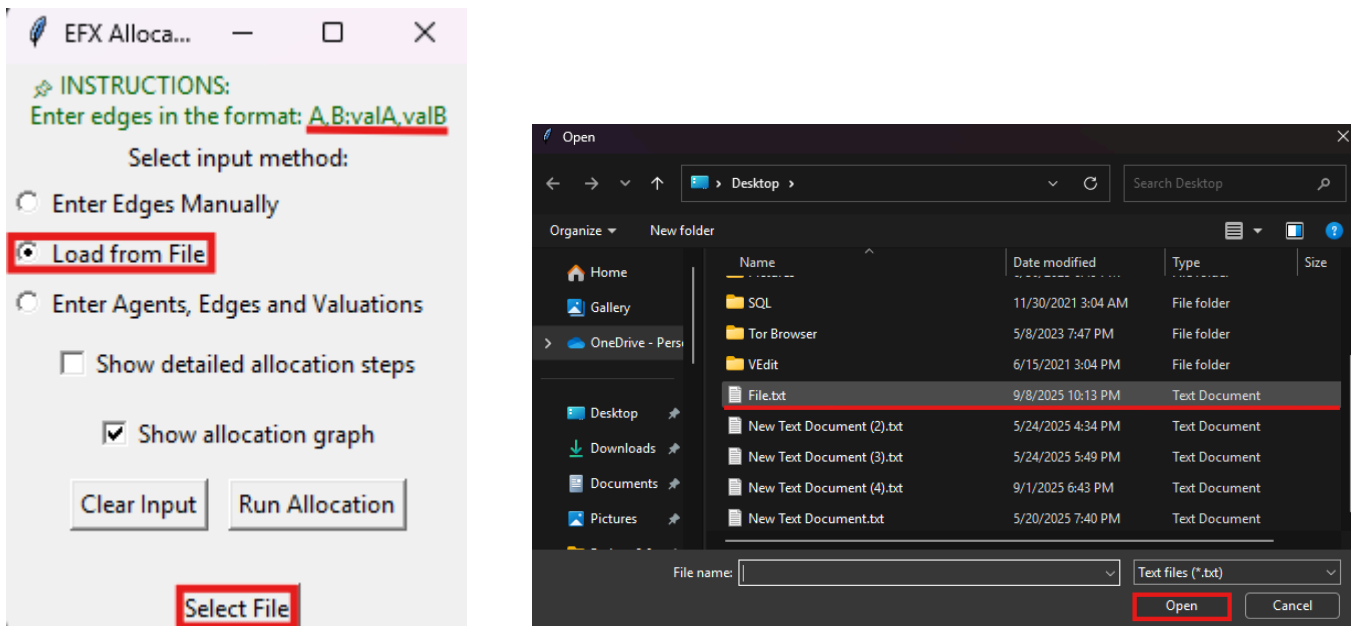
4.2.1 Manual Text



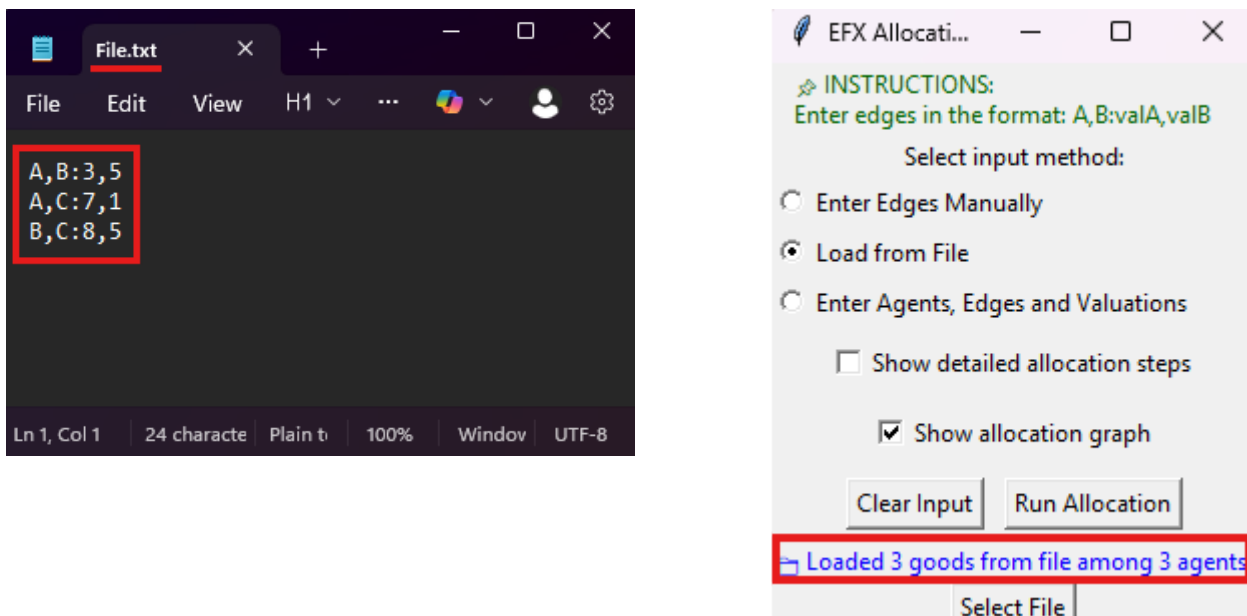
Εικόνα 4.2.1: Εισαγωγή ακμών με manual text: μορφή "A,B:valA,valB".

Η εισαγωγή δεδομένων γίνεται με την εισαγωγή ακμών και αποτιμήσεων στο πεδίο κειμένου με το πρότυπο **A,B:valA,valB** ανά γραμμή, όπου **A,B** είναι οι προσκείμενοι κόμβοι της ακμής (A,B) και **valA,valB** είναι οι αποτιμήσεις του **A** και του **B** για την ακμή (A,B) αντίστοιχα. Με το *Run Allocation* ξεκινά η διαδικασία υπολογισμού και τα αποτελέσματα προβάλλονται στα tabs του παραθύρου αποτελεσμάτων.

4.2.2 Load from File



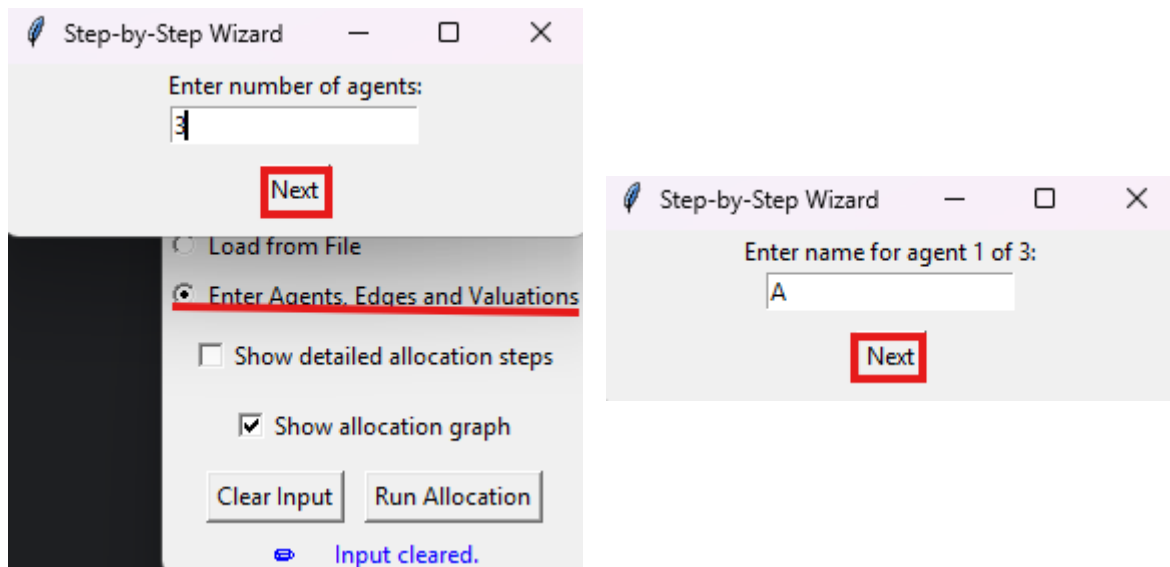
Εικόνα 4.2.2.1-2: Είσοδος δεδομένων με τη μορφή αρχείου και μετάβαση στον φάκελο αρχείων για επιλογή του .txt με τα δεδομένα..



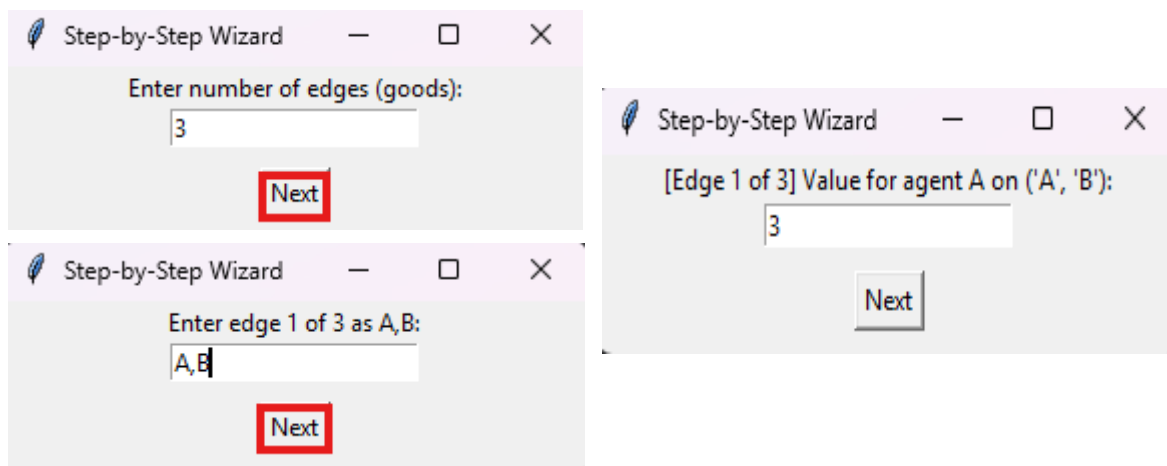
Εικόνα 4.2.2.3-4: Το περιεχόμενο του αρχείου .txt που επιλέξαμε και εμφάνιση μηνύματος κατάστασης με το πλήθος agents/ακμών που αναγνωρίστηκαν.

Η εισαγωγή δεδομένων γίνεται διαλέγοντας το αρχείο εισόδου, με περιεχόμενο της μορφής εισόδου στο 4.2.1 (βλ, Εικόνα 4.2.2.3) και στη συνέχεια εμφανίζεται μήνυμα κατάστασης με το πλήθος agents/ακμών που αναγνωρίστηκαν. Η εκτέλεση ακολουθείται πατώντας *Run Allocation*.

4.2.3 Wizard (βήμα-βήμα)



Εικόνα 4.2.3.1-2: Είσοδος δεδομένων βήμα-βήμα.
Εισαγωγή πλήθους των agents και στη συνέχεια ονομασία του καθένα

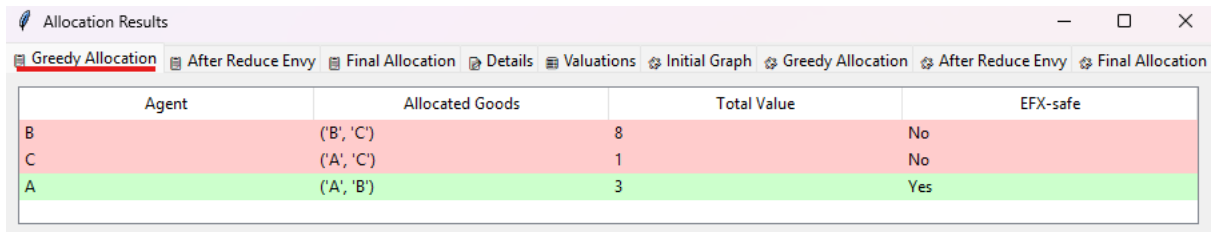


Εικόνα 4.2.3.3-5: Εισαγωγή πλήθους αγαθών-ακμών, επισημάνση-εισαγωγή ακμών και εισαγωγή αποτίμησης των προσκείμενων agents για την ακμή.

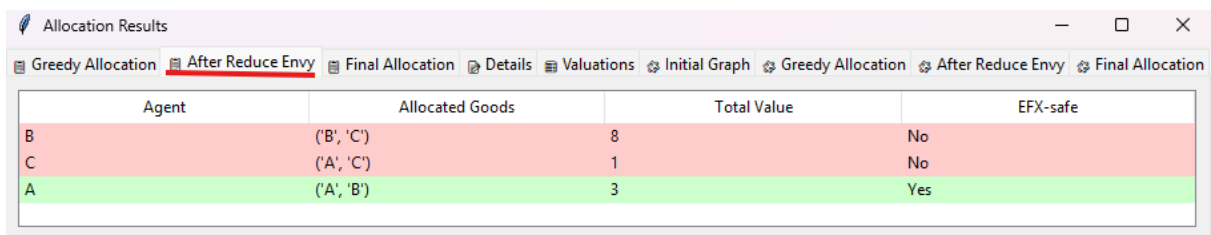
Ο οδηγός εισαγωγής ζητά αρχικά το πλήθος και τα ονόματα των agents, έπειτα τον αριθμό ακμών και τέλος συμπλήρωση της κάθε ακμής και των δύο αποτιμήσεων της από τους agents (μία για κάθε άκρο). Με την ολοκλήρωση, εμφανίζεται μήνυμα επιτυχίας και τα δεδομένα φορτώνονται αυτόματα στο σύστημα.

4.3 Παράθυρα Αποτελεσμάτων - Tabs

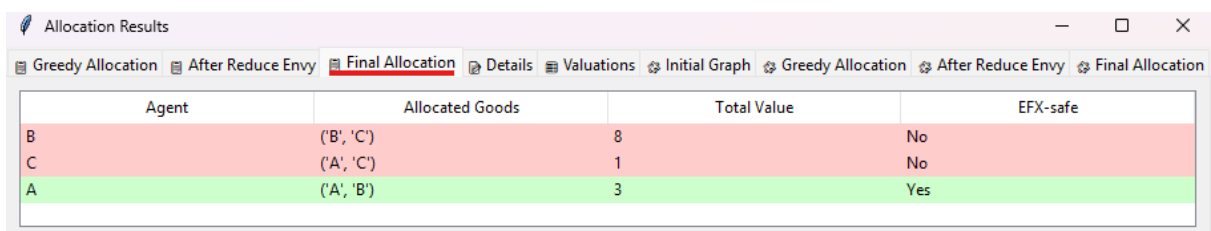
4.3.1 Allocation Tables (3 tabs)



Agent	Allocated Goods	Total Value	EFX-safe
B	('B', 'C')	8	No
C	('A', 'C')	1	No
A	('A', 'B')	3	Yes



Agent	Allocated Goods	Total Value	EFX-safe
B	('B', 'C')	8	No
C	('A', 'C')	1	No
A	('A', 'B')	3	Yes



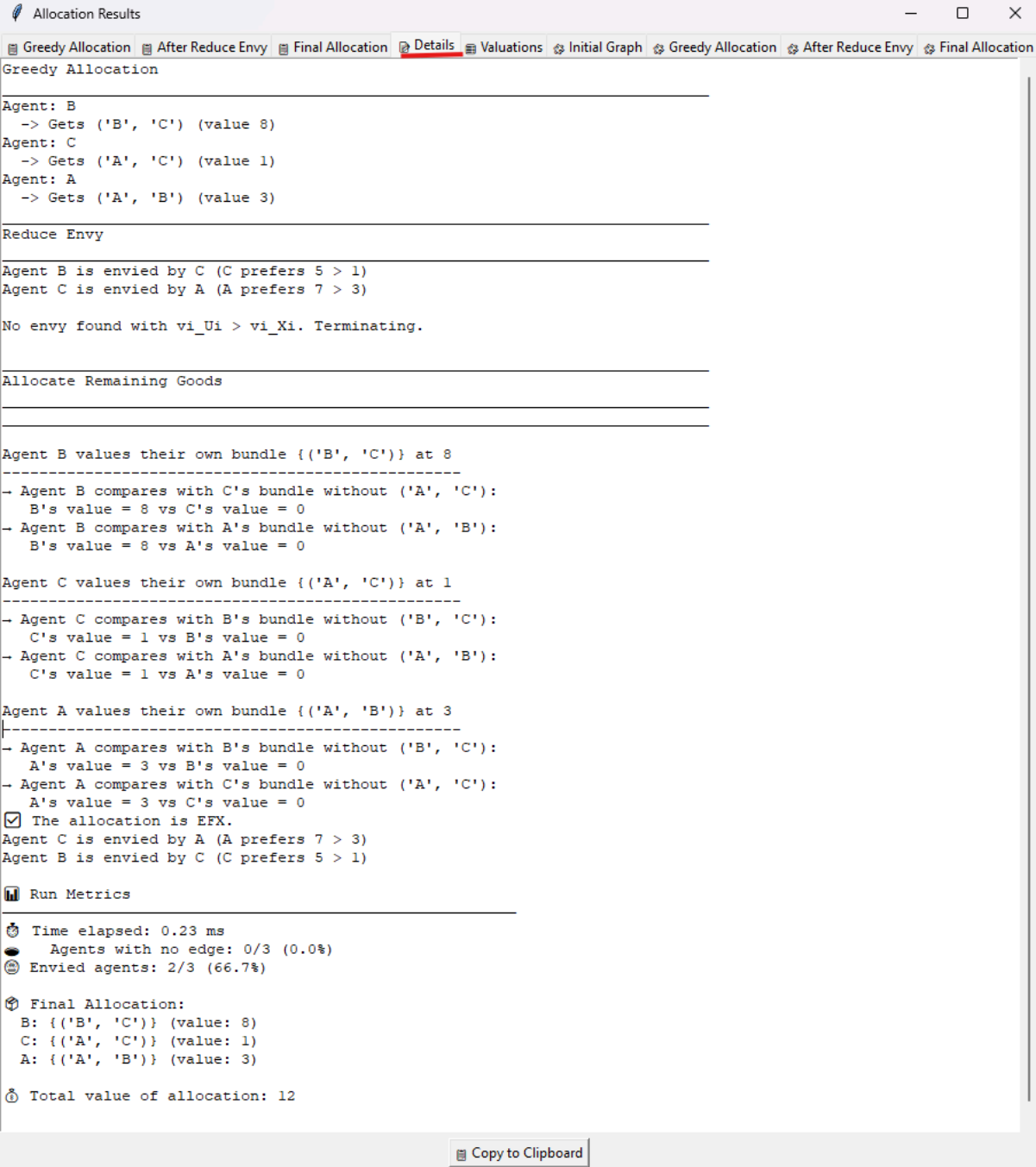
Agent	Allocated Goods	Total Value	EFX-safe
B	('B', 'C')	8	No
C	('A', 'C')	1	No
A	('A', 'B')	3	Yes

Εικόνα 4.3.1.1-3 : Πίνακες κατανομών ανά στάδιο.

Για κάθε agent: δέσμη ακμών, συνολική αξία και ένδειξη EFX-safe (πράσινο/κόκκινο).

Τα τρία tabs (*Greedy*, *After Reduce Envy*, *Final Allocation*) παρουσιάζουν τη δέσμη ακμών που κατέχει κάθε agent, τη συνολική του αξία και την ένδειξη EFX-safe. Η χρωματική σήμανση ανά γραμμή (πράσινο/κόκκινο) αντιστοιχεί σε non-envied/envied agents στο αντίστοιχο στάδιο.

4.3.2 Details



Allocation Results

Greedy Allocation

Agent: B
-> Gets ('B', 'C') (value 8)
Agent: C
-> Gets ('A', 'C') (value 1)
Agent: A
-> Gets ('A', 'B') (value 3)

Reduce Envy

Agent B is envied by C (C prefers 5 > 1)
Agent C is envied by A (A prefers 7 > 3)
No envy found with $v_i(u_i) > v_j(u_j)$. Terminating.

Allocate Remaining Goods

Agent B values their own bundle {'B', 'C'} at 8

- Agent B compares with C's bundle without ('A', 'C'):
B's value = 8 vs C's value = 0
- Agent B compares with A's bundle without ('A', 'B'):
B's value = 8 vs A's value = 0
Agent C values their own bundle {'A', 'C'} at 1

- Agent C compares with B's bundle without ('B', 'C'):
C's value = 1 vs B's value = 0
- Agent C compares with A's bundle without ('A', 'B'):
C's value = 1 vs A's value = 0
Agent A values their own bundle {'A', 'B'} at 3

- Agent A compares with B's bundle without ('B', 'C'):
A's value = 3 vs B's value = 0
- Agent A compares with C's bundle without ('A', 'C'):
A's value = 3 vs C's value = 0
☒ The allocation is EFX.
Agent C is envied by A (A prefers 7 > 3)
Agent B is envied by C (C prefers 5 > 1)

Run Metrics

Time elapsed: 0.23 ms
Agents with no edge: 0/3 (0.0%)
Envied agents: 2/3 (66.7%)

Final Allocation:
B: {'B', 'C'} (value: 8)
C: {'A', 'C'} (value: 1)
A: {'A', 'B'} (value: 3)

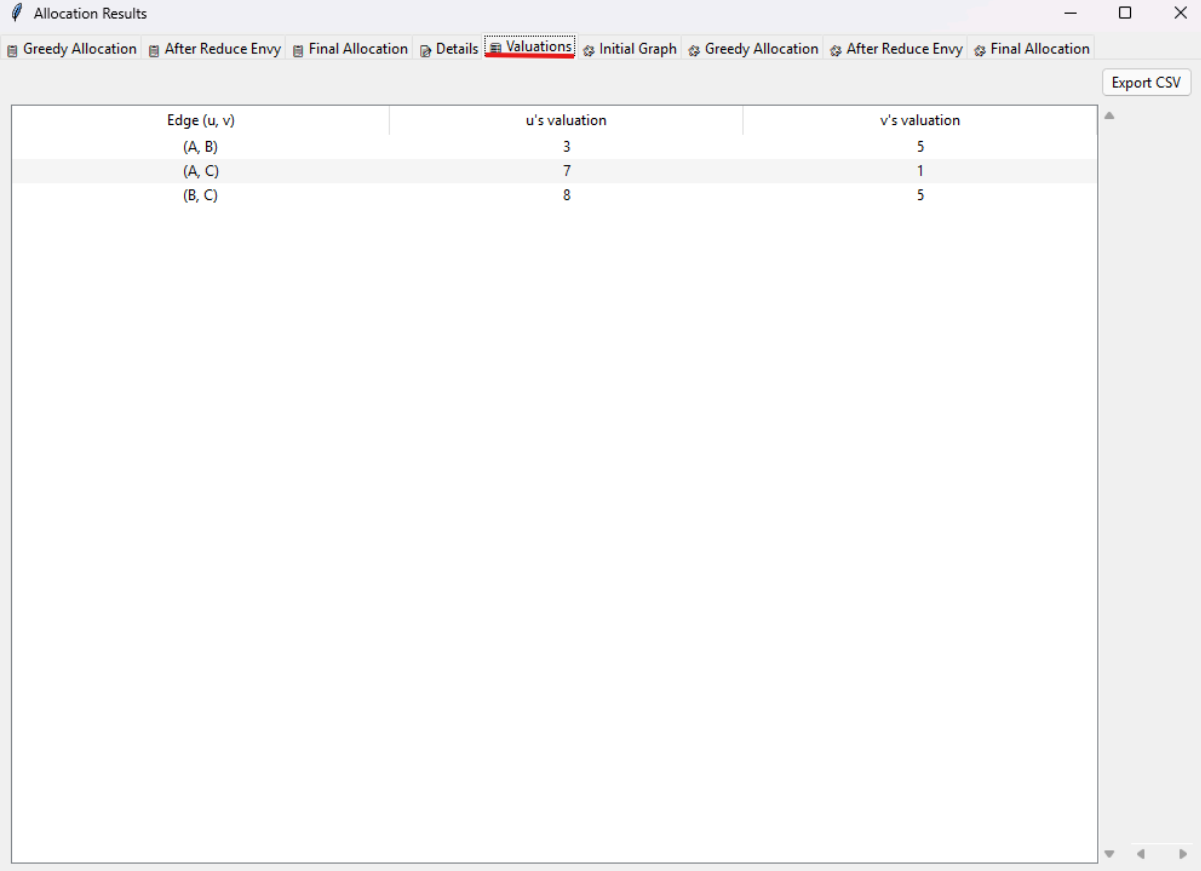
Total value of allocation: 12

Copy to Clipboard

Εικόνα 4.3.2: βήμα-βήμα του αλγορίθμου, έλεγχος EFX και μετρικές εκτέλεσης (χρόνος, % χωρίς ακμή, % envied), με δυνατότητα αντιγραφής.

Σε αυτό το tab προβάλλονται τα αναλυτικά βήματα των φάσεων (Greedy→Reduce Envy→Allocate Remaining), ο έλεγχος EFX με συγκρίσεις bundles και το τελικό συμπέρασμα. Τέλος, συνοψίζονται οι μετρικές εκτέλεσης (χρόνος αλγορίθμου, χρόνος ελέγχου EFX, % agents χωρίς ακμή, % envied, total value κατανομής), ενώ υπάρχει και κουμπί αντιγραφής του κειμένου.

4.3.3 Valuations Tab

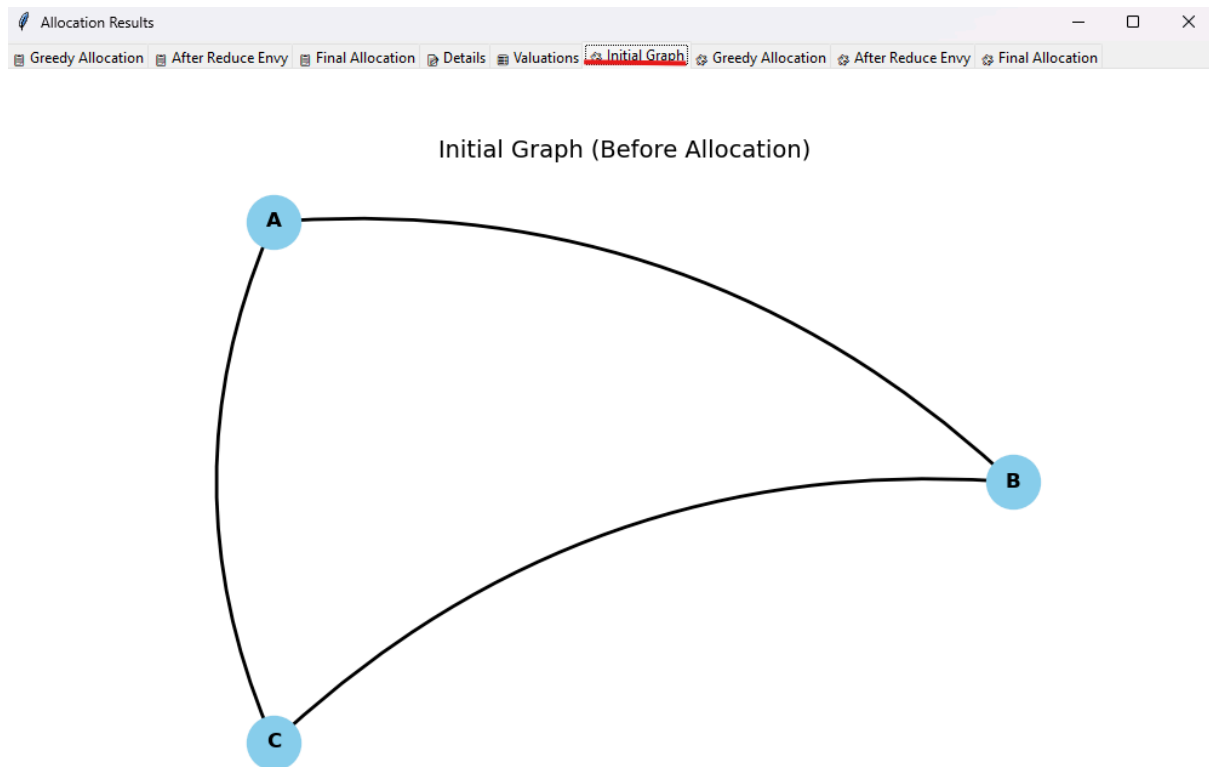


Edge (u, v)	u's valuation	v's valuation
(A, B)	3	5
(A, C)	7	1
(B, C)	8	5

Εικόνα 4.3.3: Πίνακας με τις αποτιμήσεις του κάθε άκρου ανά ακμή, με επιλογή εξαγωγής σε CSV.

Πίνακας αποτιμήσεων ανά ακμή: εμφανίζεται το ζεύγος (u, v) και οι αποτιμήσεις των u και v για την ακμή. Από εδώ μπορεί να εξαχθεί και αρχείο CSV για περαιτέρω ανάλυση.

4.3.4 Graph Views (4 tabs)

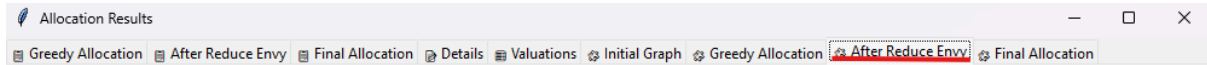
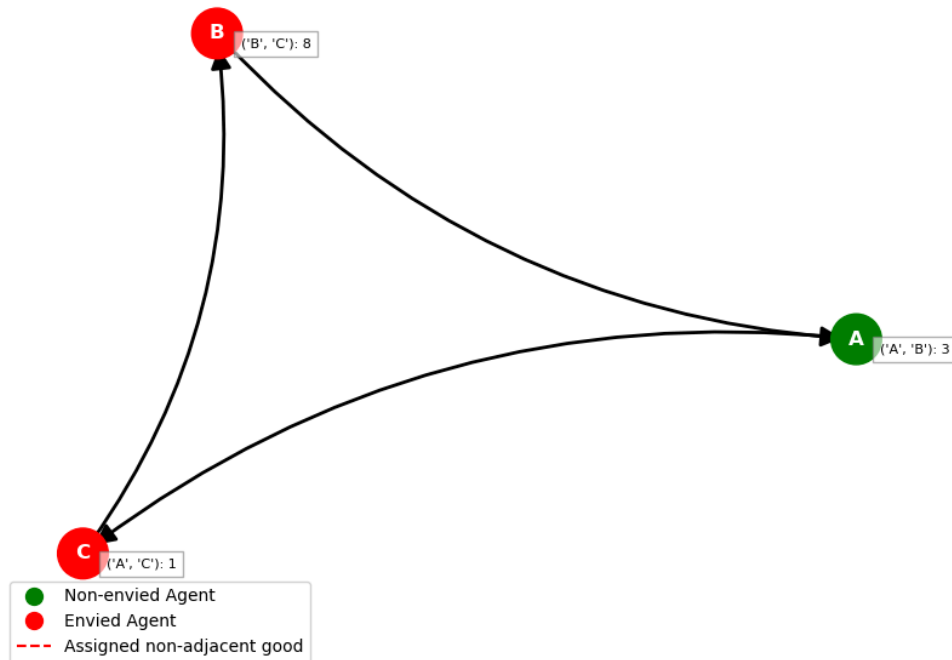


Εικόνα 4.3.4.1: Initial Graph: κυκλική διάταξη, ουδέτερο χρώμα κόμβων, ακμές χωρίς βέλη (μόνο για τοπολογία).

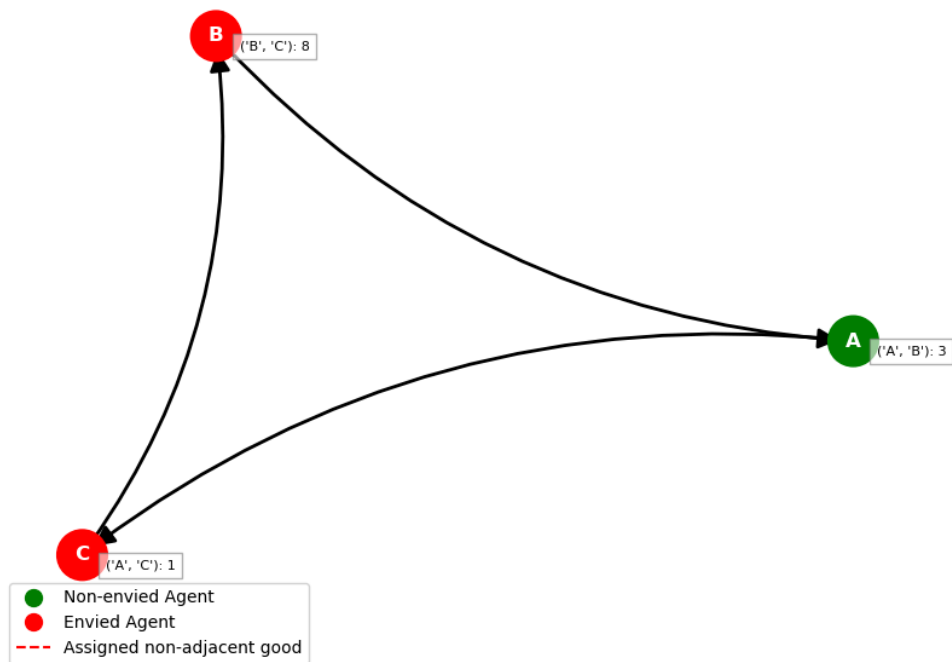
Το tab **Initial Graph** προβάλλει τον γράφο εισόδου πριν από οποιονδήποτε αλγοριθμικό χειρισμό, λειτουργώντας ως σημείο αναφοράς για την τοπολογία και τα δεδομένα.

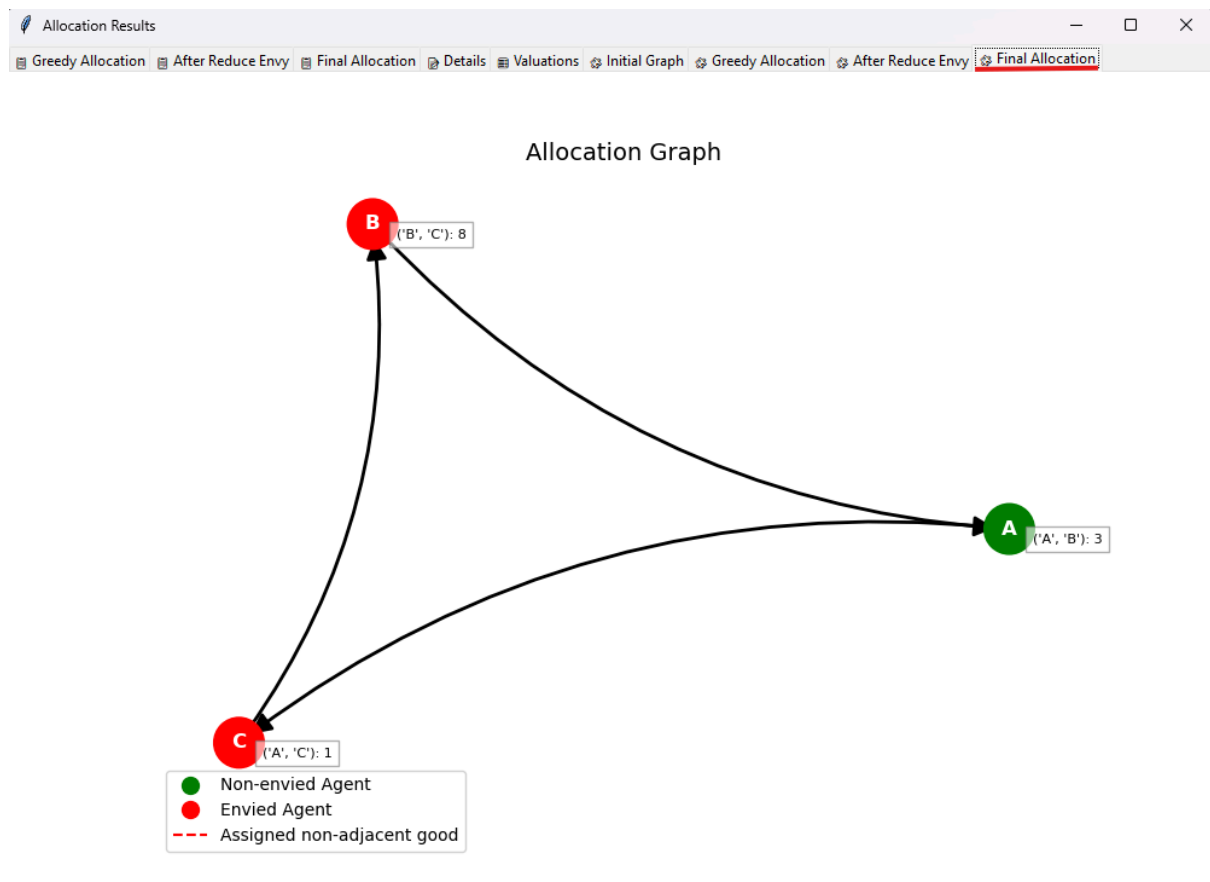


Allocation Graph



Allocation Graph





Εικόνα 4.3.4.2-4: Greedy Allocation / After Reduce Envy / Final Allocation:
κατευθυνόμενες ακμές όπου το βέλος δείχνει σε ποιον agent έχει κατανεμηθεί το αγαθό.

Στα εν λόγω tabs παρουσιάζονται γραφικές απεικονίσεις της κατανομής ανά φάση, όπως αυτές ορίστηκαν στο *Κεφάλαιο 3*. Τα tabs **Greedy Allocation**, **After Reduce Envy** και **Final Allocation** απεικονίζουν διαδοχικά την κατάσταση του συστήματος μετά την εφαρμογή των αντίστοιχων φάσεων του αλγορίθμου, ώστε να καθίσταται ορατή η εξέλιξη της κατανομής από το αρχικό στάδιο έως το τελικό αποτέλεσμα. Σε όλες τις απεικονίσεις εφαρμόζονται ομοιόμορφα οι συμβάσεις της 3.2.

ΚΕΦΑΛΑΙΟ 5: Πειραματική Ανάλυση

Στο κεφάλαιο αυτό παρουσιάζονται και αναλύονται τα πειραματικά αποτελέσματα που προέκυψαν από την υλοποίηση και εφαρμογή του αλγορίθμου κατανομής EFX. Ο κύριος σκοπός των πειραμάτων είναι να αξιολογηθεί η συμπεριφορά του αλγορίθμου σε διαφορετικές τοπολογίες δικτύου (τυχαίοι γράφοι, κύκλοι και δέντρα) και να εξεταστεί η αποδοτικότητά του ως προς τον χρόνο εκτέλεσης, τη δικαιοσύνη (μέσω του ποσοστού ζήλιας) και την ποιότητα της λύσης σε σχέση με το θεωρητικά βέλτιστο άθροισμα αποτιμήσεων.

Επιπλέον, εξετάζεται πώς επηρεάζεται η απόδοση του αλγορίθμου από τη μεταβολή του εύρους αποτίμησης των ακμών (από 1–10 σε 1–100), ώστε να εκτιμηθεί η σταθερότητα και η ευαισθησία του σε διαφορετικές κλίμακες τιμών.

Κάθε συνδυασμός τοπολογίας, εύρους τιμών και κατανομής εκτελέστηκε **10 φορές** με διαφορετικούς τυχαίους seed, ώστε να υπολογιστούν αξιόπιστα στατιστικά μεγέθη και μέσοι όροι.

5.1 Στόχος και ερευνητικά ερωτήματα

Σκοπός της πειραματικής μελέτης είναι η εμπειρική αξιολόγηση του αλγορίθμου κατανομής με εγγύηση EFX σε γραφικά μοντέλα, ως προς:

1. **Αποδοτικότητα:** χρόνος εκτέλεσης ανά φάση και συνολικά.
2. **Εξέλιξη ζήλιας:** ποσοστό πρακτόρων που ζηλεύουν μετά από κάθε φάση (*Greedy* → *Reduce Envy* → *Final*).
3. **Δομική επίδραση:** ρόλος τοπολογίας (γενικοί γράφοι, δέντρα, κύκλοι), πλήθους κόμβων n και—στους γράφους—πυκνότητας δ .
4. **Ποιότητα λύσης:** λόγος συνολικής αξίας της κατανομής του αλγορίθμου, έναντι

$$\text{βέλτιστου αθροίσματος} \sum_{e=(u,w) \in E} \max(v_u(e), v_w(e)).$$

5.2 Πειραματική Διάταξη

5.2.1 Τοπολογίες και παράμετροι

- **Γενικοί γράφοι (graphs)** με n κόμβους και $m = \delta \cdot n(n - 1)/2$ ακμές, όπου $\delta \in [0.2, 0.4, 0.6, 0.8, 1.0]$ είναι η πυκνότητα του γράφου.
- **Δέντρα (trees)** με n κόμβους και $m = n - 1$ ακμές.
- **Κύκλοι (cycles)** με n κόμβους και $m = n$ ακμές.

Τα πειράματα γίνονται σε πολλαπλά μεγέθη n , όπου $n \in [10, 30, 60, 80, 100]$ (κλιμάκωση), επαναλήψεις ανά ρύθμιση (διαφορετικοί seeds) και τρεις **πιθανοτικές κατανομές τιμών**:

- **uniform:** ομοιόμορφα τυχαίες τιμές.
- **center:** συγκέντρωση γύρω από μεσαίες τιμές (ηπιότερες αντιθέσεις).
- **extremes:** διμορφία σε χαμηλές και υψηλές τιμές (ισχυρές αντιθέσεις).

5.2.2 Εύρος αποτιμήσεων

Εξετάζονται δύο σενάρια τιμών:

- **Σενάριο A:** Minimum Value: 1, Maximum Value: 10.
- **Σενάριο B:** Minimum Value: 1, Maximum Value: 100.

Όλες οι υπόλοιπες παράμετροι (τοπολογίες, λίστα n , δ , κατανομές, πλήθος επαναλήψεων, seeds) διατηρούνται ίδιες, ώστε η σύγκριση να απομονώνει το **εύρος τιμών**.

5.2.3 Μετρικές

Ανά run (και κατόπιν συγκεντρωτικά AVG ανά ρύθμιση) καταγράφονται:

- **Time_ms:** συνολικός χρόνος εκτέλεσης .
- **Envied_% ανά φάση:** ποσοστό envied agents σε μετά από κάθε φάση εκτέλεσης του αλγορίθμου (Envied_Greedy_%, Envied_Reduce_%, Envied_Final_%).
- **Unoriented_%:** ποσοστό ακμών που έχουν κατανομηθεί σε μη-προσκειμένους κόμβους.
- **OurTotal:** συνολικό άθροισμα αποτιμήσεων της τελικής κατανομής.
- **OptimalSum:** η μέγιστη συνολική αξία που πετυχαίνεται αν κάθε αγαθό δοθεί στον πράκτορα που το εκτιμά περισσότερο.

$$\sum_{e=(u,w) \in E} \max(v_u(e), v_w(e)).$$

- **OurOverOptimal_%:** $\frac{OurTotal}{OptimalSum} \cdot 100\%$
- **IsEFX:** έλεγχος ορθότητας τελικής κατανομής.

5.3 Αποτελέσματα ανά τοπολογία με εύρος αποτίμησης 1–10

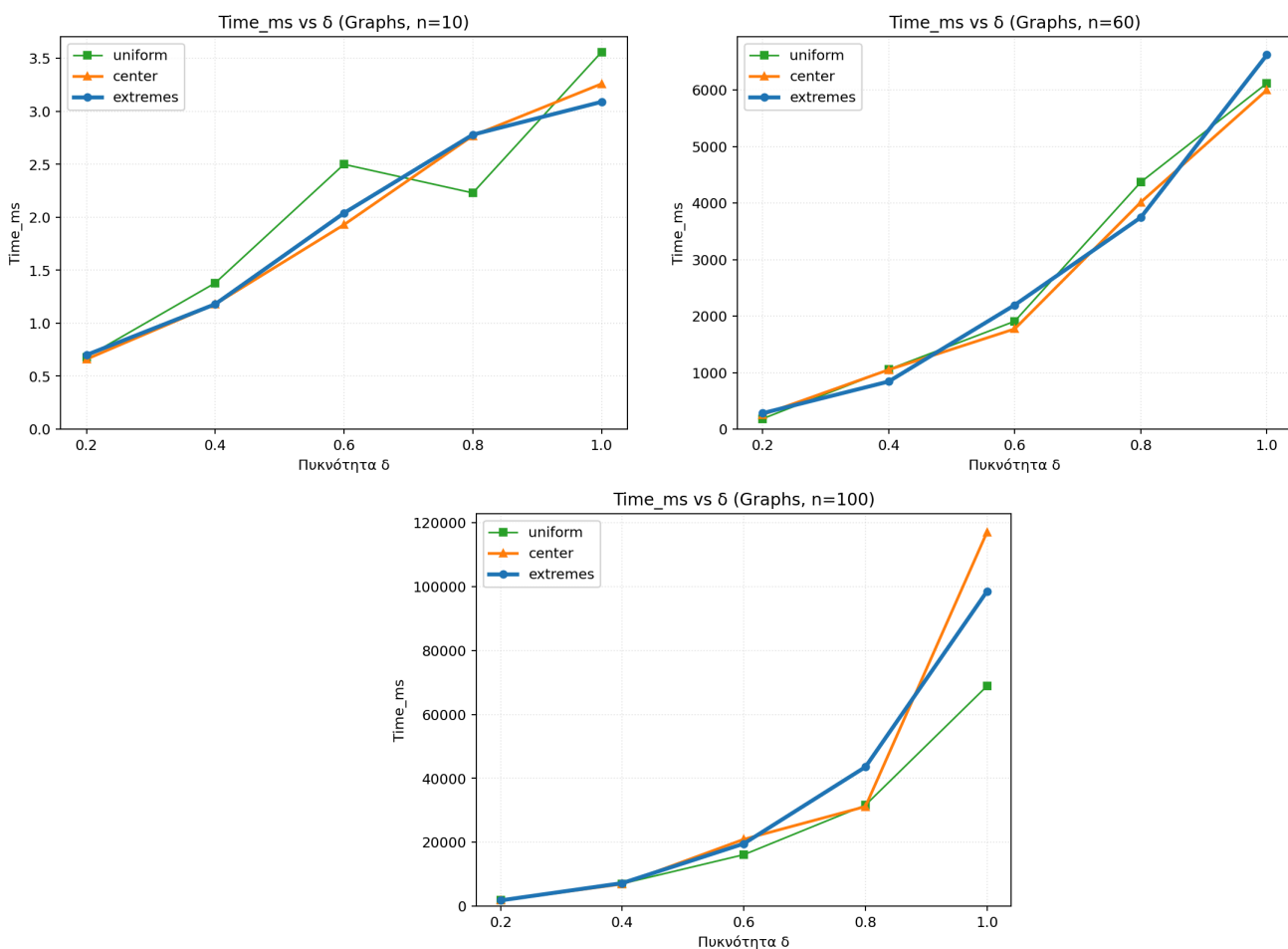
Σε αυτή την ενότητα παρουσιάζονται τα αποτελέσματα των πειραμάτων με αποτιμήσεις ακμών με εύρος 1–10.

Οι πίνακες και τα διαγράμματα που ακολουθούν δείχνουν πώς μεταβάλλονται οι τρεις βασικές μετρικές —ο χρόνος εκτέλεσης, το ποσοστό ζήλιας και η ποιότητα λύσης— ανάλογα με την τοπολογία, την πυκνότητα των ακμών και το πλήθος των κόμβων. Για κάθε τοπολογία υπολογίζονται οι μέσες τιμές των μετρικών για κάθε κατανομή τιμών.

Τα διαγράμματα δίνουν μια οπτική απεικόνιση των σχέσεων αυτών, ενώ οι πίνακες επιτρέπουν ακριβή ποσοτική σύγκριση.

5.3.1 Random Graphs

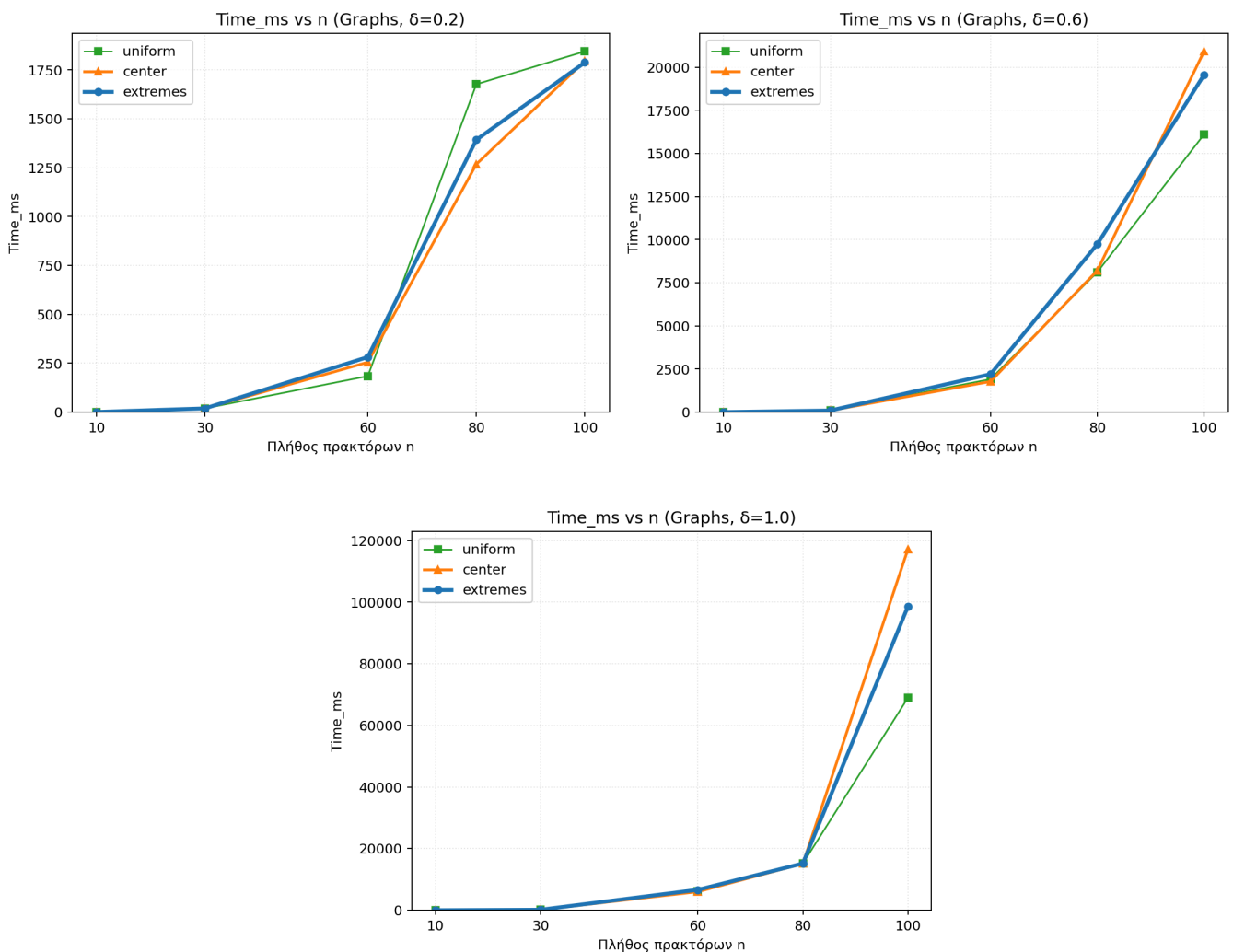
- Χρόνος (ms) ως προς την πυκνότητα γράφου



Εικόνα 5.3.1-3: Γραφήματα χρόνου σε ms ως προς την πυκνότητα δ των γράφων. Οι συγκεκριμένοι γράφοι έχουν πλήθος κόμβων 10, 60 και 100 αντίστοιχα.

Όπως φαίνεται στα παραπάνω γραφήματα (Εικόνα 5.3.1-3) χρόνος εκτέλεσης αυξάνεται μονοτονικά όσο πυκνώνει ο γράφος: περισσότερες ακμές σημαίνουν περισσότερα «υποψήφια» αγαθά και περισσότερους ελέγχους ζήλιας/διορθώσεων μέχρι να σταθεροποιηθεί η κατανομή. Οι τρεις κατανομές τιμών (uniform, center, extremes) παραμένουν σχετικά κοντά· το **δομικό μέγεθος** του προβλήματος (η ίδια η πυκνότητα) εξηγεί το μεγαλύτερο μέρος της αύξησης, ενώ οι διαφορές ανά κατανομή είναι δευτερεύουσες.

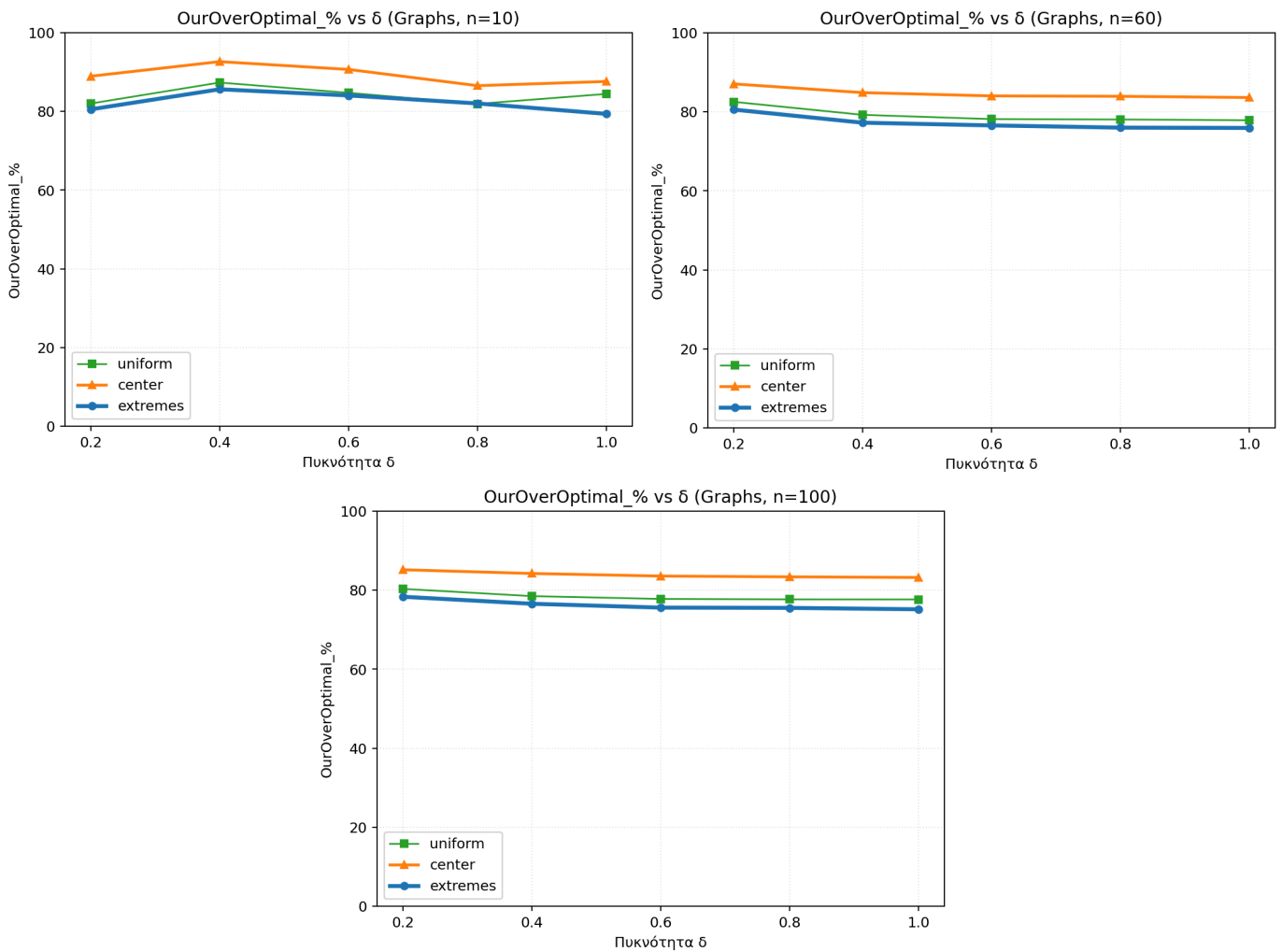
- **Χρόνος (ms) ως προς το πλήθος των κόμβων του γράφου**



Εικόνα 5.3.4-6: Γραφήματα χρόνου σε ms ως προς το πλήθος n των γράφων. Οι συγκεκριμένοι γράφοι έχουν πυκνότητα 0.2, 0.6 και 1 αντίστοιχα.

Η κλιμάκωση με το n είναι καθαρή· όσο μεγαλώνει το πλήθος πρακτόρων, ο χώρος αποφάσεων/ελέγχων διευρύνεται και ο χρόνος αυξάνεται προβλέψιμα. Για σταθερή δ , η κλίση είναι μεγαλύτερη σε πυκνότερους γράφους, ενώ οι καμπύλες των τριών κατανομών παραμένουν σχεδόν επικαλυπτόμενες—ένδειξη ότι η επίδραση του n κυριαρχεί έναντι του σχήματος της κατανομής τιμών.

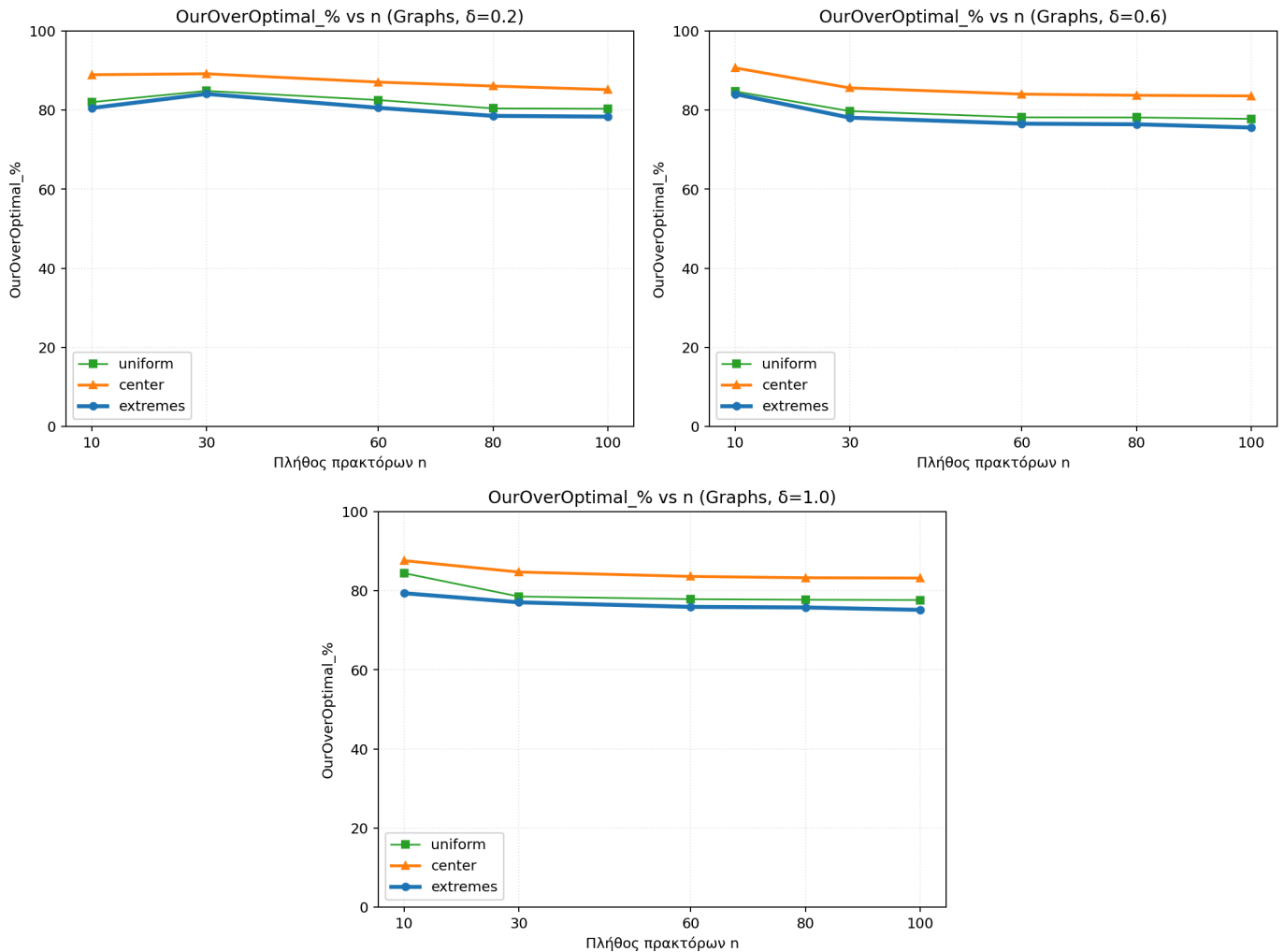
- Ποιότητα λύσης (OurOverOptimal_%) ως προς την πυκνότητα γράφου



Εικόνα 5.3.7-9: Γραφήματα ποσοστού ποιότητας λύσης vs πυκνότητα δ των γράφων. Οι συγκεκριμένοι γράφοι έχουν πλήθος κόμβων 10, 60 και 100 αντίστοιχα.

Όπως φαίνεται στις εικόνες (Εικόνα 5.3.7-9), η ποιότητα της λύσης (OurOverOptimal_%) παραμένει υψηλή αλλά μειώνεται ήπια με την αύξηση της δ . Η κατανομή center έχει σταθερά την καλύτερη επίδοση, ακολουθεί η uniform και τελευταία η extremes. Η πόλωση των τιμών στην extremes οδηγεί σε επιλογές που μεγιστοποιούν τοπικά την αξία αλλά δεν είναι πάντα συμβατές με το βέλτιστο.

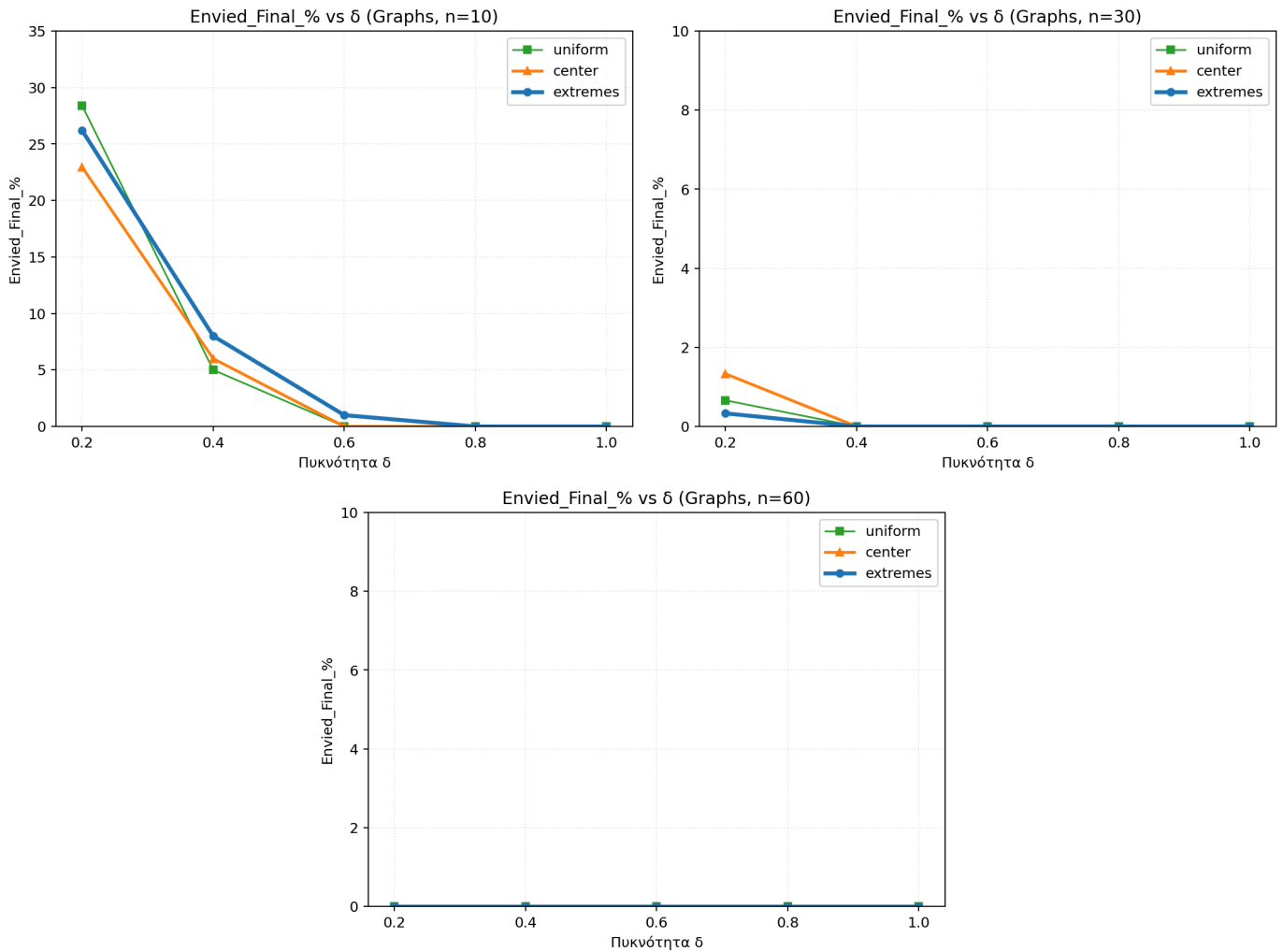
- Ποιότητα λύσης (**OurOverOptimal_%**) ως προς το πλήθος των κόμβων του γράφου



Εικόνα 5.3.10-12: Γραφήματα ποσοστού ποιότητας λύσης vs πλήθος n των γράφων. Οι συγκεκριμένοι γράφοι έχουν πυκνότητα 0.2, 0.6 και 1 αντίστοιχα.

Παρατηρείται ήπια μείωση της ποιότητας όσο αυξάνεται το n , χωρίς όμως να χάνεται η συνολικά υψηλή επίδοση. Και εδώ η **center** διατηρεί ένα μικρό προβάδισμα, η **uniform** ακολουθεί, ενώ η **extremes** υστερεί ελαφρά.

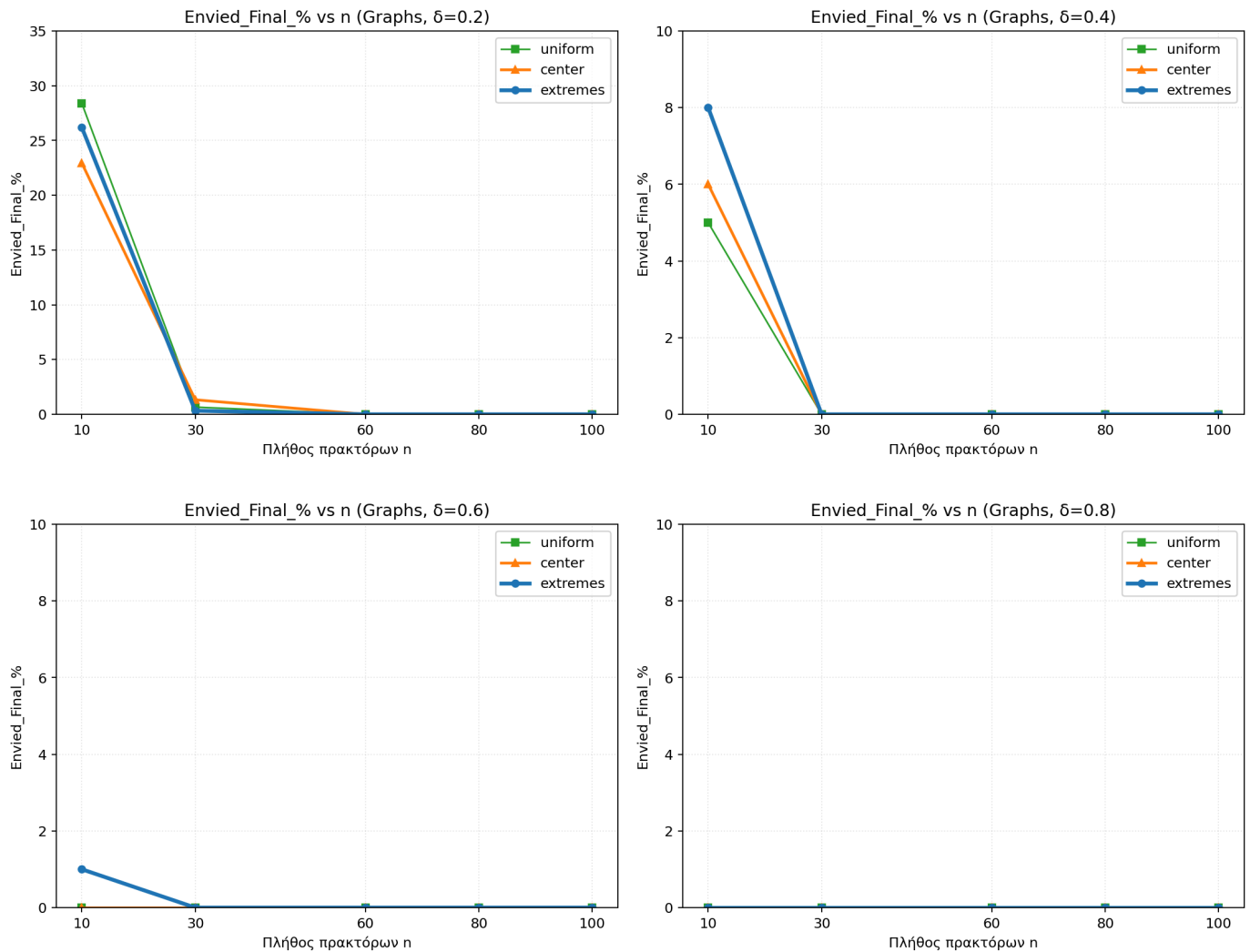
- Ποσοστό *envied* πρακτόρων (*Envied_Final_%*) ως προς την πυκνότητα γράφου



Εικόνα 5.3.13-15: Γραφήματα ποσοστού *envied* πρακτόρων ως προς την πυκνότητα δ των γράφων. Οι συγκεκριμένοι γράφοι έχουν πλήθος κόμβων 10, 30 και 60 αντίστοιχα.

Το τελικό ποσοστό ζήλιας φθίνει καθαρά με την πυκνότητα και τείνει σε μηδενισμό σε πολύ πυκνούς/πλήρεις γράφους (Εικόνα 5.3.13-15). Η υψηλή συνδεσιμότητα προσφέρει πολλές εναλλακτικές «διεξόδους» στη φάση διόρθωσης, οπότε και οι τρεις κατανομές συγκλίνουν προς πολύ μικρές τιμές. Σε αραιούς γράφους, η **center** έχει συνήθως το ελαφρώς καλύτερο προφίλ.

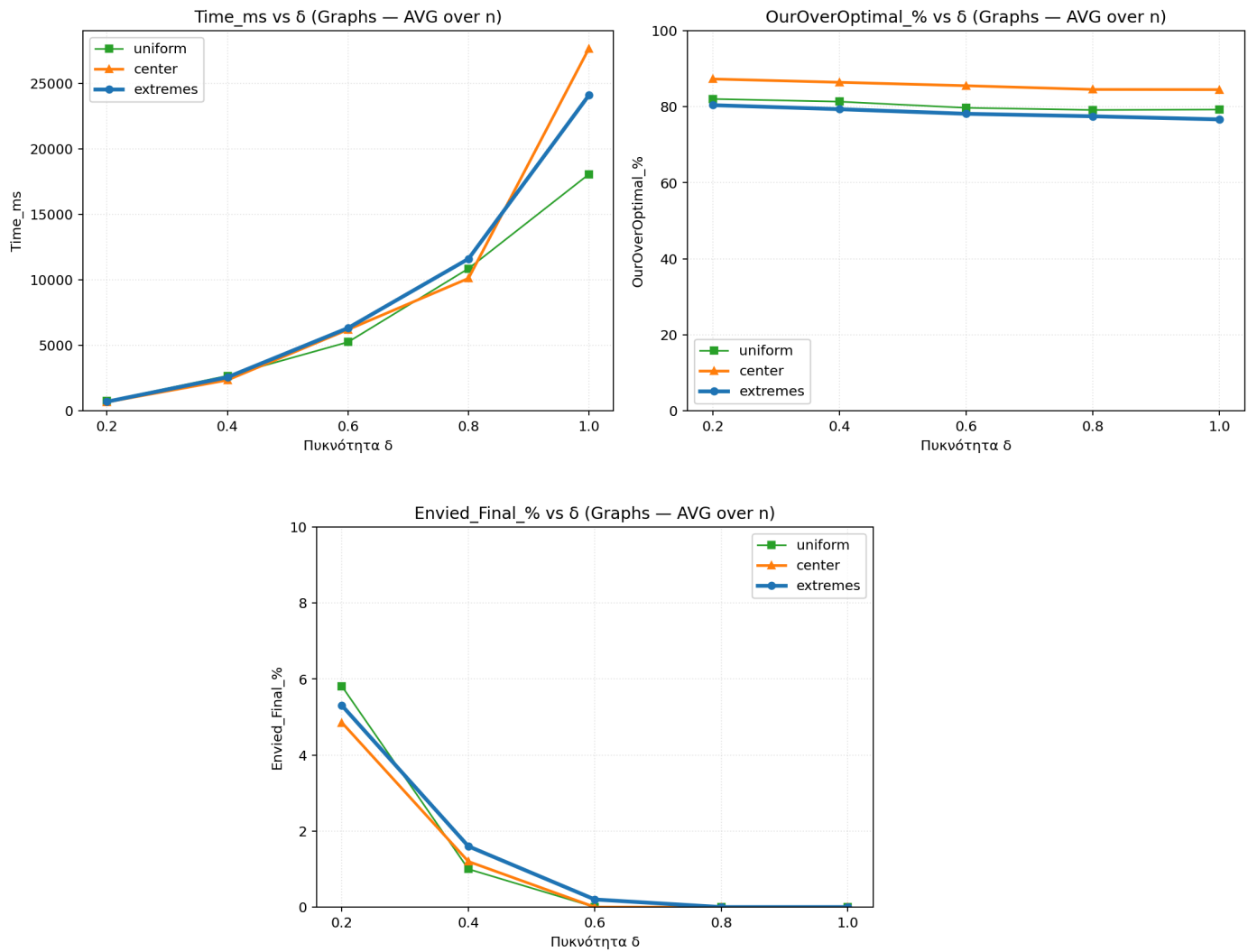
- Ποσοστό envied πρακτόρων (Envied_Final_%) ως προς το πλήθος κόμβων του γράφου



Εικόνα 5.3.16-19: Γραφήματα ποσοστού envied πρακτόρων ως προς το πλήθος κόμβων n του γράφου. Οι συγκεκριμένοι γράφοι έχουν πυκνότητα 0.2, 0.4, 0.6 και 0.8 αντίστοιχα.

Η τελική ζήλια μειώνεται όσο αυξάνεται το n . Σε μεγαλύτερα γραφήματα, ιδίως όταν η συνδεσιμότητα είναι επαρκής, ο αλγόριθμος βρίσκει ευκολότερα τελικές αναθέσεις χωρίς ζήλια. Οι διαφορές ανά κατανομή παραμένουν μικρές.

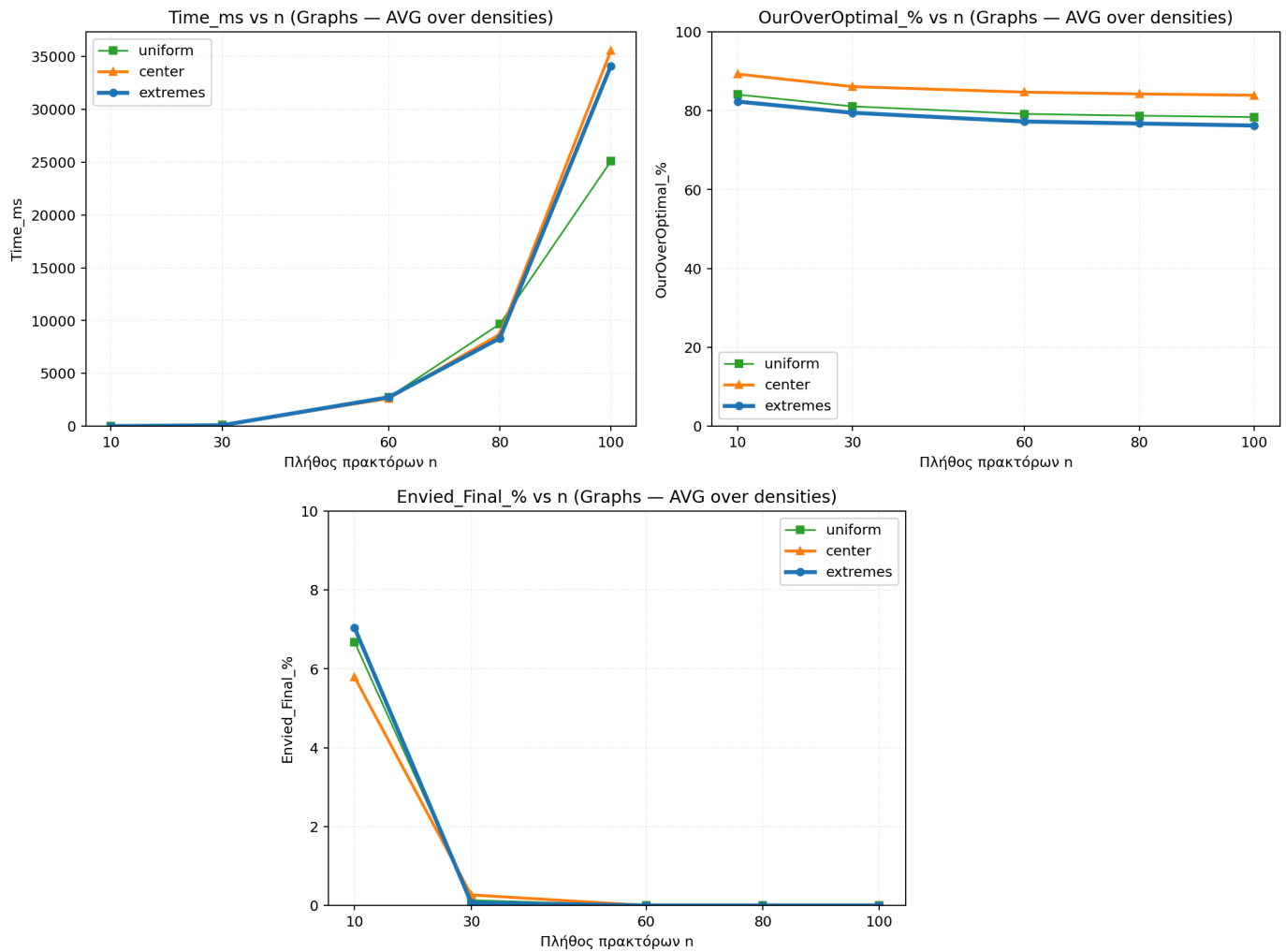
- Γραφήματα ΜΟ για όλα τα πλήθι πρακτόρων vs πυκνότητα των γράφων



Εικόνα 5.3.20-22: Μέσος όρος χρόνου σε ms, ποσοστού ποιοτικής λύσης και ποσοστού envied πρακτόρων vs πυκνότητα.

Οι μέσοι όροι ανά πυκνότητα συνοψίζουν τις τάσεις: (α) ο χρόνος αυξάνεται μονοτονικά με τη δ , (β) η ποιότητα παραμένει υψηλή με ήπια πτώση και (γ) το Envied_Final_% μειώνεται προς το 0. Η center κρατά σταθερά το καλύτερο ισοζύγιο χαμηλής ζήτησης/υψηλής ποιότητας. Τα plots αυτά είναι ο Μ.Ο. όλων των πληθών έναντι της πυκνότητας των γράφων.

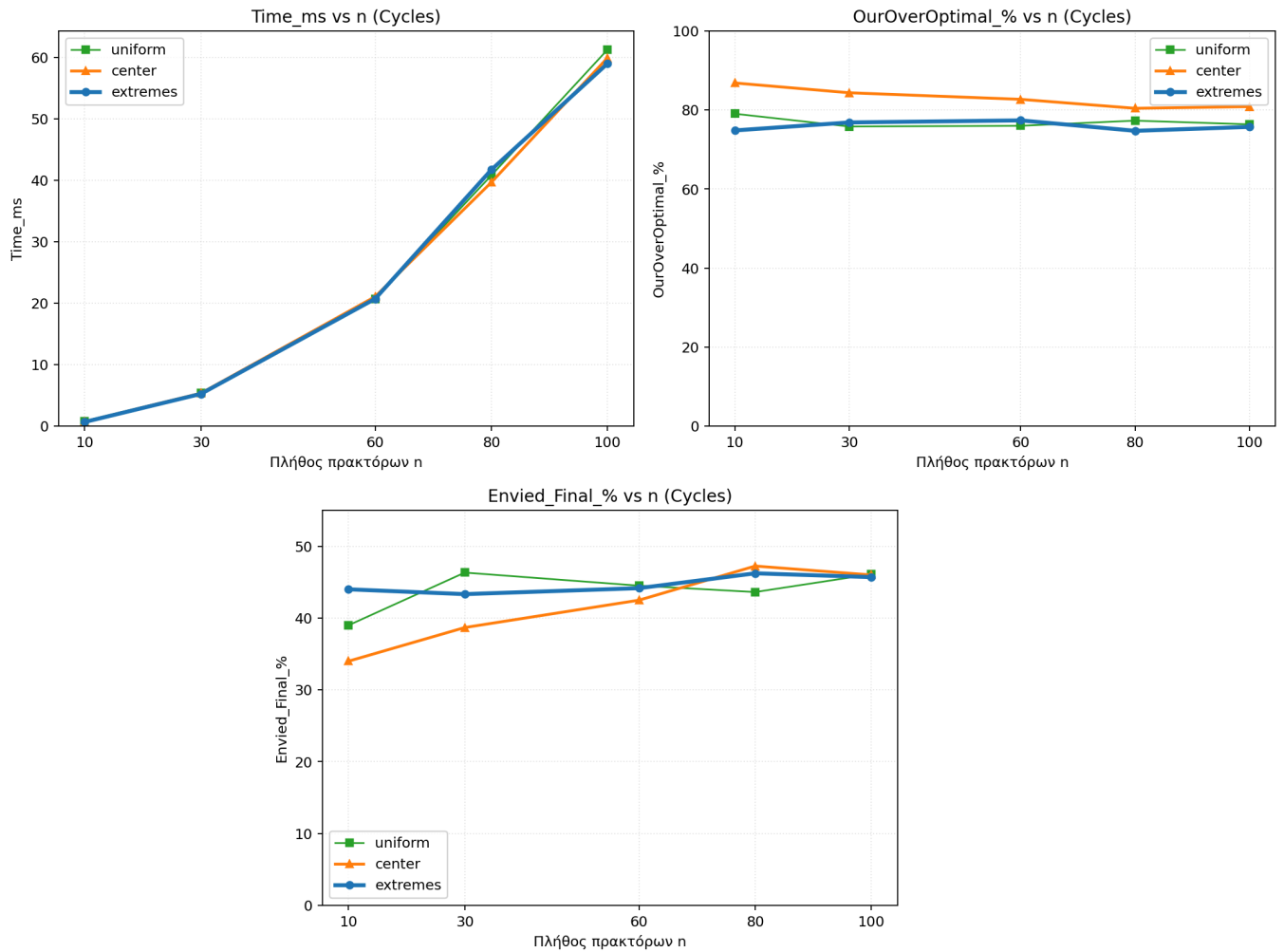
- Γραφήματα ΜΟ για όλες τις πυκνότητες γράφων vs πλήθος πρακτόρων



Εικόνα 5.3.23-25: Μέσος όρος χρόνου σε ms, ποσοστού ποιοτικής λύσης και ποσοστού *envied* vs πλήθος πρακτόρων.

Αντιστρόφως, οι μέσοι όροι ανά n δείχνουν καθαρή κλιμάκωση χρόνου με το μέγεθος, υποχώρηση της ζήτησης, καθώς μεγαλώνει ο γράφος και ήπια μείωση της ποιότητας—πάντα με τη **center** να προηγείται οριακά. Τα plots αυτά είναι είναι ο Μ.Ο. όλων των πυκνοτήτων έναντι του πλήθους των πρακτόρων.

5.3.2 Cycles



Εικόνα 5.3.26-28: Χρόνος σε ms, ποσοστό ποιοτικής λύσης και Ποσοστό envied πρακτόρων vs πλήθος πρακτόρων.

Στους κύκλους, η διαδικασία ολοκληρώνεται **ήδη μετά τη φάση Greedy**: ο προσανατολισμός των ακμών κατά Greedy αποδίδει κάθε ακμή σε έναν από τους δύο γείτονες, με αποτέλεσμα να κατανέμονται όλες οι ακμές και, κατ' επέκταση, να μη «τρέχουν» οι φάσεις **Reduce Envy** και **Allocate Remaining Goods**. Άρα, τα μεγέθη που απεικονίζονται στα σχήματα αντανakλούν καθαρά τη λύση του Greedy, χωρίς μεταγενέστερη επεξεργασία.

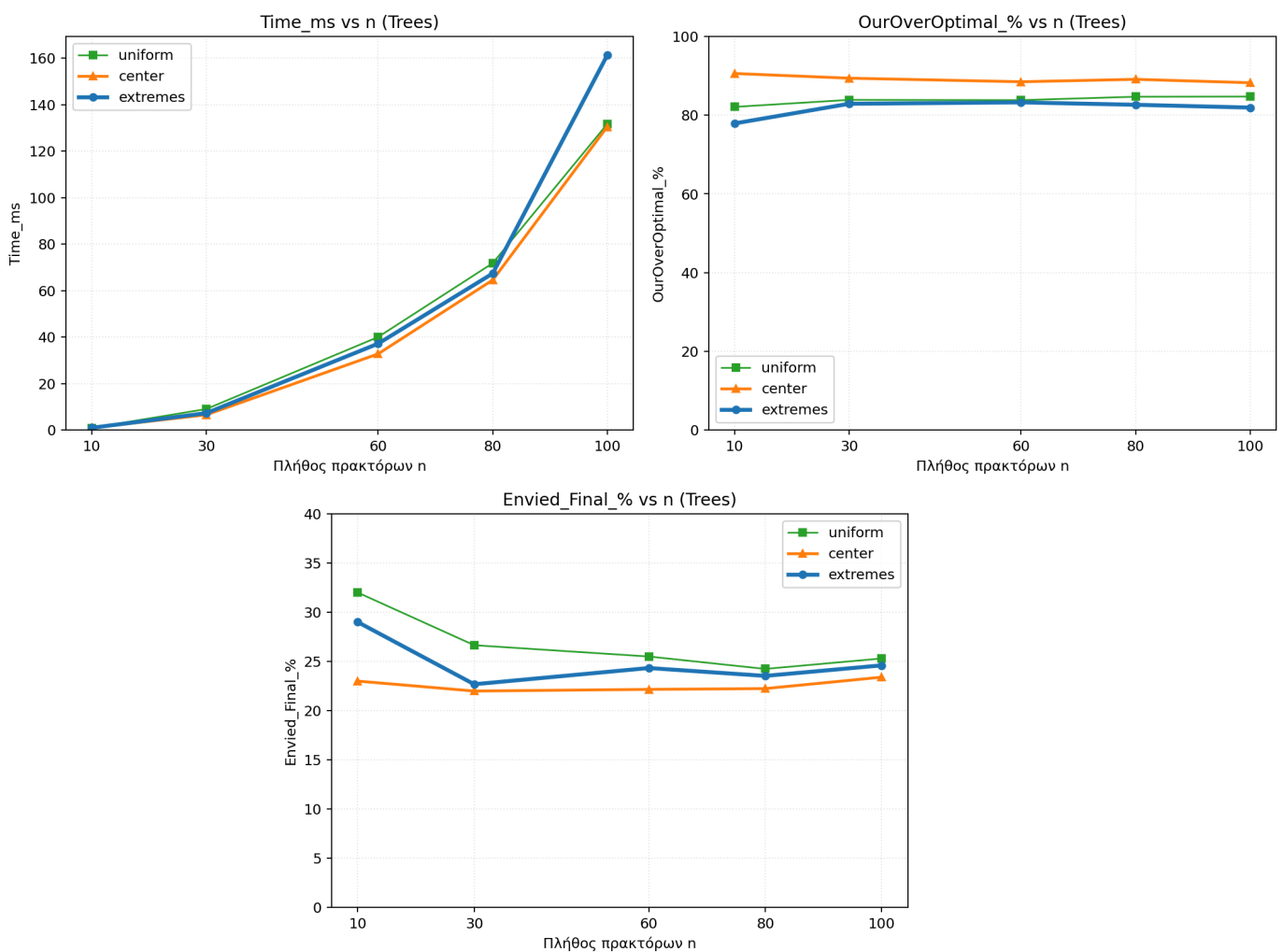
Επειδή τερματίζουμε νωρίς, οι χρόνοι παραμένουν πολύ χαμηλοί και αυξάνονται ήπια με το n (Εικόνα 5.3.26). Η τοπολογία κύκλου (βαθμός 2) περιορίζει τον χώρο επιλογών και, ελλείψει μεταγενέστερων διορθωτικών φάσεων, το συνολικό κόστος είναι μικρό. Οι τρεις κατανομές τιμών (uniform, center, extremes) εμφανίζονται σχεδόν επικαλυπτόμενες.

Χωρίς φάσεις εξισορρόπησης μετά το Greedy, η ζήλια παραμένει **μέτρια** (περίπου 42–45% σε μέσο όρο)(Εικόνα 5.3.27). Αυτό δεν είναι ανωμαλία· είναι **δομικό αποτέλεσμα** του τερματισμού στην Greedy: σε κύκλους οι «ισχυρές» ακμές τείνουν να συσσωρεύονται

τοπικά, χαρίζοντας –χωρίς δεύτερη σκέψη– πλεονέκτημα σε συγκεκριμένους πράκτορες. Συνήθως η **center** δίνει οριακά χαμηλότερη ζήλια από uniform/extremes, διότι οι μεσαίες αποτιμήσεις περιορίζουν ακραίες ανισοροπίες που το Greedy θα μονιμοποιούσε.

Παρά την απουσία «μετα-επεξεργασίας», η ποιότητα παραμένει **υψηλή** (~76–83%)(Εικόνα 5.3.28). Η **center** υπερέχει, η **uniform** ακολουθεί, και η **extremes** υστερεί ελαφρά. Η πόλωση της extremes παράγει λίγες «υπερ-ακριβές» ακμές που το Greedy κλειδώνει, αφήνοντας όμως κενά αλλού –και χωρίς επόμενη φάση να «μαζέψει» τις απώλειες.

5.3.3 Random Trees



Εικόνα 5.3.29-31: Χρόνος σε ms, ποσοστό ποιοτικής λύσης και Ποσοστό *enviéd* πρακτόρων vs πλήθος πρακτόρων

Στα δέντρα ο χρόνος εκτέλεσης παραμένει χαμηλός και αυξάνεται ομαλά με το πλήθος των κόμβων (όπως φαίνεται στην Εικόνα 5.3.29). Αυτό συμβαίνει επειδή η τοπολογία του δέντρου έχει **n-1 ακμές και δεν περιέχει καθόλου κύκλους**, άρα ο αλγόριθμος έχει λιγότερες επιλογές να εξετάσει και λιγότερα πιθανά «μονοπάτια ζήλιας». Η υπολογιστική

πολυπλοκότητα παραμένει μικρή, και η Greedy μπορεί να αποδώσει τις περισσότερες ακμές χωρίς να απαιτηθούν πολλές διορθώσεις στις επόμενες φάσεις.

Η ποιότητα λύσης παραμένει υψηλή σε όλο το εύρος του n (Εικόνα 5.3.30). Η δομή δέντρου, παρότι περιορίζει τις διορθώσεις, δεν εμποδίζει τη Greedy να εντοπίζει γρήγορα αρκετά ποιοτικές αναθέσεις (τοπικά συμβατές με το υπόλοιπο γράφημα). Εδώ ξεχωρίζει η κατανομή **center**, που τείνει να παράγει πιο ισορροπημένα σύνολα αποτιμήσεων· μειώνει την πιθανότητα να συγκεντρωθούν υπερ-ακριβές ακμές σε λίγους πράκτορες, άρα κρατά ψηλά το συνολικό άθροισμα, χωρίς να προκαλεί απότομες ανισορροπίες που δύσκολα διορθώνονται χωρίς κύκλους. Έπεται η **uniform** και με μικρή απόκλιση η **extremes**, γεγονός που οφείλεται στην πόλωση τιμών, καθώς «κλειδώνει» δύσκαμπτες επιλογές στις πρώιμες φάσεις.

Η ζήλια είναι περίπου 23–27% (Εικόνα 5.3.31). Η έλλειψη κύκλων συνεπάγεται ότι για να μεταφερθεί αξία υπάρχει μόνο ένα απλό μονοπάτι ανάμεσα σε δύο πράκτορες. Οι δυνατές διορθώσεις μετά το Greedy είναι λιγότερες από ότι σε γενικούς γράφους. Χωρίς εναλλακτικά κλειστά μονοπάτια, οι τοπικές ανισορροπίες δεν μπορούν πάντα να εξισορροπηθούν χωρίς να προκύψει νέα ζήλια από κάποιον πράκτορα. Παρόλα αυτά, επειδή το γράφημα είναι αραιό και η δυναμική του απλή, το ποσοστό ζήλιας συγκρατείται σε επίπεδα σημαντικά χαμηλότερα από τους κύκλους.

5.4 Συγκριτική ανάλυση: εύρος αποτίμησης 1–10 έναντι 1–100

Στην ενότητα αυτή συγκρίνουμε συστηματικά την επίδοση του αλγορίθμου όταν οι αποτιμήσεις των ακμών ανήκουν στο εύρος **1–10** έναντι του **1–100**. Για να απομονωθεί η επίδραση της κλίμακας τιμών, όλες οι λοιπές παράμετροι (τοπολογία, n , πυκνότητα δ στους γράφους, κατανομή τιμών, αριθμός επαναλήψεων) διατηρούνται ίδιες με την προηγούμενη ενότητα. Τα αποτελέσματα παρουσιάζονται με **συνοπτικούς πίνακες** (συγκεντρωτικοί μέσοι όροι, ανά δ , ανά n). Ως βασικούς δείκτες χρησιμοποιούμε: **Time_ms**, **Envied_Final_%**, **OurOverOptimal_%**.

Οι πίνακες που ακολουθούν περιέχουν για κάθε τοπολογία (γράφους, κύκλους και δέντρα) τους **μέσους όρους** των βασικών μετρικών για τα δύο εύρη τιμών και δύο επιπλέον στήλες, Δ και $\Delta\%$, οι οποίες δείχνουν τη μεταβολή της αντίστοιχης μετρικής όταν περνάμε από 1–10 σε 1–100.

- Η στήλη **[1–10]** δείχνει τον μέσο όρο των τιμών της μετρικής (τίτλος πίνακα) στο εύρος αποτιμήσεων 1-10.
- Η στήλη **[1–100]** δείχνει τον αντίστοιχο μέσο όρο στο εύρος αποτιμήσεων 1-100.
- Η στήλη Δ αντιπροσωπεύει τη **διαφορά** μεταξύ των δύο, δηλαδή $\Delta = [1-100] - [1-10]$.
- Η στήλη $\Delta\%$ δείχνει το **ποσοστό μεταβολής** σε σχέση με το εύρος 1-10, υπολογισμένο ως $\Delta\% = \frac{\Delta}{[1-10]} \cdot 100$.

Θετικές τιμές του $\Delta\%$ δηλώνουν **αύξηση** της μετρικής στο εύρος 1–100, ενώ αρνητικές τιμές δηλώνουν **μείωση**. Οι διαφορές αυτές αναλύονται στη συνέχεια ξεχωριστά για κάθε τοπολογία, συνοδευόμενες από σχόλια.

5.4.1 Random Graphs (1-10 vs 1-100)

- Πίνακες **MO όλων των n και δ**

Time_ms (AVG όλων των n & δ)				
Κατανομή	[1–10]	[1–100]	Δ	$\Delta\%$
uniform	7516.7	10964.6	3447.9	45.9
center	9398.7	11288.6	1889.9	20.1
extremes	9055.7	9453.5	397.8	4.4

Πίνακας 5.4.1: Μέσοι χρόνοι εκτέλεσης ανά κατανομή (*uniform*, *center*, *extremes*) και μεταβολές Δ , $\Delta\%$ από [1–10] σε [1–100], από όλα τα n και όλες τις πυκνότητες δ .

OurOverOptimal_% (AVG όλων των n & δ)				
Κατανομή	[1–10]	[1–100]	Δ	Δ%
uniform	80.3	79.04	-1.26	-1.56
center	85.66	83.35	-2.31	-2.7
extremes	78.41	75.63	-2.78	-3.54

Πίνακας 5.4.2: Μέσος λόγος (σε %) του Our/Optimal ανά κατανομή, από [1–10] σε [1–100], από όλα τα n και όλες τις δ.

Envied_Final_% (AVG όλων των n & δ)				
Κατανομή	[1–10]	[1–100]	Δ	Δ%
uniform	1.36	1.59	0.22	16.51
center	1.21	1.25	0.04	3.3
extremes	1.42	1.82	0.4	28.29

Πίνακας 5.4.3: Τελικό ποσοστό ζήλιας ανά κατανομή, με Δ, Δ% από [1–10] σε [1–100], μεσοποιημένα σε όλα τα n και όλες τις δ.

Στους γενικούς γράφους, ο αλγόριθμος καλείται να επιτύχει EFX κατανομή σε τοπολογίες αυθαίρετης πυκνότητας, όπου οι σχέσεις ζήλιας μπορεί να είναι πολύπλοκες. Τα πειραματικά αποτελέσματα στον Πίνακα 5.4.1 δείχνουν ότι ο **χρόνος εκτέλεσης (Time_ms)** αυξάνεται καθαρά με το πέρασμα στο εύρος 1–100. Η αύξηση είναι μεγαλύτερη για την κατανομή **uniform** (περίπου +46%), μετριότερη για τη **center** (+20%) και σχεδόν αμελητέα για την **extremes** (+4%). Η συμπεριφορά αυτή εξηγείται από το γεγονός ότι η Greedy, με μεγαλύτερη διασπορά τιμών, πρέπει να εξετάσει περισσότερα συγκρουόμενα «τοπικά μέγιστα» πριν καταλήξει σε σταθερή επιλογή. Οι επιπλέον φάσεις (*Reduce Envy, Allocate Remaining*) επίσης χρειάζονται περισσότερους ελέγχους για να αποκαταστήσουν ισορροπία.

Στον Πίνακα 5.4.2 παρουσιάζεται η **ποιότητα της λύσης (OurOverOptimal_%)**: αυτή μειώνεται ήπια, κατά 1–3 ποσοστιαίες μονάδες στο εύρος 1–100. Αυτό δείχνει ότι, αν και η συνολική αξία της κατανομής αυξάνεται απόλυτα, η αναλογία της προς τη βέλτιστη λύση μειώνεται ελαφρώς. Η κατανομή **center** εξακολουθεί να υπερτερεί, καθώς οι τιμές της συγκεντρώνονται κοντά στο μέσο του εύρους, αποφεύγοντας ακραία κλειδώματα που θα δυσκόλευαν την εξισορρόπηση.

Η **τελική ζήλια (Envied_Final_%)**, όπως φαίνεται στον Πίνακα 5.4.3, παραμένει εξαιρετικά χαμηλή και στις δύο περιπτώσεις. Μικρές αυξήσεις παρατηρούνται κυρίως για αραιές πυκνότητες ($\delta \leq 0.4$), όπου λίγες ακμές υψηλής αξίας μπορεί να προκαλέσουν τοπικά φαινόμενα ανισορροπίας που δεν διορθώνονται πλήρως.

• Πίνακες ΜΟ των n ανά δ

Time_ms ανά πυκνότητα δ (AVG των n)						Envied_Final_% ανά δ (AVG των n)			
Πυκνότητα δ	Κατανομή	[1–10]	[1–100]	Δ	Δ%	[1–10]	[1–100]	Δ	Δ%
0.2	uniform	745.1	1410.1	665.0	89.2	5.81	6.33	0.52	09.02
0.2	center	667.7	1161.3	493.6	73.9	4.85	5.65	0.8	16.48
0.2	extremes	696.5	1187.3	490.8	70.5	5.3	6.71	1.41	26.58
0.4	uniform	2672.2	5195.1	2522.9	94.4	1.0	1.2	0.2	20.0
0.4	center	2346.7	5051.0	2704.3	115.2	1.2	0.6	-0.6	-50.0
0.4	extremes	2562.6	4809.1	2246.5	87.7	1.6	2.0	0.4	25.0
0.6	uniform	5244.7	10997.2	5752.5	109.7	0.0	0.4	0.4	-
0.6	center	6198.6	9820.5	3621.9	58.4	0.0	0.0	0.0	-
0.6	extremes	6316.7	10943.8	4627.1	73.3	0.2	0.4	0.2	100.0
0.8	uniform	10857.9	18778.1	7920.2	72.9	0.0	0.0	0.0	-
0.8	center	10111.1	18941.4	8830.3	87.3	0.0	0.0	0.0	-
0.8	extremes	11602.8	12095.8	493.0	4.2	0.0	0.0	0.0	-
1.0	uniform	18063.7	18442.7	379.0	2.1	0.0	0.0	0.0	-
1.0	center	27669.6	21469.1	-6200.6	-22.4	0.0	0.0	0.0	-
1.0	extremes	24099.8	18231.4	-5868.4	-24.4	0.0	0.0	0.0	-

Πίνακας 5.4.4-5: Χρόνος ανά $\delta \in \{0.2, 0.4, 0.6, 0.8, 1.0\}$ για κάθε κατανομή, με [1–10], [1–100], Δ, Δ% και τελική ζήλια ανά δ, με [1–10], [1–100], Δ, Δ%.

Αρχικά, ο χρόνος εκτέλεσης αυξάνεται όσο αυξάνεται η πυκνότητα των ακμών. Για μικρές τιμές δ (0.2–0.4), οι χρόνοι είναι χαμηλοί και για τα δύο εύρη αποτιμήσεων. Στις μεσαίες πυκνότητες (0.4–0.6) παρατηρείται η μεγαλύτερη διαφορά μεταξύ 1–10 και 1–100, καθώς εκεί ο αλγόριθμος καλείται να διαχειριστεί αρκετές επιλογές χωρίς να υπάρχει πλήρης συνδεσιμότητα που θα διευκόλυνε την επίλυση. Αντίθετα, για υψηλές τιμές δ (0.8–1.0), η διαφορά μειώνεται και σε ορισμένες περιπτώσεις ο χρόνος με 1–100 είναι ελαφρώς μικρότερος. Αυτό οφείλεται στο ότι, σε πλήρεις γράφους, οι αποφάσεις της Greedy είναι πιο ξεκάθαρες και λιγότερο ευαίσθητες σε μικρές μεταβολές τιμών.

Όπως φαίνεται στον Πίνακα 5.4.5, το ποσοστό ζήτησης μειώνεται δραματικά καθώς αυξάνεται η δ. Για $\delta \geq 0.6$ η ζήλια είναι σχεδόν μηδενική και στα δύο εύρη αποτιμήσεων. Η μικρή αύξηση που εμφανίζεται με 1–100 στις χαμηλές δ είναι αποτέλεσμα της μεγαλύτερης

διαφοροποίησης των αποτιμήσεων, που κάνει τις πρώτες επιλογές της Greedy λιγότερο ισορροπημένες.

OurOverOptimal_% ανά δ (AVG των n)					
Πυκνότητα δ	Κατανομή	[1–10]	[1–100]	Δ	Δ%
0.2	uniform	82.04	81.94	-0.1	-0.12
0.2	center	87.3	84.95	-2.35	-2.69
0.2	extremes	80.42	77.85	-2.57	-3.19
0.4	uniform	81.34	80.02	-1.32	-1.62
0.4	center	86.43	84.25	-2.17	-2.51
0.4	extremes	79.35	76.93	-2.42	-3.05
0.6	uniform	79.71	78.47	-1.24	-1.56
0.6	center	85.53	83.19	-2.34	-2.74
0.6	extremes	78.14	75.35	-2.79	-3.57
0.8	uniform	79.15	77.69	-1.46	-1.84
0.8	center	84.54	82.29	-2.25	-2.66
0.8	extremes	77.49	74.14	-3.35	-4.33
1.0	uniform	79.26	77.09	-2.17	-2.73
1.0	center	84.5	82.05	-2.45	-2.9
1.0	extremes	76.67	73.9	-2.76	-3.61

Πίνακας 5.4.6: Ποιότητα λύσης (*OurOverOptimal_%*) ανά δ για [1–10] και [1–100], με Δ, Δ%.

Η ποιότητα της λύσης, σύμφωνα με τον Πίνακα 5.4.6, παραμένει υψηλή για όλες τις δ, με μικρή πτώση στις μεσαίες πυκνότητες στο εύρος 1–100. Η **center** συνεχίζει να επιτυγχάνει τις καλύτερες τιμές, αποδεικνύοντας ότι οι πιο ομαλές κατανομές βοηθούν τον αλγόριθμο να διατηρεί υψηλή απόδοση ακόμη και όταν αυξάνεται το εύρος τιμών.

Γενικότερα, η ανάλυση ανά δ δείχνει ότι η **πυκνότητα** παίζει πιο καθοριστικό ρόλο από το εύρος αποτιμήσεων. Όσο η τοπολογία γίνεται πυκνότερη, ο αλγόριθμος προσεγγίζει πολύ γρήγορα EFX κατανομές ανεξάρτητα από την κλίμακα των τιμών.

• Πίνακες MO των δ ανά n

Time_ms ανά πλήθος n (AVG των δ)						Envied_Final_% ανά πλήθος n (AVG των δ)			
Πλήθος n	Κατανομή	[1–10]	[1–100]	Δ	$\Delta\%$	[1–10]	[1–100]	Δ	$\Delta\%$
10	uniform	2.1	3.0	0.9	45.2	6.68	7.53	0.86	12.82
10	center	2.0	2.8	0.8	43.0	5.79	5.99	0.2	3.46
10	extremes	2.0	2.9	0.9	46.0	07.04	8.98	1.94	27.62
30	uniform	91.7	177.2	85.5	93.2	0.13	0.4	0.27	203.03
30	center	90.3	153.4	63.1	69.9	0.27	0.27	0.0	0.0
30	extremes	81.7	139.9	58.2	71.3	0.07	0.13	0.07	100.0
60	uniform	2728.6	3741.3	1012.7	37.1	0.0	0.0	0.0	-
60	center	2619.9	3495.7	875.8	33.4	0.0	0.0	0.0	-
60	extremes	2738.4	4187.4	1449.0	52.9	0.0	0.0	0.0	-
80	uniform	9666.2	15636.8	5970.6	61.8	0.0	0.0	0.0	-
80	center	8708.8	16418.8	7710.0	88.5	0.0	0.0	0.0	-
80	extremes	8326.5	15167.5	6841.1	82.2	0.0	0.0	0.0	-
100	uniform	25095.1	35264.9	10169.8	40.5	0.0	0.0	0.0	-
100	center	35572.9	36372.6	799.7	2.2	0.0	0.0	0.0	-
100	extremes	34130.1	27769.8	-6360.3	-18.6	0.0	0.0	0.0	-

Πίνακας 5.4.7-8: Χρόνος ανά $n \in \{10, 30, 60, 80, 100\}$ για κάθε κατανομή, με [1–10], [1–100], Δ , $\Delta\%$ και τελική ζήλια ανά n , με [1–10], [1–100], Δ , $\Delta\%$.

Ο χρόνος εκτέλεσης αυξάνεται αναμενόμενα με το πλήθος των κόμβων, καθώς αυξάνεται και ο αριθμός των ακμών που πρέπει να εξεταστούν. Στο εύρος 1–100 οι χρόνοι είναι σταθερά υψηλότεροι σε σχέση με το 1–10, με τη διαφορά να είναι εντονότερη για μεσαία και μεγάλα μεγέθη γράφων (n μεταξύ 50 και 80). Για πολύ μεγάλους γράφους (n κοντά στο 100), η διαφορά μειώνεται ελαφρά, καθώς η Greedy «κλειδώνει» γρηγορότερα τις ισχυρές ακμές, περιορίζοντας τον αριθμό των επαναλήψεων.

Η τελική ζήλια μειώνεται σταθερά με το n και τείνει στο μηδέν για μεγάλους γράφους και στα δύο εύρη αποτιμήσεων, όπως φαίνεται στον Πίνακα 5.4.8. Σε μικρά n η ζήλια είναι ελαφρώς υψηλότερη για το 1–100, γεγονός που υποδηλώνει ότι σε μικρότερα συστήματα η μεγάλη διασπορά τιμών έχει μεγαλύτερη επίδραση.

OurOverOptimal_% ανά n (AVG των δ)					
Πλήθος n	Κατανομή	[1–10]	[1–100]	Δ	Δ%
10	uniform	84.09	84.21	0.12	0.15
10	center	89.29	87.53	-1.76	-1.97
10	extremes	82.33	80.83	-1.5	-1.82
30	uniform	81.1	79.52	-1.58	-1.94
30	center	86.11	83.81	-2.31	-2.68
30	extremes	79.48	76.42	-3.06	-3.84
60	uniform	79.18	77.73	-1.45	-1.83
60	center	84.71	82.27	-2.44	-2.88
60	extremes	77.27	74.2	-3.07	-3.98
80	uniform	78.73	77.01	-1.72	-2.19
80	center	84.26	81.77	-2.49	-2.95
80	extremes	76.76	73.54	-3.22	-4.2
100	uniform	78.39	76.74	-1.65	-2.11
100	center	83.92	81.35	-2.56	-3.06
100	extremes	76.24	73.19	-3.05	-4.0

Πίνακας 5.4.9: Ποιότητα λύσης (OurOverOptimal_%) ανά n για [1–10] και [1–100], με Δ, Δ%.

Η ποιότητα της λύσης, σύμφωνα με τον Πίνακα 5.4.9, μειώνεται ήπια με την αύξηση του n και είναι συστηματικά λίγο χαμηλότερη στο εύρος 1–100. Η μεγαλύτερη απόκλιση παρατηρείται για μεσαία μεγέθη γράφων (n=40–80). Και εδώ, η κατανομή **center** παραμένει η πιο σταθερή, διατηρώντας την καλύτερη αναλογία Our/Optimal σε όλα τα μεγέθη.

5.4.2 Cycles (1-10 vs 1-100)

TIME (ms)				
Κατανομή	[1–10]	[1–100]	Δ	Δ%
uniform	25.8	13.9	-11.9	-46.2
center	25.3	13.6	-11.8	-46.5
extremes	25.5	14.1	-11.3	-44.5

Πίνακας 5.4.10: Μέσοι χρόνοι εκτέλεσης ανά κατανομή (uniform, center, extremes) και μεταβολές Δ, Δ% από [1–10] σε [1–100].

Envied_%				
Κατανομή	Κατανομή	[1–10]	[1–100]	Δ
uniform	uniform	43.91	49.28	5.36
center	center	41.68	48.3	6.62
extremes	extremes	44.68	47.21	2.53

Πίνακας 5.4.11: Μέσος λόγος (σε %) του *Our/Optimal* ανά κατανομή (*uniform*, *center*, *extremes*) και μεταβολές Δ, Δ% από [1–10] σε [1–100].

OurOverOptimal_%				
Κατανομή	[1–10]	[1–100]	Δ	Δ%
uniform	76.91	73.88	-3.03	-3.94
center	83.04	79.78	-3.25	-3.92
extremes	75.9	72.89	-3.01	-3.96

Πίνακας 5.4.12: Τελικό ποσοστό ζήλιας ανά κατανομή, με Δ, Δ% από [1–10] σε [1–100].

Στον Πίνακα 5.4.10 ο χρόνος εκτέλεσης μειώνεται δραστικά στο εύρος 1–100, καθώς η αυξημένη διασπορά τιμών εξαλείφει ισοβαθμίες και οδηγεί σε άμεσες αποφάσεις της Greedy. Ο μέσος χρόνος μειώνεται **περίπου 40–50 %**.

Ωστόσο, στον Πίνακα 5.4.11 το ποσοστό ζήλιας αυξάνεται ελαφρώς (περίπου 3–5 ποσοστιαίες μονάδες). Αυτό είναι αναμενόμενο, αφού δεν υπάρχει καμία διαδικασία *Reduce Envy* που να εξισορροπεί τις τοπικές ανισορροπίες. Αν μια ακμή μεγάλης αξίας δοθεί σε έναν πράκτορα, ο διπλανός του δεν έχει τρόπο να αποζημιωθεί χωρίς να παραβιαστεί η δομή του κύκλου.

Η **ποιότητα της λύσης** (*OurOverOptimal_%*) παρουσιάζει ήπια μείωση (περίπου 2–3 ποσοστιαίες μονάδες), όπως φαίνεται στον Πίνακα 5.4.12, αλλά παραμένει σε υψηλά επίπεδα (76–83%). Η **center** διατηρεί και εδώ την καλύτερη επίδοση, καθώς η μετριοπαθής κατανομή των τιμών οδηγεί σε πιο ισορροπημένες επιλογές.

5.4.3 Random Trees (1-10 vs 1-100)

TIME (MS)				
Κατανομή	[1–10]	[1–100]	Δ	$\Delta\%$
uniform	50.7	30.0	-20.6	-40.7
center	47.0	32.0	-15.0	-31.9
extremes	54.8	35.6	-19.2	-35.0

Πίνακας 5.4.13: Μέσοι χρόνοι εκτέλεσης ανά κατανομή (uniform, center, extremes) και μεταβολές Δ , $\Delta\%$ από [1–10] σε [1–100], από όλα τα n και όλες τις πυκνότητες δ .

Envied_%				
Κατανομή	[1–10]	[1–100]	Δ	$\Delta\%$
uniform	26.74	27.85	1.11	4.15
center	22.56	25.93	3.37	14.96
extremes	24.83	26.82	1.99	8.01

Πίνακας 5.4.14: Μέσος λόγος (σε %) του Our/Optimal ανά κατανομή (uniform, center, extremes) και μεταβολές Δ , $\Delta\%$ από [1–10] σε [1–100].

OurOverOptimal_%				
Κατανομή	[1–10]	[1–100]	Δ	$\Delta\%$
uniform	83.85	81.95	-1.9	-2.26
center	89.17	86.13	-3.04	-3.4
extremes	81.72	79.13	-2.59	-3.17

Πίνακας 5.4.15: Τελικό ποσοστό ζήλιας ανά κατανομή, με Δ , $\Delta\%$ από [1–10] σε [1–100].

Σύμφωνα με τον Πίνακα 5.4.13, ο χρόνος εκτέλεσης είναι **πολύ μικρότερος** στα δέντρα σε σχέση με τους γενικούς γράφους και μειώνεται ακόμη περισσότερο όταν οι αποτιμήσεις αυξάνονται στο εύρος 1–100. Αυτό οφείλεται στο ότι οι υψηλότερες και πιο διαφοροποιημένες τιμές επιτρέπουν στη Greedy να καταλήγει ταχύτερα στις βέλτιστες ακμές, χωρίς να απαιτούνται εκτεταμένες διορθώσεις.

Η **τελική ζήλια** (Πίνακας 5.4.14) παραμένει χαμηλή, αλλά παρουσιάζει μια μικρή αύξηση στο εύρος 1–100, περίπου 2–3 ποσοστιαίων μονάδων. Η αύξηση αυτή εξηγείται από τη φύση των δέντρων: κάθε ζεύγος κόμβων συνδέεται μόνο μέσω ενός μονοπατιού, επομένως

οι δυνατότητες ανακατανομής είναι περιορισμένες. Αν μια ακμή μεγάλης αξίας δοθεί σε έναν πράκτορα, οι υπόλοιποι δεν έχουν άλλη εναλλακτική διαδρομή για να αποζημιωθούν.

Η **ποιότητα της λύσης** (Πίνακας 5.4.15) είναι πολύ υψηλή, ξεπερνώντας το 80% και στις δύο περιπτώσεις. Με το εύρος 1–100, η μέση τιμή μειώνεται ελάχιστα, χωρίς ωστόσο να παρατηρείται ουσιαστική επιδείνωση. Η **center** κατανομή παρουσιάζει σταθερά τα καλύτερα αποτελέσματα, ενώ η *extremes* έχει τη χαμηλότερη ποιότητα αλλά και μικρότερη διακύμανση χρόνου.

5.5 Παρατήρηση για την απουσία unoriented ακμών

Σε όλα τα πειράματα που πραγματοποιήθηκαν, **δεν παρατηρήθηκε καμία unoriented ακμή**, δηλαδή δεν υπήρξε περίπτωση ακμής που να μην αποδόθηκε τελικά σε κάποιον πράκτορα ή να χρειάστηκε ανάθεση εκτός γειτνίασης. Το αποτέλεσμα αυτό δεν είναι τυχαίο, αλλά προκύπτει από τη σχεδίαση του αλγορίθμου:

1. Η **φάση Greedy** αποδίδει εξ αρχής σχεδόν όλες τις ακμές, επιλέγοντας πάντα τη μέγιστη διαθέσιμη αποτίμηση για κάθε πράκτορα.
2. Η **φάση Allocate Remaining Goods** βρίσκει με βεβαιότητα «ασφαλείς» πράκτορες μέσω των *safe sets*, οπότε κάθε ακμή τοποθετείται σε κάποιον χωρίς να δημιουργείται ζήλια.
3. Στην πράξη, ακόμα και όταν υπάρχουν ελάχιστες μη εκχωρημένες ακμές, οι πιθανότητες να μην υπάρχει ασφαλής γείτονας είναι εξαιρετικά μικρές, ειδικά σε τυχαίες τοπολογίες με μέτρια ή υψηλή συνδεσιμότητα.
4. Σε ειδικές τοπολογίες, όπως οι κύκλοι και τα δέντρα, η δομή του γράφου περιορίζει τα πιθανά σημεία ασυμφωνίας, οδηγώντας πάντα σε πλήρως προσανατολισμένες κατανομές.

Αν και θεωρητικά είναι δυνατό να υπάρξουν unoriented ακμές σε πολύ συγκεκριμένες περιπτώσεις (όπου όλες οι επιλογές προκαλούν ζήλια), στα πειράματα που εκτελέστηκαν δεν εμφανίστηκε καμία τέτοια περίπτωση. Αυτό υποδηλώνει ότι ο αλγόριθμος είναι **ιδιαίτερα σταθερός** και, για τα δεδομένα που χρησιμοποιήθηκαν, **παράγει πάντα πλήρη EFX κατανομή**.

5.6 Συμπεράσματα Πειραματικής Ανάλυσης

Από τη συνολική ανάλυση όλων των πειραμάτων προκύπτουν τα εξής βασικά συμπεράσματα:

1. Σε όλα τα πειράματα και για όλες τις τοπολογίες (γράφους, κύκλους, δέντρα), ο αλγόριθμος παρήγαγε πάντοτε EFX κατανομή. Η συνθήκη EFX ικανοποιήθηκε σε κάθε τελικό αποτέλεσμα, γεγονός που επιβεβαιώνει τη θεωρητική εγκυρότητα και την πρακτική αξιοπιστία του αλγορίθμου.
2. Η αύξηση του εύρους αποτιμήσεων από 1–10 σε 1–100 επηρεάζει μετρίως τις επιδόσεις: ο χρόνος εκτέλεσης αυξάνεται, ενώ η ποιότητα μειώνεται ελαφρά, χωρίς όμως να υποβαθμίζεται η δικαιοσύνη.
3. Η κατανομή center αποδεικνύεται η πιο σταθερή και αποδοτική, προσφέροντας την καλύτερη ισορροπία μεταξύ συνολικής αξίας και δικαιοσύνης.
4. Στους γενικούς γράφους, οι μετρικές σταθεροποιούνται σε πολύ υψηλά επίπεδα για μεγάλες τιμές πυκνότητας ($\delta \geq 0.6$).
5. Στους κύκλους, η Greedy φάση επαρκεί για να παραχθεί EFX λύση χωρίς επιπλέον διορθώσεις.
6. Στα δέντρα, η απουσία κύκλων μειώνει τις δυνατότητες εξισορρόπησης, αλλά δεν επηρεάζει την επίτευξη της EFX ιδιότητας.

Συνολικά, ο αλγόριθμος αποδείχθηκε αποτελεσματικός, κλιμακώσιμος και σταθερός σε όλες τις περιπτώσεις. Η πλήρης απουσία unoriented ακμών και η σταθερή επίτευξη EFX κατανομών σε όλα τα πειράματα επιβεβαιώνουν ότι το μοντέλο λειτουργεί αξιόπιστα ακόμη και σε τοπολογίες υψηλής πολυπλοκότητας και με μεγάλη διασπορά τιμών. Τα αποτελέσματα αυτά συνάδουν με τα θεωρητικά συμπεράσματα του paper και αποδεικνύουν ότι η προτεινόμενη υλοποίηση αποτελεί μια πρακτικά εφαρμόσιμη και αποδοτική λύση για προβλήματα δίκαιης κατανομής σε γράφους.

ΚΕΦΑΛΑΙΟ 6: Συμπεράσματα και Μελλοντικές Κατευθύνσεις

Η εκπόνηση της παρούσας πτυχιακής εργασίας αποτέλεσε μια ολοκληρωμένη διαδικασία μελέτης, σχεδίασης, υλοποίησης και αξιολόγησης ενός αλγορίθμου δίκαιης κατανομής σε γράφους. Η συνδυαστική μελέτη της βιβλιογραφίας με την πειραματική επιβεβαίωση των θεωρητικών αποτελεσμάτων έδωσε χώρο για διερεύνηση τόσο σε θεωρητικό όσο και σε πρακτικό επίπεδο.

Βασικό άξονα πάνω στον οποίο στηρίχθηκε η εργασία αποτελούν οι βασικές αρχές της θεωρίας της δίκαιης κατανομής πόρων (fair division) και ειδικότερα οι έννοιες της ζήλιας (envy), των κριτηρίων EFX (Envy-Free up to any good) και EF1 (Envy-Free up to one good). Τονίστηκε το πώς οι διάφορες μορφές δικαιοσύνης προσπαθούν να ισορροπήσουν την ατομική ικανοποίηση και τη συλλογική βελτιστοποίηση, και γιατί η EFX αποτελεί την πιο αυστηρή και απαιτητική συνθήκη.

Η μελέτη του paper “*EFX Allocations on Graphs*” των Christodoulou, Amos Fiat, Elias Koutsoupias, and Alkmini Sgouritsa (2023) συνέβαλε αποφασιστικά στην κατανόηση της θεωρητικής βάσης του προβλήματος και στην εξοικείωση με έννοιες όπως τα safe sets, οι τοπολογίες κύκλων και δέντρων, και οι συνθήκες υπό τις οποίες η EFX κατανομή είναι εγγυημένα εφικτή. Η βιβλιογραφία μου έδωσε επίσης τη δυνατότητα να αντιληφθώ τη δυσκολία της γενικής περίπτωσης (NP-hardness) και να κατανοήσω γιατί η αναζήτηση αποδοτικών, πρακτικών αλγορίθμων έχει ιδιαίτερη ερευνητική αξία.

Σε πρακτικό επίπεδο, η υλοποίηση του αλγορίθμου και η εκτέλεση εκτεταμένων πειραμάτων με οπτικοποίηση αποτελεσμάτων συνέβαλαν καθοριστικά στην καλλιέργεια δεξιοτήτων αλγοριθμικής σκέψης, προγραμματισμού σε Python, στατιστικής ανάλυσης δεδομένων και ανάπτυξης γραφικών περιβαλλόντων (GUI).

Κατά την πορεία της εργασίας, συνδύασα τη μαθηματική κατανόηση του προβλήματος με την πρακτική πειραματική επιβεβαίωση. Το γεγονός ότι σε όλα τα πειράματα —ανεξάρτητα από το πλήθος κόμβων, την πυκνότητα ή το εύρος αποτιμήσεων— ο αλγόριθμος **παρήγαγε πάντοτε EFX κατανομή**, αποτέλεσε ένδειξη τόσο της ορθότητας της υλοποίησης όσο και της αξιοπιστίας του θεωρητικού μοντέλου.

Μελλοντικές κατευθύνσεις και προτάσεις

Παρά τα θετικά αποτελέσματα, υπάρχουν πολλαπλές κατευθύνσεις που θα μπορούσαν να αποτελέσουν συνέχεια ή επέκταση της παρούσας εργασίας:

1. **Επέκταση σε πιο γενικές δομές γράφων**, όπως πολυγράφους, κατευθυνόμενους γράφους ή δυναμικά δίκτυα που μεταβάλλονται στο χρόνο. Μια τέτοια μελέτη θα επέτρεπε τη διερεύνηση της EFX ιδιότητας σε πιο σύνθετα περιβάλλοντα, με εφαρμογές σε κοινωνικά ή επικοινωνιακά δίκτυα.
2. **Εξέταση διαφορετικών μοντέλων αποτίμησης**, πέρα από τις τρεις κατανομές που χρησιμοποιήθηκαν (uniform, center, extremes). Ιδιαίτερο ενδιαφέρον θα παρουσίαζε η μελέτη μη γραμμικών ή υποπροσθετικών συναρτήσεων αξίας, όπου η EFX συνθήκη μπορεί να συμπεριφέρεται διαφορετικά.
3. **Ανάπτυξη και ενσωμάτωση νέων αλγοριθμικών προσεγγίσεων** στο υπάρχον γραφικό περιβάλλον, όπως EF1 ή MMS αλγορίθμων, ώστε να μπορούν να συγκρίνονται απευθείας οι αποδόσεις τους.
4. **Εφαρμογή σε πραγματικά δεδομένα**, όπως δίκτυα συνεργασίας, διαμοιρασμού πόρων ή κοινωνικά συστήματα, όπου η έννοια της EFX δικαιοσύνης θα μπορούσε να έχει άμεση πρακτική σημασία.

Τελική αποτίμηση

Η παρούσα πτυχιακή εργασία αποτέλεσε μια **ολοκληρωμένη εμπειρία μάθησης**, που συνδύασε τη θεωρητική έρευνα με την πρακτική εφαρμογή. Μέσα από αυτήν απέκτησα σφαιρική κατανόηση της έννοιας της δικαιοσύνης στους αλγορίθμους κατανομής, αλλά και εμπειρία στην ανάλυση, αξιολόγηση και παρουσίαση πειραματικών αποτελεσμάτων.

Η **επιβεβαίωση της EFX κατανομής σε όλα τα πειράματα** και η **απουσία unoriented ακμών** αποτελούν ισχυρή ένδειξη της σταθερότητας και πληρότητας του μοντέλου που υλοποιήθηκε. Παράλληλα, η διαδικασία με δίδαξε πώς η θεωρητική γνώση μπορεί να μετατραπεί σε **πραγματική αλγοριθμική λύση**, ικανή να αντιμετωπίζει πολύπλοκα προβλήματα δικαιοσύνης σε δίκτυα.

Η εργασία αυτή αποτέλεσε επομένως όχι μόνο ένα επιστημονικό εγχείρημα αλλά και μια σημαντική προσωπική μαθησιακή εμπειρία, που με βοήθησε να συνδυάσω τη θεωρία, τον προγραμματισμό και την ερευνητική ανάλυση σε ένα ενιαίο πλαίσιο.

ΒΙΒΛΙΟΓΡΑΦΙΑ

- Akrami, H., Rezvan, R., & Seddighin, M. (2022). *An EF2X Allocation Protocol for Restricted Additive Valuations* (No. arXiv:2202.13676). arXiv.
<https://doi.org/10.48550/arXiv.2202.13676>
- Amanatidis, G., Markakis, E., & Ntokos, A. (2020). Multiple birds with one stone: Beating $1/2$ for EFX and GMMS via envy cycle elimination. *Theoretical Computer Science*, 841, 94–109. <https://doi.org/10.1016/j.tcs.2020.07.006>
- Berendsohn, B. A., Boyadzhyska, S., & Kozma, L. (2022). *Fixed-point cycles and EFX allocations* (No. arXiv:2201.08753). arXiv.
<https://doi.org/10.48550/arXiv.2201.08753>
- Budish, E. (2011). The Combinatorial Assignment Problem: Approximate Competitive Equilibrium from Equal Incomes. *Journal of Political Economy*, 119(6), 1061–1103.
<https://doi.org/10.1086/664613>
- Caragiannis, I., Garg, J., Rathi, N., Sharma, E., & Varricchio, G. (2025). *New Fairness Concepts for Allocating Indivisible Items* (No. arXiv:2206.01710). arXiv.
<https://doi.org/10.48550/arXiv.2206.01710>
- Caragiannis, I., Kurokawa, D., Moulin, H., Procaccia, A. D., Shah, N., & Wang, J. (2019a). The Unreasonable Fairness of Maximum Nash Welfare. *ACM Transactions on Economics and Computation*, 7(3), 1–32. <https://doi.org/10.1145/3355902>
- Caragiannis, I., Kurokawa, D., Moulin, H., Procaccia, A. D., Shah, N., & Wang, J. (2019b). The Unreasonable Fairness of Maximum Nash Welfare. *ACM Transactions on Economics and Computation*, 7(3), 1–32. <https://doi.org/10.1145/3355902>
- Chaudhury, B. R., Kavitha, T., Mehlhorn, K., & Sgouritsa, A. (2020). *A Little Charity*

Guarantees Almost Envy-Freeness.

Christodoulou, G., Fiat, A., Koutsoupias, E., & Sgouritsa, A. (2023). Fair allocation in graphs. *Proceedings of the 24th ACM Conference on Economics and Computation*, 473–488. <https://doi.org/10.1145/3580507.3597764>

Christodoulou, G., & Mastrakoulis, S. (2025). *Exact and approximate maximin share allocations in multi-graphs* (No. arXiv:2506.20317). arXiv. <https://doi.org/10.48550/arXiv.2506.20317>

Foley, D. K. (1966). *RESOURCE ALLOCATION AND THE PUBLIC SECTOR - ProQuest*. <https://www.proquest.com/openview/cac1d2d0d079f04d1989726c2c2181fa/1?pq-origsite=gscholar&cbl=18750&diss=y>

Lipton, R. J., Markakis, E., Mossel, E., & Saberi, A. (2004). On approximately fair allocations of indivisible goods. *Proceedings of the 5th ACM Conference on Electronic Commerce*, 125–131. <https://doi.org/10.1145/988772.988792>

Plaut, B., & Roughgarden, T. (2018). *Almost Envy-Freeness with General Valuations*.

Sgouritsa, A., & Sotiriou, M. M. (2025). *On the existence of EFX allocations in multigraphs* (No. arXiv:2502.09777). arXiv. <https://doi.org/10.48550/arXiv.2502.09777>

Steinhaus, H. (1949). Sur la division pragmatique. *Econometrica*, 17, 315–319. <https://doi.org/10.2307/1907319>

Varian, H. R. (1973). *Equity, envy, and efficiency*.

ΠΑΡΑΡΤΗΜΑ

[allocationEFX.py](#)

```
1 allocationEFX.py
2 -----
3
4 Υλοποίηση αλγορίθμου κατανομής σε γράφους με στόχο EFX (Envy-Free up to any good).
5 Το αρχείο περιλαμβάνει:
6   • Greedy allocation (αρχική απόδοση αγαθών)
7   • Reduce envy (προσπάθεια μείωσης ζήλιας με χρήση μη αποδοθέντων αγαθών)
8   • Allocate remaining goods (απόδοση υπολοίπων με βάση safe sets)
9   • Έλεγχος EFX (is_efx)
10  • Συγκεντρωτική εκτέλεση/μέτρηση μετρικών (run_allocation)
11
12 """
13 import random
14 import io
15 import sys
16 import copy
17 import copy
18 import time
19 from inputMethods import get_input
20 from inputMethods import get_input_from_edges, get_input_from_file
21
22 def greedy_allocation(agents, goods, valuations): 6 usages
23     """
24     Greedy Allocation
25     -----
26     Δέχεται:
27     • agents: λίστα πρακτόρων (κόμβοι του γράφου)
28     • goods: λίστα "αγαθών" (ακμές, κάθε good είναι ζεύγος (u, v))
29     • valuations: λεξικό {agent: {edge: value}} με τις αποτιμήσεις
30     Επιστρέφει:
31     • allocation: λεξικό {agent: good} με ΜΙΑ ακμή ανά πράκτορα (όπου επιλέχθηκε)
32
33     Λογική:
34     - Κρατά σύνολο αδιερευνήτων πρακτόρων N και αδιάθετα αγαθά M.
35     - Επιλέγει πράκτορα i (από ουρά προτεραιότητας ή τυχαία από N).
36     - Δίνει στον i την "καλύτερη" διαθέσιμη γειτονική ακμή (μέγιστη αποτίμηση).
37     - Αντιστοίχως, προτεραιοποιεί τον «άλλο» κόμβο της ίδιας ακμής ώστε να επεξεργαστεί άμεσα.
38     """
39     N = set(agents) # unprocessed agents
40     M = set(goods) # unallocated goods
41     allocation = {} # agent -> good
42     queue = [] # priority queue for next check (προώθηση «γειτονικού» πράκτορα)
```

```

44 print(f"Greedy Allocation")
45 print(f"-----")
46
47 while N or queue:
48     if queue:
49         i = queue.pop(0) # Εξυπηρέτηση πράκτορα από «ουρά προτεραιότητας»
50     else:
51         i = random.choice(list(N)) # Τυχαίος πράκτορας από N
52
53     if i not in N:
54         continue # Ίσως επεξεργάστηκε ήδη ή έχει μετακινηθεί στην ουρά
55
56     N.remove(i)
57     if i in allocation:
58         continue # Έχει ήδη πάρει ακμή
59
60     print("Agent:", i)
61
62     # Βρες την καλύτερη διαθέσιμη γειτονική ακμή για τον i βάσει αποτίμησης v_i
63     best_good = max(
64         (e for e in M if e in goods and i in e), # μόνο αδιάθετες και γειτονικές ακμές
65         key=lambda e: valuations[i].get(e, 0),
66         default=None
67     )
68
69     # Απόδοση ακμής και προτεραιοποίηση του άλλου άκρου
70     if best_good:
71         allocation[i] = best_good # απόδωσε την ακμή στον i
72         M.remove(best_good) # Διέγραψε την από τα αδιάθετα
73         print(f" -> Gets {best_good} (value {valuations[i].get(best_good, 0)})")
74         other = best_good[0] if best_good[1] == i else best_good[1] # το «άλλο» άκρο της ακμής
75         if other not in allocation and other in N:
76             queue.insert(_index: 0, other) # βάλε τον other μπροστά στην ουρά
77     print(f"-----")
78     return allocation

```



```

81 def is_envied(agent, allocation, valuations): 17 usages
82     """
83     Επιτρέπει True αν ΟΠΟΙΟΣΔΗΠΟΤΕ άλλος πράκτορας ζηλεύει τον 'agent'.
84     Προσοχή: ο έλεγχος εδώ είναι «EF1-style» ως προς τον agent που ζηλεύεται:
85     - Υπολογίζει για κάθε other αν προτιμά το bundle του agent από το δικό του.
86     Χρησιμοποιείται ως «ένδειξη» ζήλιας στη ροή του αλγορίθμου.
87     """
88     Xi = allocation.get(agent, set())
89     if not isinstance(Xi, set):
90         Xi = {Xi}
91
92     for other in valuations:
93         if other == agent:
94             continue
95
96         Xj = allocation.get(other, set())
97         if not isinstance(Xj, set):
98             Xj = {Xj}
99
100         # value_other_has: πώς αποτιμά ο other τα δικά του
101         value_other_has = sum(valuations[other].get(g, 0) for g in Xj)
102         # value_other_wants: πώς αποτιμά ο other το bundle του agent
103         value_other_wants = sum(valuations[other].get(g, 0) for g in Xi)
104
105         if value_other_wants > value_other_has:
106             print(f"Agent {agent} is envied by {other} ({other} prefers {value_other_wants} > {value_other_has})")
107             return True
108     return False
109
110
111 def reduce_envy(allocation, valuations, goods): 6 usages
112     """
113     Προσπάθεια μείωσης ζήλιας με χρήση αδιάθετων αγαθών.
114     Βασική ιδέα:
115     - Αν βρεθεί πράκτορας i που είναι ζηλευτός (κάποιος άλλος τον ζηλεύει),
116       και το σύνολο Ui των αδιάθετων γειτονικών ακμών του i βελτιώνει την αξία του,
117       τότε αντικαθιστούμε το τρέχον bundle του i με Ui (όπου είναι δυνατό).
118     - Προσπαθούμε επίσης να μετακινήσουμε άλλους πράκτορες σε αδιάθετες ακμές που
119       βελτιώνουν την κατάστασή τους.
120     Τερματισμός όταν δεν αλλάζει τίποτα (changed == False).
121     """

```

```

122 print("Reduce Envy")
123 print(f"-----")
124 # Κανονικοποίηση bundles σε σύνολα
125 for agent in valuations:
126     if agent not in allocation:
127         allocation[agent] = set()
128     elif not isinstance(allocation[agent], set):
129         allocation[agent] = {allocation[agent]}
130
131 all_goods = set(goods)
132 allocated_goods = set(g for bundle in allocation.values() for g in bundle)
133 unallocated_goods = all_goods - allocated_goods
134
135 while True:
136     changed = False
137     for i in valuations:
138         if not is_envied(i, allocation, valuations):
139             continue
140
141         # Ui: αδιάθετες γειτονικές ακμές του i
142         Ui = [g for g in unallocated_goods if i in g]
143         vi,Ui = sum(valuations[i].get(g, 0) for g in Ui)
144         vi,Xi = sum(valuations[i].get(g, 0) for g in allocation.get(i, set()))
145
146         # Αν το Ui βελτιώνει αξία για i, αντικατάσταση bundle
147         if vi,Ui > vi,Xi:
148             print(
149                 f"\nAgent {i} is envied and prefers unallocated goods. Replacing Xi (value {vi,Xi}) with Ui(X) (value {vi,Ui})")
150             print(f"  Old allocation: {allocation.get(i, set())}")
151             print(f"  New allocation: {Ui}")
152             # Επιστροφή παλιών αγαθών στα αδιάθετα
153             for g in allocation.get(i, set()):
154                 unallocated_goods.add(g)
155             # Νέο bundle για i = Ui
156             allocation[i] = set(Ui)
157             for g in Ui:
158                 unallocated_goods.remove(g)
159             changed = True
160

```

```

161         # Προσπάθεια βελτίωσης και για άλλους j με αδιάθετες ακμές
162         for j in valuations:
163             if j == i:
164                 continue
165             Uj = [g for g in unallocated_goods if j in g]
166             vj_Xj = sum(valuations[j].get(g, 0) for g in allocation.get(j, set()))
167             for g in Uj:
168                 if valuations[j].get(g, 0) > vj_Xj:
169                     print(f" -> Agent {j} reallocated to {g} (value {valuations[j].get(g, 0)})")
170                     for old in allocation.get(j, set()):
171                         unallocated_goods.add(old)
172                         allocation[j] = {g}
173                         unallocated_goods.remove(g)
174                         changed = True
175             break
176
177         if not changed:
178             print("\nNo envy found with  $v_i u_i > v_i x_i$ . Terminating.\n")
179             break
180
181         # Επιστροφή σε «μονοστοιχείο» όπου ισχύει (αντί για set μεγέθους 1)
182         for agent in allocation:
183             if isinstance(allocation[agent], set) and len(allocation[agent]) == 1:
184                 allocation[agent] = list(allocation[agent])[0]
185
186         print(f"-----")
187
188         return allocation
189
190
191 def get_safe_set(agent, allocation, valuations, unallocated_goods): 2 usages
192     """
193     Υπολογισμός safe set για δοθέντα agent:
194     - Υποψήφιοι είναι πράκτορες 'other' που ΔΕΝ είναι ζηλευτοί και
195       ο agent, αξιολογώντας  $x_j$  υ  $u_i$ , δεν θα «υστερεί» σε αξία από το δικό του  $x_i$ .
196     - Επιστρέφει σύνολο πρακτόρων που είναι «ασφαλείς» για ανάθεση υπολοίπων αγαθών.
197     """

```

```

198     safe_set = set()
199     Xi = allocation.get(agent)
200     if not isinstance(Xi, set):
201         Xi = {Xi}
202     vi_Xi = sum(valuations[agent].get(g, 0) for g in Xi) # αξία τρέχοντος bundle του agent
203     Ui = {g for g in unallocated_goods if agent in g} # αδιάθετες γειτονικές ακμές του agent
204
205     for other in allocation:
206         if other == agent:
207             continue
208         Xj = allocation.get(other)
209         if not isinstance(Xj, set):
210             Xj = {Xj}
211         Xj_union_Ui = Xj.union(Ui)
212         vi_Xj_union_Ui = sum(valuations[agent].get(g, 0) for g in Xj_union_Ui)
213
214         # Αν ο 'other' δεν είναι ζηλευτός ΚΑΙ ο agent δεν «χάνει» σε αξία έναντι Xj ∪ Ui,
215         # τότε ο other είναι safe για τον agent
216         if not is_envied(other, allocation, valuations):
217             if vi_Xi >= vi_Xj_union_Ui:
218                 safe_set.add(other)
219
220     return safe_set
221
222
223 def allocate_remaining_goods(allocation, goods, valuations): 6 usages
224     """
225     Απόδοση των υπόλοιπων (αδιάθετων) ακμών:
226     1) Αν ένας από τους δύο άκρους (a ή b) δεν είναι ζηλευτός, δώσε του την ακμή.
227     2) Αν και οι δύο είναι ζηλευτοί, υπολόγισε τα safe sets και δώσε την ακμή
228         σε τυχαίο «ασφαλή» πράκτορα από την τομή τους.
229     Επιστρέφει:
230     • νέο allocation
231     • count unoriented (μετρητής περιπτώσεων «και οι δύο ζηλευτοί» που απαιτούν safe-set επιλογή)
232     """
233     unoriented_count = 0
234     print(f"Allocate Remaining Goods")
235     print(f"-----")
236     all_goods = set(goods)

```

```

237     allocated_goods = set(
238         g for bundle in allocation.values()
239         for g in (bundle if isinstance(bundle, set) else {bundle})
240     )
241     unallocated_goods = all_goods - allocated_goods
242
243     for good in list(unallocated_goods):
244         a, b = good # τα άκρα της ακμής
245
246         # Έλεγχος αν οι άκρες είναι ζηλευτές στη τρέχουσα ανάθεση
247         a_envied = is_envied(a, allocation, valuations)
248         b_envied = is_envied(b, allocation, valuations)
249         print(f"\n{good}:")
250         print(f"  Agent {a} envied: {a_envied}")
251         print(f"  Agent {b} envied: {b_envied}")
252
253         if not a_envied:
254             # Περίπτωση 1: δώσε στον a (μη ζηλευτό)
255             allocation[a] = allocation.get(a, set()) | {good} if isinstance(allocation.get(a), set) else {
256                 allocation.get(a), good}
257             print(f"Unallocated good {good} given to non-envied agent {a}")
258             unallocated_goods.remove(good)
259         elif not b_envied:
260             # Περίπτωση 2: δώσε στον b (μη ζηλευτό)
261             allocation[b] = allocation.get(b, set()) | {good} if isinstance(allocation.get(b), set) else {
262                 allocation.get(b), good}
263             print(f"Unallocated good {good} given to non-envied agent {b}")
264             unallocated_goods.remove(good)
265         else:
266             # Περίπτωση 3: και οι δύο ζηλευτοί → υπολογισμός safe sets & επιλογή από την τομή
267             print(f"  [Safe Set Check] both agents envied")
268             safe_a = get_safe_set(a, allocation, valuations, unallocated_goods)
269             safe_b = get_safe_set(b, allocation, valuations, unallocated_goods)
270             safe_agents = safe_a.intersection(safe_b)
271             print(f"  Safe agents among {'{a}', '{b}'}: {safe_agents}")

```

```

273         if safe_agents:
274             unoriented_count += 1
275             chosen_agent = random.choice(list(safe_agents)) # αυθαίρετη (τυχαία) επιλογή από safe agents
276             allocation[chosen_agent] = allocation.get(chosen_agent, set()) | {good} if isinstance(
277                 allocation.get(chosen_agent), set) else {allocation.get(chosen_agent), good}
278             print(f"Unallocated good {good} given to safe agent {chosen_agent}")
279             unallocated_goods.remove(good)
280         else:
281             # Ιδεατό δεν θα πρέπει να συμβεί (με βάση την θεωρία/ροή)
282             print(f" ERROR: No safe agent found (should not happen)")
283     print(f"-----")
284     return allocation, unoriented_count
285
286
287 def is_efx(allocation, valuations, verbose=False, eps=1e-12): 4 usages
288     """
289     Έλεγχος EFX:
290     Για κάθε  $i \neq j$  και κάθε  $e \in X_j$  με  $v_i(e) > 0$  ισχύει:
291      $v_i(X_i) \geq v_i(X_j \setminus \{e\})$ 
292     Ισοδύναμα:
293      $v_i(X_i) \geq v_i(X_j) - \min_{\{e \in X_j, v_i(e) > 0\}} v_i(e)$ 
294     Επιστρέφει (True/False, μήνυμα/log)
295     """
296     # Κανονικοποίηση bundles σε σύνολα
297     bundles = {a: (set(B) if isinstance(B, set) else ({B} if B else set()))
298                 for a, B in allocation.items()}
299     agents = list(bundles.keys())
300
301     # Προϋπολογισμός  $v_i(X_i)$ 
302     vi_self = {i: sum(valuations[i].get(g, 0.0) for g in bundles[i]) for i in agents}
303     log = []

```

```

305     for i in agents:
306         for j in agents:
307             if i == j:
308                 continue
309             Xj = bundles[j]
310             if not Xj:
311                 continue
312             # Υπολογισμός v_i(X_j) και του min^+ (ελάχιστο θετικό v_i(e))
313             S = 0.0
314             min_pos = None
315             min_g = None
316             for g in Xj:
317                 v = valuations[i].get(g, 0.0)
318                 S += v
319                 if v > eps and (min_pos is None or v < min_pos):
320                     min_pos, min_g = v, g
321             # Αν δεν υπάρχει αγαθό με θετική αξία για τον i στο Xj, η συνθήκη είναι τετριμμένη
322             if min_pos is None:
323                 continue
324             rhs = S - min_pos
325             if rhs > vi_self[i] + eps:
326                 if verbose:
327                     log.append(f"EFX violation (i={i}, j={j}): v_i(X_i)={vi_self[i]:.6f} "
328                               f"< v_i(X_j)-min^+={rhs:.6f} (min at {min_g}={min_pos:.6f}, v_i(X_j)={S:.6f})")
329                     return False, "\n".join(log)
330             return False, (f"EFX violation for (i={i}, j={j}); best-removal for i is {min_g}")
331         if verbose:
332             log.append("✅ EFX holds.")
333             return True, "\n".join(log)
334     return True, "✅ EFX holds."
335
336
337 def run_allocation(agents, goods, valuations, show_details=False): 8 usages
338     """
339     Κεντρική ροή εκτέλεσης:
340     1) Greedy allocation
341     2) Reduce Envy
342     3) Allocate Remaining Goods
343     + Μετρήσεις: χρόνος, envy ανά φάση, τελικό EFX, unoriented count, total/optimal/ratio.
344     Αν show_details=True, επιστρέφει επιπλέον λεπτομερές log εκτύπωσης.
345     """

```

```

346     if show_details:
347         buffer = io.StringIO()
348         sys_stdout = sys.stdout
349         sys.stdout = buffer
350
351     start_t = time.perf_counter()
352
353     total_agents = len(agents)
354
355     # A1: Greedy
356     greedy_alloc = greedy_allocation(agents, goods, valuations)
357     envied_greedy = sum(1 for a in agents if is_envied(a, greedy_alloc, valuations))
358     nonenvied_greedy = total_agents - envied_greedy
359     pct_envied_greedy = (envied_greedy / total_agents * 100.0) if total_agents else 0.0
360     pct_nonenvied_greedy = 100.0 - pct_envied_greedy
361
362     # A2: Reduce Envy
363     envy_reduced_input = copy.deepcopy(greedy_alloc)
364     envy_reduced_alloc = reduce_envy(envy_reduced_input, valuations, goods)
365     envied_reduce = sum(1 for a in agents if is_envied(a, envy_reduced_alloc, valuations))
366     nonenvied_reduce = total_agents - envied_reduce
367     pct_envied_reduce = (envied_reduce / total_agents * 100.0) if total_agents else 0.0
368     pct_nonenvied_reduce = 100.0 - pct_envied_reduce
369
370     # A3: Allocate Remaining
371     before_final_alloc = copy.deepcopy(envy_reduced_alloc)
372     final_alloc, unoriented_count = allocate_remaining_goods(before_final_alloc, goods, valuations)
373
374     elapsed_ms = (time.perf_counter() - start_t) * 1000.0
375
376     # EFX check (τελικός έλεγχος)
377     efx_ok, efx_log = is_efx(final_alloc, valuations)
378     print(efx_log)
379
380     # --- Helper για bundle size ---
381     def _bundle_size(b):
382         if b is None:
383             return 0
384         if isinstance(b, set):
385             return len(b)
386         return 1

```



```

387
388 # Πράκτορες χωρίς ακμές στο τέλος
389 zero_edge_agents = sum(
390     1 for a in agents
391     if a not in final_alloc or _bundle_size(final_alloc.get(a)) == 0
392 )
393 pct_zero_edge = (zero_edge_agents / total_agents * 100.0) if total_agents else 0.0
394
395 # Envied στο τέλος
396 envied_final = sum(1 for a in agents if is_envied(a, final_alloc, valuations))
397 nonenvied_final = total_agents - envied_final
398 pct_envied_final = (envied_final / total_agents * 100.0) if total_agents else 0.0
399 pct_nonenvied_final = 100.0 - pct_envied_final
400
401 # --- Metrics text ---
402 metrics_text = (
403     "\n📊 Run Metrics\n"
404     "-----\n"
405     f"⌚ Time elapsed: {elapsed_ms:.2f} ms\n"
406     f"👤 Agents with no edge (final): {zero_edge_agents}/{total_agents} ({pct_zero_edge:.1f}%) \n\n"
407     f"👥 Envied agents per phase (count / % envied | % non-envied)\n"
408     f"  • Greedy:      {envied_greedy}/{total_agents} ({pct_envied_greedy:.1f}% | {pct_nonenvied_greedy:.1f}%) \n"
409     f"  • Reduce Envy: {envied_reduce}/{total_agents} ({pct_envied_reduce:.1f}% | {pct_nonenvied_reduce:.1f}%) \n"
410     f"  • Final:       {envied_final}/{total_agents} ({pct_envied_final:.1f}% | {pct_nonenvied_final:.1f}%) \n"
411 )
412
413 # --- Τελική κατανομή & συνολική αξία ---
414 final_text = metrics_text + "\n🔴 Final Allocation:\n"
415 total_value = 0
416 for agent, bundle in final_alloc.items():
417     if not isinstance(bundle, set):
418         bundle = {bundle}
419     value = sum(valuations.get(agent, {}).get(g, 0) for g in bundle)
420     final_text += f"  {agent}: {bundle} (value: {value}) \n"
421     total_value += value
422
423 # --- Optimal sum ---
424 # Προσοχή: εδώ ο "βέλτιστος" ορίζεται ως άθροισμα  $\max(v_u(e), v_v(e))$  ανά ακμή  $e=(u,v)$ ,
425 # δηλαδή η καλύτερη δυνατή «τοπική» ανάθεση ανά ακμή, ανεξάρτητα από παγκόσμιους περιορισμούς.
426 optimal_sum = 0

```

```

427     for e in goods:
428         if not (isinstance(e, (tuple, list)) and len(e) == 2):
429             continue
430         u, v = e
431         vu = valuations.get(u, {}).get(e, 0)
432         vv = valuations.get(v, {}).get(e, 0)
433         optimal_sum += max(vu, vv)
434
435     ratio_vs_opt = (total_value / optimal_sum * 100.0) if optimal_sum else 0.0
436
437     # --- Unoriented (count & %) ---
438     total_goods = len(goods)
439     pct_unoriented = (unoriented_count / total_goods * 100.0) if total_goods else 0.0
440
441     final_text += (
442         "\n🔵 Orientation & Optimality\n"
443         "-----\n"
444         f"📊 Σύνολο αγαθών (edges): {total_goods}\n\n"
445         f"🕒 Unoriented goods: {unoriented_count} / {total_goods} ({pct_unoriented:.1f}%) \n\n"
446         f"👤 Total value of allocation: {total_value}\n"
447         f"🧮 Optimal sum ( $\sum \max(u_i(e), u_j(e))$ ): {optimal_sum}\n"
448         f"📊 Αναλογία (Our/Optimal): {ratio_vs_opt:.1f}%\n"
449     )
450
451     if show_details:
452         print(final_text)
453         sys.stdout = sys_stdout
454         return final_alloc, buffer.getvalue(), greedy_alloc, envy_reduced_alloc
455     else:
456         return final_alloc, final_text, greedy_alloc, envy_reduced_alloc

```